SMARTBillHub

Step 1 · measured at 2.261.3 · fbd1ee00

THE READ

00 Scope & surfaces
01 The split you ruled
02 The four doors
03 All sixty-one

CONNECTOR SCREENS · 03

THE GRID

CON-01 The grid

CONNECTORS · 28 × 2

CFG-01 Microsoft 365
VIEW-01 Microsoft 365
CFG-02 Google Workspace
VIEW-02 Google Workspace
CFG-03 Custom SMTP
VIEW-03 Custom SMTP
CFG-04 Bulk email service
VIEW-04 Bulk email service
CFG-05 Twilio
VIEW-05 Twilio
CFG-06 Neptune 360
VIEW-06 Neptune 360
CFG-07 Badger BEACON
VIEW-07 Badger BEACON
CFG-08 Itron
VIEW-08 Itron
CFG-09 Sensus
VIEW-09 Sensus
CFG-10 InvoiceCloud
VIEW-10 InvoiceCloud
CFG-11 Paymentus

SMARTBILLHUB · STEP 1 · MEASURE BEFORE YOU BUILD

Integration Blueprint

Is integrations a connector, does runbooks carry the business logic, and where do they overlap? The line is drawn, and drawn well. Integrations hands over at four points; three are built and deliberately shut, the fourth is built and held by a switch. What is missing is not the line — it is that nothing crosses it in either direction.

MEASURED AT	COMMIT	YOUR SCREEN	SCREENS DRAWN	FRAMES READ	BACKEND RUN	BROWSER RUN
2.261.3	fbd1ee00	2.253.0	75	41 + 75	Yes	No

THIRD PASS — AND THE FIRST TWO WERE DRAWN FROM THE WRONG SOURCE

Passes 1 and 2 drew these screens from the wireframe pack. That is the layout authority, but it is not what shipped. Every drawing below is now rebuilt from the component source, with the pack shown beside it as the specification. Three drawings changed materially — the access log is not on the keys screen at all, the mappings screen has pills rather than tabs, and the runbooks dashboard shows em-dashes where the pack shows numbers.

00 Scope, surfaces, and what I left out

In: apps/integrations (18 manifests, 8 connectors) and its four screens; apps/runbooks (28 definitions) and its seven screens; the six runbooks that read integrations; the seven vendors you named; and the boundary between the two.

Out, deliberately: the other 22 runbook definitions, which you said you would handle separately. Notifications, payments, metering and ledger except where a connector crosses into them. The customer portal entirely. Say so if that boundary is

VIEW-11 Paymentus
CFG-12 US Payments
VIEW-12 US Payments
CFG-13 WorldPay
VIEW-13 WorldPay
CFG-14 Global Payments
VIEW-14 Global Payments
CFG-15 Payment kiosk network
VIEW-15 Payment kiosk network
CFG-16 Test payments (sandbox)
VIEW-16 Test payments (sandbox)
CFG-17 Aclarian
VIEW-17 Aclarian
CFG-18 Workday
VIEW-18 Workday
CFG-19 SAP
VIEW-19 SAP
CFG-20 Infor
VIEW-20 Infor
CFG-21 Microsoft Dynamics GP
VIEW-21 Microsoft Dynamics GP
CFG-22 Accounting export file
VIEW-22 Accounting export file
CFG-23 Bank & lockbox file
VIEW-23 Bank & lockbox file
CFG-24 Print vendor
VIEW-24 Print vendor
CFG-25 Esri ArcGIS
VIEW-25 Esri ArcGIS
CFG-26 Census geocoder
VIEW-26 Census geocoder
CFG-27 Public map tiles
VIEW-27 Public map tiles
CFG-28 Single sign-on
VIEW-28 Single sign-on

wrong.

"I measured it" means five different things. This says which I did.

SURFACE	WHAT MEASURING IT MEANS	DID I?
An API / connector	Called for real, response read	**Yes** — 6 real messages
A job	Run, and you read what it wrote	**Registry inspected, not fired**
A screen	Clicked, in a browser, at the real address	**No** — no browser here
A money path	Run on real PostgreSQL	**No** — no PostgreSQL here
A portal page	Opened at 390px as well as 1280	Out of scope

> **The honest limit**
>
> No PostgreSQL and no front-end build in this container, so no screen was clicked. Every drawing is read out of component source. The one screen I could check against your screenshot immediately showed three things reading had not — so treat every other drawing as carrying that same risk.

01 The split you ruled, quoted

"business logic , scheduling , processing we will take it in run book"

PO ruling, 2026-09-06 — the sentence that moved scheduling

Integrations — how do I reach them, and may I?

- Who the vendor is — name and endpoint
- Their keys, where we hold them
- Our key, where they hold it
- Payload shape, records per call

Runbooks — what do I do with it?

- When it runs — schedule, one-off, their call
- The data check, before anything
- What their success and failure codes mean
- What the data means — ACHCHECK → Bank transfer

MAPPINGS · KEYS · LOG

- MAP-01 Mappings
- MAP-02 Mappings · yours
- KEY-01 Issue a key
- ACC-01 Access log

RUNBOOK SCREENS · RB

- RB Twenty-three, on four tabs

TAB 1 · RUNBOOKS

- RBS-003 The list
- RBS-004 Setup · Overview
  - RBS-004 ▸ Steps Setup · Steps
  - RBS-004 ▸ Scope Setup · Scope
  - RBS-004 ▸ Guards Setup · Guards
  - RBS-004 ▸ Limits Setup · Limits
  - RBS-004 ▸ Runs Setup · Runs
  - RBS-004 ▸ Activity Setup · Activity

RUN WIZARD · 8 FRAMES

- RBS-005.1 1 Scope
- RBS-005.2 2 Checking
- RBS-005.3 3 Review
- RBS-005.4 4 Approve
- RBS-005.5 5 Applying
- RBS-005.6 6 Done
- RBS-005.7 ✗ Cap refused
- RBS-005.8 ✗ Step failed

TAB 2 · TEMPLATES · THE PATH

- RBS-002.1 1 Templates
- RBS-002.2 2 Review it
- RBS-002.3 3 Add it
- RBS-004 ▸ Just added 4 Set it up

TAB 3 · WORK LIST

- RBS-007 Work list

TAB 4 · RUN HISTORY

- RBS-008 Run history

- Payload shape · records per call
- Test the connection
- On / off for this vendor
- Where they may call us from, how often
- The queue and the worker — plumbing

- What the data means — ACHCHECK → Bank transfer
- Walking records one at a time
- Writing when conditions map
- Recording why a record failed
- The result, and closing the batch

> **The test that settles every argument**
>
> **Purpose, not shape.**
>
> A lookup table looks like configuration and a status code looks like a vendor property — you moved both to Runbooks anyway, because the question each answers is *what does this record mean*.

---

02 The four doors

Verified at runtime. Each empty return has a test that goes red the day a runbook plugs in.

DIRECTION	CARD SAYS	RUNNER	HANDS OVER TO	RETURNS	STATE
in · us	We collect	`runners/fetch.py`	`seams/runbooks.project_readings()`	`()`	**Built · shut**
out · us	We send	`runners/deliver.py`	`seams/notifications.py:265`	a real result	**Built · works**
in · them	They send	`inbound.py`	`inbound.runbooks_apply()`	`None`	**Built · shut**
out · them	They read	`runners/serve.py`	`serve.partner_fields()`	`{}`	**Built · shut**

> **Read the three shut doors as good news**
>
> Each empty return is a deliberate, documented, tested absence — not a forgotten stub. And the four words on the cards are your own:
> **We collect · We send · They send · They read**

RBS-006 Run result

BUILDING IT

B 83 drawings, 14 files

THE READING

F Every field

04 Every table

05 Detail · add · edit

06 Against the UI rules

07 Every crossing

FINDINGS

08 Your seven

09 What I ran

10 Routed

11 Stated absences

12 Not filed

ANSWERS

OP Many APIs, one connector

NM Naming our APIs

CR Integrations → Connectors

S Specification register

T What changed

13 What to do first

> , live in the catalogue at `integrations.kicker.*`.

A connector that goes both ways: two tags, never a fifth

Your question, and it has a signed answer already. **BR-INT-028 puts direction and initiative on the OPERATION, not on the connector** — never classify a vendor, classify a call. So a connector is not *in* a quadrant. It is the union of the quadrants its APIs are in, and the card is reporting that union.

Which rules out the fifth tag on its own. **Both ways** would be one word standing where two facts were, and it destroys exactly the information the four exist to carry: a reader could no longer tell *we send and they send* from *we collect and they read*. That is the same trade S8-1 already refused one line down, where a card with three operations shows no aggregate status word rather than averaging a broken one into a green one.

THE QUESTION	THE ANSWER	WHY
A fifth tag — **Both ways**	**No**	One word replacing two facts. It cannot say which two ways, and the four words stop being a closed set.
Two tags, in the doors' own order	**Yes**	Honest, bounded, and it reads: **Email · We send · They send**. Three of the 28 need it.
Show only the main one	**No**	There is no main one. A bounce that never arrives is as broken as a bill that never sends.
Does the doors table grow a fifth row?	**No — it stays four**	Those four rows are **runners**, one per door. A connector spanning two doors uses two runners. A vendor is not a door.

So the cards carry the words the connector actually has, capped by the four that exist. **Nine of the twenty-eight span two doors**, and each is drawn that way on both of its screens. The other nineteen show one word.

CONNECTOR	SPANS	THE OPERATION THAT MAKES IT TWO-WAY
Bulk email service	We send · They send	**Delivery feedback** — they post a bounce to us
Twilio	We send · They send	**Delivery status** — they post the outcome to us
Test payments (sandbox)	We send · They send	**Payment confirmed** — they post the confirmation to us

InvoiceCloud · Paymentus · US Payments · WorldPay · Global Payments	We collect · We send	**Settlement report** — a nightly file *we* fetch, beside three calls we push
Payment kiosk network	They send · They read	**Account lookup** — they ask us what an account owes, so the data goes out

And the tag is computed, never declared. Each card reads the direction off its own operation list rather than carrying a field beside it. A hand-set word next to a list of calls is two sources for one fact, which is exactly how the forms and the view screens drifted apart in CR-3 — so this one is closed by construction rather than by care.

And the filter follows the same rule. The direction facet on `+ Filter ▾` matches on ANY operation, so a two-way connector appears under both words rather than under a category of its own. That is the part that goes quietly wrong if nobody writes it down: a facet that matched the card's first word would hide Twilio from everyone looking for what calls us.

> **CR-8** **Three drawn rows said We send where the rule says They send**
>
> CORRECTED, AND ONE MORE WORTH A LOOK
>
> The API tables carried **Delivery feedback**, **Delivery status** and **Payment confirmed** as *We send* with a trigger of *They call us* — which is the two halves of the same row disagreeing. A webhook the vendor posts to us is **in · them**, and this document's own OP section says so in as many words: an email provider is *we send* for the messages we hand it and *they send* for the bounce it reports back.
>
> Corrected on all six screens that draw them, and it is what produced the two-tag cards above — the three connectors only span two doors once their inbound row is labelled correctly.
>
> **WAS: one more left alone, as a judgment rather than a slip** — the five payment processors carrying **Settlement report · We send · Nightly**. **NOW: changed.** You asked for every tag to be right, so it is *We collect*: a nightly settlement file is one we fetch. See CR-9, which also found a second.
>
> `BR-INT-028 — direction and initiative are declared per operation, not per connector`
>
> `Before — 3 rows read We send · They call us · After — They send · They call us`

> **CR-9** **Two more were wrong, and the tag is now computed rather than declared**

ALL 56 SCREENS RE-TAGGED

Reading every operation on all 28 rather than trusting the word beside it found two more, and they are opposite mistakes.

Settlement report, on all five payment processors. It read *We send*. A nightly settlement file is one **we fetch**, so it is *We collect* — and that makes each processor two-way, which none of them showed.

Account lookup, on the payment kiosk network. It read *They send*, along with its two neighbours. But a kiosk asking *what does this account owe* is them initiating and the data going **out** from us — which is *They read*, the fourth door, and the kiosk was the one connector on the grid never drawn as touching it.

The structural half matters more than either. A connector no longer carries a direction field at all: each card and each header reads the union off its own operation list. A hand-set word beside a list of calls is two sources for one fact, and this document already has a section on what that costs (CR-3). The drift is now impossible rather than merely corrected.

```
Changed — 5 rows Settlement report: We send → We collect · 1 row Account lookup: They send → They read
```

```
Two-way connectors: 3 → 9 of 28, recomputed rather than counted by hand
```

```
dirs() computes the union in the doors' seed order; the declared field is gone
```

03 The connector screens

Sixty-one screens — the connectors grid, then a set-up form and a connection view for each of the 28 connectors, then mappings (twice), keys and the access log. Every API on every connector is named, on both of its screens, from one list — 86 APIs in all. Each is drawn at 1440 desktop, scaled as a unit and never reflowed, in the product's own tokens — `#ec3013` accent, Archivo 800, square corners at every size. Every card and group is the kit's Panel: 1px divider border, 16px padding, and `border-t-2 border-t-accent` where a panel leads.

One drawing per screen — the old ones are gone

This pack now holds exactly one picture of each screen, and it is the one to build. Where an as-built drawing and a revised drawing both existed, the as-built drawing has been removed: the connectors grid, the connect modal, the generic connection detail, the mappings screen, the keys screen and the access-log panel each appear once.

What the as-built state was is still recorded — in the findings, in words, with file and line. That is the right place for it: a builder reading a drawing should never have to work out which of two pictures is current, and a reviewer asking *what does it do today* should get a sentence and a citation rather than a second picture.

Counted: 1 grid + 28 set-up forms + 28 connection views + mappings ×2 + keys + access log = 61. The twenty-three runbook drawings are section RB, on four tabs, because they are a different product.

These 28 pairs are ONE page each — read section B before you build

The set-up form is drawn 28 times and the connection view 28 times. That is **2 pages, not 56**. Both render from the manifest, and a new vendor must cost no new screen — BR-INT-037. The drawings exist to prove one page renders all 28 correctly. Section B has the whole mapping.

Which ids are real, and which this document minted

Real register ids:

S8 (now drawn as CON-01), S143 (now MAP-01), S144 (now KEY-01) and RBS-001…008.

This document's own labels, not register ids:

CON-01, CFG-01…28, VIEW-01…28, MAP-01, KEY-01, ACC-01. That is deliberate — an id is taken from the register through the allocator, which asks the remote, and minting S-numbers by hand in a document is how two lanes take the same number. When these are commissioned they get real ids then, and the mapping is one line per screen.

Three screens were retired rather than renumbered:

S141, the connect modal — the set-up forms replace it; S142, the generic connection detail — the 28 connection views replace it; and the as-built connectors grid — CON-01 replaces it.

Every API row says where it came from

Three marks, and the endpoint column obeys them. A hostname like `api.sap.example` reads as a fact and somebody will build against it, so a row nobody has verified does not get to carry one.

MARK	MEANS	ENDPOINT DRAWN AS	ROWS TODAY
none	The connector ships at 2.261.3, so its operations are declared on a manifest in the repo	its own host	**31** on 15 connectors
PROPOSED	A shape copied from the nearest existing manifest. Nothing is declared anywhere yet	`api.vendor.example`, always	**47** on 13 connectors
Unknown	We know the call must exist and nobody has read the vendor's docs	a dash	**0** — no row is in this state today, which is worth saying rather than leaving blank

Seven invented hostnames went with it: `api.sap.example`, `api.workday.example`, `api.invoicecloud.example`, `api.paymentus.example`, `api.itron.example`, `api.sensus.example` and `beacon.badgermeter.example` are all stand-ins now. The real ones arrive with the manifest, in the database, not here.

THE PACK SPECIFIES (FRAME 47.1 + TEMPLATE I1)

- Settings is a left vertical sub-nav
- Six chips: ALL · EMAIL · BANKING · ACCOUNTING · METERS · PAYMENTS
- No search box
- Each card carries one full-width button
- One status word per card, detail inline: Working · billing@•••.example
- Footnote: *Nine cards, one manifest each.*

WHAT SHIPS

- Settings is a horizontal tab strip, 9 tabs
- Ten chips, generated from the registry
- A search box, "Search services..."
- Buttons are small, left-aligned; a link row sits under them
- A card with more than one operation shows no card-level status at all — one line per operation instead
- Footnote: *18 services you can connect.*

S8-1 A multi-operation card has no status word — and the biggest one has sixteen lines

DESIGN CONSEQUENCE

The pack draws one state word per card. The build instead lists every operation with its own state and deliberately shows no aggregate — correctly, because averaging three operations into one word would hide a broken one.

But Neptune declares sixteen operations. On a three-across grid that card becomes a sixteen-row list while its neighbours are four lines tall. Nothing in the pack anticipates this, because when the pack was drawn the largest connector had three. Worth a decision before a utility ever connects it — and it is exactly the kind of thing a browser pass would have shown in a second and reading cannot.

> `IntegrationsTab.tsx:296-307 — operation list when operations.length > 1, else a single StateWord`

> `manifests/meters_neptune360.py — 16 operations declared`

S8-2 Settings is tabs in the product and a left rail in the specification

ROUTE: DEFECT

The layout document says it in words, not only in the frame: "StaffShell → the Settings left nav". The product has no vertical sub-nav anywhere — a tab strip ported from the earlier r10 prototype, whose own comment says so.

Two signed sources disagree and neither records it. Ambiguous is a defect, not a change — filing our misunderstanding as a "change" converts our error into your new requirement. Likely outcome: the document is amended to match a product that has shipped this way for months.

> `SettingsPage.tsx:440 — class="settings-tabs" role="tablist"`

> `grep aside|sub-nav|SideNav|vertical in that file — no hits`

S8-3 The provider cards do not use the kit's Panel — they are a hand-rolled copy of it

ROUTE: DEFECT

The staff kit has a Panel component: 1px border-divider, surface fill, 16px padding, five tones, and an accent prop that draws a 2px --color-accent rule across the top. It is used in 81 feature files. IntegrationsTab is not one of them — it writes its own `<article class="flex flex-col gap-3 border border-neutral-200 bg-bg p-4">`.

The integrations component inventory instructs the opposite in as many words — "Compose the staff kit; do not fork it." — and then specifies the card as a hand-rolled recipe, so the fork is specified rather than accidental. The cost is small and real: the card's border is neutral-200 where every other panel in

the product is divider, so two boxes that should match do not.

CON-01 draws the cards in it.

> `IntegrationsTab.tsx:271-276 — the hand-rolled article; no Panel import`
>
> `components/Panel.tsx:233, :289 — const ACCENT = "border-t-2 border-t-accent", accent?: boolean`
>
> `81 feature files import Panel; <Panel accent> is live in tenancy, rating, customers and work`

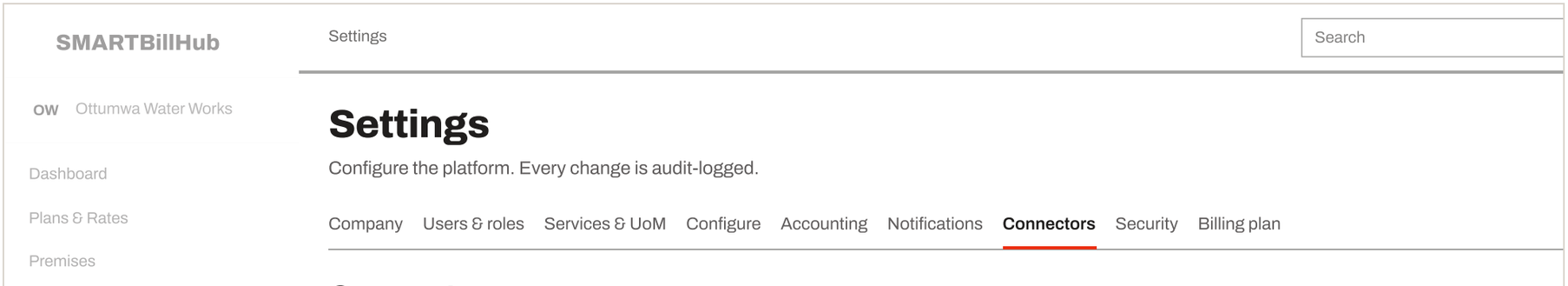
A change request — the word and the list both move

"Integrations" becomes "Connectors", and every connector carries its vendor's real name. The product ships generic names on purpose today — Water meter cloud, Card processor, Third-party GL, City GIS — and one card, Itron / Sensus, carries two vendors at once. This is a change, not a defect: the product does what it promised, and the promise is changing.

The list goes from 18 to 28. Ten ship as they are, two are renamed (Neptune 360, Esri ArcGIS), two are split out of one (Itron and Sensus), eleven are new — five payment processors and five ERPs you named, plus Badger — and three keep a category name because you have not named a vendor for them.

The as-built grid above stays exactly as it is, because that is what ships. What follows is the proposal: the new grid, then every connector twice — set it up, then live with it. Fields for the eleven new ones are a proposed shape, taken from the nearest existing manifest; fields for the rest are read from their own.

CON-01 **Settings → Connectors** `/:tenant/settings/connectors` THE ONE CONNECTORS SCREEN



Connectors

Every connector is configured and ready. Add your keys to switch one on.

① Search connectors...

② + Filter

④ All | Email | Messaging | Meters | Payments | ERP | Accounting | Banking | Print | Maps | Sign-in

EMAIL · WE SEND

Microsoft 365

Send bills from your own mailbox.

- Needs a token

1 API

Connect

EMAIL · WE SEND

Google Workspace

Send bills from your own mailbox.

- Needs a token

1 API

Connect

EMAIL · WE SEND

Custom SMTP

Any mail server with a host and a pas

- Not working

1 API · 1 working

the mail server is not answering

Production · billing@example.invalid

View | Fix it

EMAIL · WE SEND · THEY SEND

NEEDS A VENDOR NAME

Bulk email service

Hand a large batch to a sending service.

- Needs a token

2 APIs

Connect

MESSAGING · WE SEND · THEY SEND

Twilio

Send text messages to customers.

- Coming soon · v1.1

2 APIs

Connect

METERS · WE COLLECT

RENAMED

Neptune 360

Collect readings and keep the meter l

- Not working

16 APIs · 14 working

Meter alerts is refusing

Production · 3 of 3 running

View | Set up

METERS · WE COLLECT

NEW CONNECTOR

Badger BEACON

Collect readings from the BEACON head-end.

- Needs keys

4 APIs

Connect

METERS · WE COLLECT

NEW · SPLIT OUT

Itron

Collect readings from the Itron head-end.

- Needs keys

4 APIs

Connect

METERS · WE COLLECT

NEW · SPLIT OUT

Sensus

Collect readings from the Sensus hea

- Needs keys

4 APIs

Connect

PAYMENTS · WE COLLECT · WE SEND

NEW CONNECTOR

InvoiceCloud

PAYMENTS · WE COLLECT · WE SEND

NEW CONNECTOR

Paymentus

PAYMENTS · WE COLLECT · WE SE

NEW CONNECTOR

US Payments

Take card and bank payments through InvoiceCloud.

- Needs a token

4 APIs

Connect

Take card and bank payments through Paymentus.

- Needs a token

4 APIs

Connect

Take card and bank payments throug

- Needs a token

4 APIs

Connect

PAYMENTS · WE COLLECT · WE SEND

NEW CONNECTOR

WorldPay

Take card and bank payments through WorldPay.

- Needs a token

4 APIs

Connect

PAYMENTS · WE COLLECT · WE SEND

NEW CONNECTOR

Global Payments

Take card and bank payments through Global Payments.

- Needs a token

4 APIs

Connect

PAYMENTS · THEY SEND · THEY RE

Payment kiosk network

Take payments at a machine, and he

- Working

3 APIs · 3 working

Production · 3 of 3 running

View | Set up

Mappings | Keys we issue

PAYMENTS · WE SEND · THEY SEND

Test payments (sandbox)

A test processor for the whole payment journey.

- Working

3 APIs · 3 working

Sandbox

View | Set up

Mappings

ERP · THEY READ

NEW CONNECTOR

Aclarian

Your finance system collects payments and bills from here.

- Reading from us

3 APIs · 2 working

Read customers is refusing

06:14 today

View | Set up

Mappings | Keys we issue

ERP · THEY READ

NEW CONNECTOR

Workday

Your finance system collects paymer

- Needs keys

3 APIs

Connect

ERP · THEY READ

NEW CONNECTOR

SAP

Your finance system collects payments and bills from here.

- Needs keys

3 APIs

Connect

ERP · THEY READ

NEW CONNECTOR

Infor

Your finance system collects payments and bills from here.

- Needs keys

3 APIs

Connect

ERP · THEY READ

NEW CONNECTOR

Microsoft Dynamics GP

Your finance system collects paymer

- Needs keys

3 APIs

Connect

ACCOUNTING · WE SEND

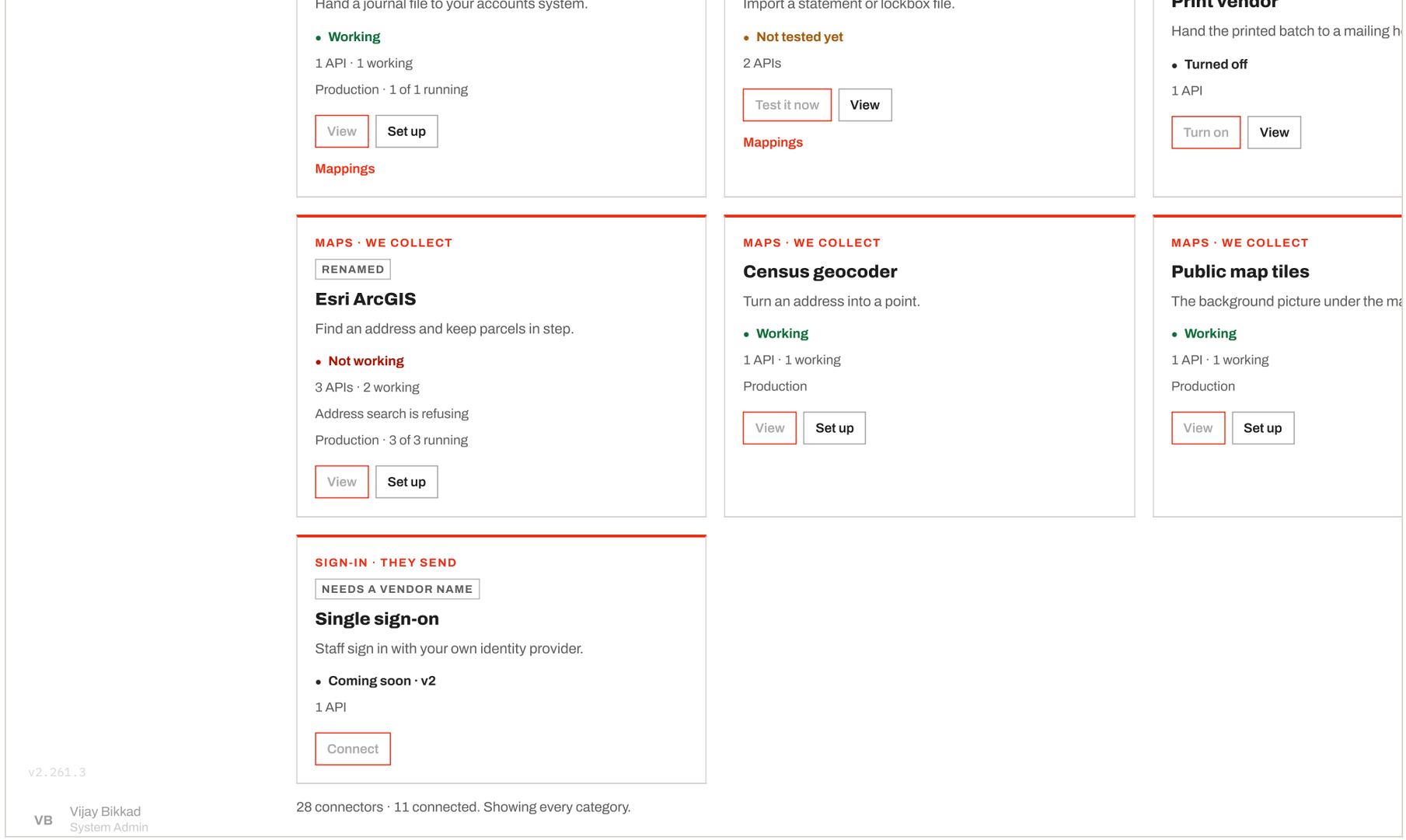
Accounting export file

BANKING · WE COLLECT

Bank & lockbox file

PRINT · WE SEND

NEEDS A VENDOR NAME



The toolbar, and what each of its four parts does

UI-RULES 12 is one toolbar in four parts — search · filter chips · `+ Filter ▾` · ▦ `Columns` — and this screen shipped with the first two. The last two are drawn here. Eleven chips need a line to themselves, so the chips take the second row and the numbers below run in reading order rather than in the rule's order. Each legend line carries four things and no more: what it is · where its data comes from · which rule it satisfies · new, changed or unchanged.

#	WHAT IT IS	DATA	RULE	FLAG
①	Search box, the kit's .input — name and summary	client filter over the loaded grid. No `?q=` is ever sent	UI-RULES 12 (toolbar) · **NFR-3 at the limit**	Unchanged
④	Category chips, generated from the registry — eleven, never a typed list	`payload.categories`, already served	UI-RULES 12 (filter chips) · B14 rule 37	Unchanged
②	`+ Filter ▾` — three dimensions the chips cannot carry: **direction** (We send · We collect · They send · They read), **state** (Working · Needs keys · Not working · Turned off · Coming soon), **environment** (Production · Sandbox)	every one is **already in the grid payload** — `kicker.direction`, `display_state`, `connection.environment`	UI-RULES 12 (+ Filter ▾)	**New**
③	▦ `Columns` — which facts each card shows: the per-API status lines, the failure reason, the environment line, the link row	local, per-viewer. Nothing is asked of the server	UI-RULES 14 (show · hide · reorder · reset)	**New**

A card grid has no columns, and the control is still right

A card is a row turned on its side, so the column selector chooses which *facts* each card carries rather than which columns a table draws. That matters most on this screen: Neptune's card is sixteen operation lines tall beside neighbours four lines tall (S8-1), and hiding the per-API lines is the one thing that makes the grid scannable again. Naming it ▦ `Columns` keeps the kit's own control rather than minting a second one — B14 rule 36.

If you would rather have a real table view behind a grid/table toggle, that is a different and larger change. Say so and I will draw it.

Three of the four are free; the search is not

`+ Filter ▾` needs no new read — direction, state and environment are all in the payload the grid already fetches. ▦ `Columns` is per-viewer and touches no server at all.

The search is the one that carries debt. It filters an array the browser already holds, and no `?q=` reaches the endpoint (FLD-2). At 28 manifests that is defensible, because the list is bounded by our code rather than by tenant data — and it is the same shape NFR-3 forbids, so it should be written down rather than tidied out of sight.

CR-3 The view screens called them "Operation 1". The forms called them by name.

MY ERROR · CORRECTED · 98 ROWS

You found it on Badger. The set-up form said Collect readings, Collect meter inventory, Collect alerts, Request billing export. The view screen for the same connector said Operation 1, 2, 3, 4. Counted across the pack: zero placeholder names in the 28 set-up forms, 98 of them across all 28 view screens. Every view screen disagreed with its own form.

Cause. The view screens were generated before the API name list existed, so they fell back to counting; the forms were regenerated afterwards and picked the real names up. Two generators, one subject, no check that they agreed. Fixed by making them read one source — the same list now feeds the form, the view screen, the run history, the connector card and the runbook panels, so the two cannot drift apart again.

And your second point was the sharper one: no form let anybody set the name. That is now ruled. A clerk does not name an API and does not add one — we do, when we set the connector up. So the set-up form lists the APIs and does not offer to edit them, and the disabled + Add an API button is gone: a control nobody may use is not a control, it is a promise. The fact sits under the list instead.

What that ruling buys, and it is the practical part: **no schema change.** If the name were the clerk's, it would need a per-connection field that does not exist. Because it is ours at onboarding, the name stays on the manifest, exactly where it already lives, and this whole item becomes drawing work rather than migration work.

```
Counted — 0 placeholder rows in 28 set-up forms · 98 across 28 view screens
registry.py — Operation.label is the name, declared on the manifest
Now — 0 placeholder rows anywhere in this pack
```

CR-10 The token lands on thirteen connectors, and two of them have no token to generate

RULED · 27 SCREENS

Ruled, 16 September: a connector we dial gets **one token, generated once, and every API on it authenticates with that token.** That is the shape the key already has — the screens have said *One key. All 16 APIs use it.* since the pack was drawn — so it costs no new relationship in the model, only the token row itself.

Thirteen connectors have a We send operation, which is the trigger. Their set-up forms gain a Token panel, their connection views show the token's state rather than a key's, and the eight cards that read *Needs keys* now read **Needs a token**.

But two of the thirteen cannot have one, and I had counted them in. My earlier figure said ten need the exchange fields. Eight do. **Custom SMTP** authenticates with a username and a password, because that is what the protocol has, and **Accounting export file** writes to a folder or hands over a download, and neither carries a token. Drawing *Generate token* on those two would be precisely the dead control this document spends a section on, so each says so instead.

And it leaves the product with two credential shapes on the outbound side. Eight connectors we dial hold a token; eight more we dial — the four meter head-ends, the bank file, Esri, the census geocoder and the tile server — still hold a key, because they carry no We send operation and so fell outside the ruling. A connection that dials out authenticates whichever way the data then flows, so that split is worth a decision rather than an accident.

```
Ruled — 13 connectors · 26 screens · plus the grid
Of the 13: 8 exchange · 2 already OAuth · 2 protocols with no token · 1 sandbox with no credential
Outside the ruling — 8 connectors we dial that keep a key: meters ×4, bank file, Esri, census, tiles
```

CR-11 Every API carries a rate limit, and the same column means two opposite things

RULED · DISPLAY ONLY · 78 ROWS

Ruled, 16 September: every API carries a rate limit, held in the database, **display only** on screen. It takes the same standing as the API's own name under CR-3 — ours when we configure the connector, never a clerk control — so no form offers to edit one and no button pretends otherwise.

One column, two meanings, and the door decides which. On an outward call it is the ceiling *they* allow *us*: exceed it and the vendor starts refusing, which on a nightly meter pull means an estimated bill three weeks later. On an inward call it is the ceiling *we* allow *them*: a kiosk network looping on account lookups is throttled at our door. The nine two-way connectors carry both, on the same table, one row apart — so the screen says which way it points rather than leaving a reader to work it out.

Nothing stores it today. Seventy-eight rows now draw a figure and not one figure is served. That makes this a back-end change wearing a wireframe, and the shape of it is worth naming: the API list lives on the **manifest**, which is code and is the same for every utility, while a ceiling is bought per contract — a

larger utility buys a larger quota. So the limit cannot live where the operation lives. It is a row per connection and per operation, with the manifest carrying the default that provisioning copies in.

⚠ **And every figure drawn is an illustration**, under B14 rule 37. No value here was read from anything; they are shaped by the trigger so that the drawing is internally consistent, and nothing more. A builder taking `10 / min` off a meter row as the real ceiling would be doing exactly what that rule exists to stop.

The half that is missing is the count, not the ceiling. A limit with no consumption beside it cannot answer the only question a clerk has, which is whether we are near it. The pack already draws that half — a budget line reading *Calls used this week — 4,180 of 10,000* — and it has never rendered, which S141-1 already files. The limit and the count are one feature and should land together.

> Ruled — 78 operation rows across 56 screens, display only

> NOT SERVED — no table holds a rate limit; a row per connection and operation is what it needs

> Enforcement — this document records throttling a partner as built; nothing records a throttle keeping US under a vendor's ceiling, so the outward figure promises more than the inward one

CR-4 One key opens every API on the connector

SECURITY SHAPE · RULED: SCOPE IT

This is the one item on the list that is not about usability. A key we issue is scoped to a connection and to nothing else, and the database keeps one active credential per connection. So a partner handed a key to push meter readings can call every other inbound API on that connector — because there is no field that says which ones they may call.

Ruled: a key names the APIs it may call. KEY-01 draws it — tick what they need, and a key with nothing ticked cannot be issued. The view screens for the six connectors that issue keys now carry a May call column.

This one does need the model to move. issue_key() takes a connection and a label; there is nowhere to put a scope. That is a new column and a check at the door, not a drawing change.

> machine_auth.py:227 — issue_key(connection=…), no operation argument

> machine_auth.py:215 — _active_credential(connection), one active key per connection

> Affected — 6 connectors issue keys today: the five ERPs and the kiosk network

CR-6 The manifest ships their words and has nowhere to ship our answer

ONE MISSING FIELD · EVERY TENANT PAYS FOR IT

A manifest already ships the vendor's vocabulary. The kiosk ships its fourteen tender words; Nacha's return codes ship in code. So MAP-01 has rows to show before a single payment has arrived — that part is right, and it is the framework working.

What it cannot ship is the right-hand column. Vocabulary carries entity_type · label · values · summary · open — their words, and no field for what each one means here. So every tenant maps the same fourteen words to the same seven tenders, by hand, and the answer is identical every time.

It is identical because our tenders are seeded identically. Provisioning seeds the seven tenders into every tenant from one platform list — so ACHCHECK → Bank transfer is not a utility's opinion, it is a fact about InvoiceCloud. Asking a clerk is asking the same question at every go-live and hoping for the same answer.

The fix is one field on Vocabulary and one step at provisioning. MAP-01 is drawn as it would then look: 14 of 14 mapped, a review rather than a form.

> registry.py — Vocabulary(entity_type, label, values, summary, open), five fields, none for the answer

> manifests/payments_kiosk.py:93 — values=KIOSK_TENDER_WORDS, their side already shipped

> tenancy/services.py — "the seven tenders themselves" are seeded per tenant from one list

> Two exceptions, permanently — ledger_account and route are the utility's own data, drawn on MAP-02

CR-7 A runbook's clock is UTC, and the utility's time zone is sitting right there

ONE LINE · FIRES 6 HOURS EARLY

The tenant record already holds a time zone — an IANA name, on tenancy.Tenant. add_template never reads it, so every runbook a utility adds starts at UTC.

What that costs, in the plainest terms: an Iowa utility sets a nightly job for 01:00 and it runs at 7pm the previous evening. Collection notices at dinner time. A month-end tie-out running in the wrong month.

It does not bite yet, because nothing can be scheduled at all — which is why it is worth fixing in the same change as the schedule form rather than after somebody notices.

> tenancy/models.py:193 — timezone = TextField(null=True), an IANA name

> runbooks/models.py — timezone = TextField(default="UTC")

> `runbooks/services.py:656 — add_template sets nine fields and not this one`

CR-5 A runbook asks "is there a door", and one door answers for sixteen

REAL GAP · CROSSES THE LINE YOU DREW

This is the connector/business-logic line being crossed in the direction nobody watches. A runbook asks the integrations seam for live connections and gets back, per connector, `{key, display_name, category, status, enabled}`. There is no operation in that answer. So a runbook step that needs Request billing export proceeds because Collect readings is green — one status standing in for sixteen.

Drawn on five runbook screens as a **Connectors this runbook needs** panel: connector, which API, and that API's own state. The seam grows one field; the runbook keeps holding no credential and no config, which is the part of the line that is right today and should stay right.

> `runbooks/seams/integrations_seam.py:68 — live_connections(provider_keys), five fields, no operation`

> `cash_application.py:441 · bank_reconciliation.py:354 — both filter on status == "working", per connector`

> `IntegrationRun.operation — the run row already records which API it was, and is indexed by it`

CR-2 My draft put the manifest key on screen — that is a database id, and rule 31 forbids it

MY ERROR, CORRECTED

Every connection view read `email_microsoft · 1 operation` under the title. email_microsoft is the manifest key — an internal identifier, and B14 rule 31 is exactly this: no database id is ever the visible value of a field. Rule 32 bars internal code from clerk-facing copy for the same reason.

The as-built screen does not do this — it shows the connection's own describe line, which is the thing a clerk recognises: the mailbox it sends from, the folder it watches, the site it collects from. I substituted the key because I was generating 28 screens and had no describe value to hand. That is how a machine identifier reaches a clerk: not by decision, by default.

Worth keeping as a finding rather than a silent fix. It is the second thing in two turns that a carry-over table or a rule pass would have caught and my generator did not — and it is the same class as the real one on S143, where a screen that ships today draws rounded pills in a product with no rounded corners.

> Fixed on all 28 views — email_microsoft · 1 operation → billing@ottumwa-water.example · 1 operation

> The key stays in the address — /settings/connectors/email_microsoft — where it belongs and no clerk reads it

CR-1 **My first draft of this grid had no way to reach any of the 28 connection screens**

MY ERROR, CORRECTED

The as-built card carries a link row — Recent activity, Mappings, Keys we issue — and Recent activity is the only door to the connection screen. I dropped that row when I redrew the grid, so every one of the 28 view screens I had just drawn was unreachable from the card that owns it.

That is the exact failure this document spends a section on — a screen built, drawn and tested with no inbound link — and I introduced it into my own remedy. What would have caught it is the carry-over table I did not write for this drawing. Here it is, and it is the reason the rule exists: reading the picture never catches a lost control, and the subtraction always does.

> Fixed above: every connected card now leads with View, and the contextual links are back

Carry-over for the proposed grid. A remedy for an existing screen never loses anything.

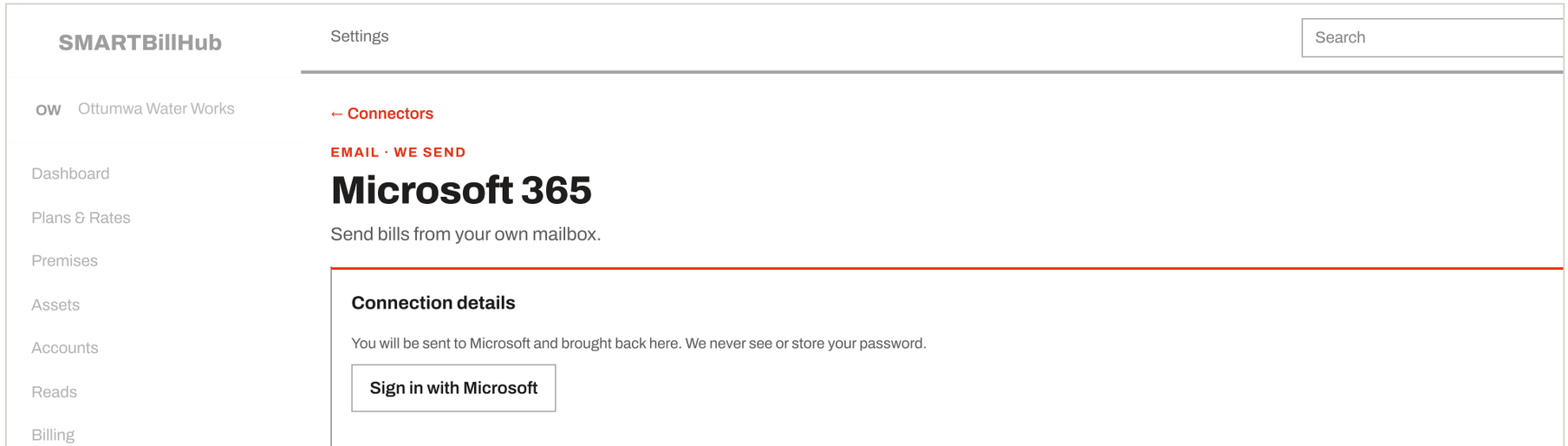
ON THE AS-BUILT CARD	DISPOSITION
Category · direction kicker	Unchanged
Provider name	Relabelled — the vendor's real name
Summary sentence	Unchanged
Status word · failure reason · environment line	Unchanged ×3
Per-operation status lines	Unchanged
The one state-driven action button	Unchanged
"Recent activity" link	Relabelled — becomes View, and leads
"Mappings" link · "Keys we issue" link	Unchanged ×2

Search box · category chips · footnote	Unchanged ×3
"Set up" action	New — the form has an address now, so a connected card can reopen it
New-connector flag	New — marks the 13 that do not ship today
"+ Filter ▾ "	New — the three dimensions the category chips do not carry
"▦ Columns"	New — which facts each card shows

```
As-built  12 = unchanged 10 + relabelled 2 + removed 0
Remedy    16 = 12 − 0 removed + 4 new            reconciles
```

> **Why "View" rather than keeping "Recent activity"**
>
> The screen it opens is titled Connector and carries the operations, the credentials, the actions and the runs. "Recent activity" names the last section of it and undersells the rest — a clerk looking for *where do I see this connection* does not read that label as the answer. It is a relabel, not a removal: the door is the same door.

CFG-01 **Microsoft 365 — set up** `/:tenant/settings/connectors/email_microsoft/configure` **PROPOSED FORM**



Payments

Work

Finance

Runbooks

Reports

Settings

Send bills from *

billing@yourutility.example

The address bills are sent from.

Where it runs

Production | Sandbox

API on this connector 1

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T
Send messages `api.vendor.example`	We send	When queued	`120 / min`	

All 1 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

Signing in above gives us the token. We generate it once and it refreshes itself.

One token. All 1 API use it.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

Save and test | Cancel

Signing in is all ther

VB Vijay Bikkad
System Admin

VIEW-01 Microsoft 365 — the connection

`/:tenant/settings/connectors/email_microsoft`

S142, FOR THIS CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

← Connectors

CONNECTOR

Microsoft 365

billing@ottumwa-water.example 1 API • Working

Email · We send · Production · connected by Alex Reed

1 API on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN	RESULT
Send messages `api.vendor.example`	We send	When queued	`120 / min`	Today 04:12	• Refused Retry The site refused the

Sign-in on this connector

There is no password here. You sign in with the provider and we keep only the permission they give us.

Token

Renews

One token. All 1 API use it.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Send messages	1,240 of 1,240	`8F2B9C1D`
Yesterday, 06:00	Send messages	412 of 412	`4C71A0E5`

1–2 of 2 · Rows per page 50

Re-authorise now | Disconnect

Runs are kept exactly as they happ

CFG-02 **Google Workspace — set up** `/:tenant/settings/connectors/email_google/configure` PROPOSED FORM

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

← Connectors

EMAIL · WE SEND

Google Workspace

Send bills from your own mailbox.

Connection details

You will be sent to Google and brought back here. We never see or store your password.

Sign in with Google

Send bills from *

billing@yourutility.example

The address bills are sent from.

Where it runs

Production | Sandbox

API on this connector 1

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T
Send messages `api.vendor.example`	We send	When queued	`120 / min`	

All 1 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

Signing in above gives us the token. We generate it once and it refreshes itself.

One token. All 1 API use it.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Signing in is all ther

VIEW-02 **Google Workspace — the connection** `/:tenant/settings/connectors/email_google` **S142, FOR THIS CONNECTOR**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Google Workspace

billing@ottumwa-water.example 1 API • Working

Email · We send · Production · connected by Alex Reed

1 API on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Send messages `api.vendor.example`	We send	When queued	`120 / min`	Today 04:12

Sign-in on this connector

There is no password here. You sign in with the provider and we keep only the permission they give us.

Token

Renews

One token. All 1 API use it.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Send messages	1,240 of 1,240	8F2B9C1D
Yesterday, 06:00	Send messages	412 of 412	4C71A0E5

1–2 of 2 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Re-authorise now | Disconnect

Runs are kept exactly as they happ

CFG-03 **Custom SMTP — set up** `/:tenant/settings/connectors/email_smtp/configure` PROPOSED FORM

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

← Connectors

EMAIL · WE SEND

Custom SMTP

Any mail server with a host and a password.

Connection details

Mail server *
api.vendor.example

Port *
587

Username *

Password *
Typed once, never shown again

Stored encrypted. Never shown again.

Send bills from *
billing@yourutility.example

The address bills are sent from.

Where it runs

Production | Sandbox

API on this connector 1

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T
Send messages `api.vendor.example`	We send	When queued	`120 / min`	

All 1 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Sign-in on this connector

No token here. This one signs in with the details above, on every call.

⚠ A token is generated on every connector we dial that speaks over an API. This one does not — a mail server authenticates with a username and a password. Named rather than g
nobody could press.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Testing sends one real messa

VIEW-03 Custom SMTP — the connection

`/:tenant/settings/connectors/email_smtp`

S142, FOR THIS CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard

Plans & Rates

Premises

← Connectors

CONNECTOR

Custom SMTP

smtp.ottumwa-water.example : 587 1 API • Working

Email · We send · Production · connected by Alex Reed

Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

Email · We send · Production · connected by Alex Reed

1 API on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Send messages `api.vendor.example`	We send	When queued	`120 / min`	Today 04:12

Key we hold

•••• 7Q2F 12 Aug 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One key. All 1 APIs use it.

⚠ No token on this one — a mail server authenticates with a username and a password.

Recent activity

Support reference | All | + Filter ▾

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Send messages	1,240 of 1,240	`8F2B9C1D`
Yesterday, 06:00	Send messages	412 of 412	`4C71A0E5`

1–2 of 2 · Rows per page 50

Test all | Replace key | Disconnect

Runs are kept exactly as they happ

CFG-04 **Bulk email service — set up** `/:tenant/settings/connectors/email_bulk/configure` NEEDS A VENDOR NAME

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

← Connectors

EMAIL · WE SEND · THEY SEND

Bulk email service

Hand a large batch to a sending service.

> Still generic. You have not named a vendor for this one — it keeps a category name until you do.

Connection details

API key *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 2 APIs.

Sending domain *

api.vendor.example

Send bills from *

billing@yourutility.example

The address bills are sent from.

Where it runs

Production | Sandbox

APIs on this connector 2

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T
Send messages `api.vendor.example`	We send	When queued	`120 / min`	
Delivery feedback `api.vendor.example`	They send	They call us	`300 / min`	

All 2 use the address above.

Outward the limit is what they allow us; inward it is what we allow them.

> ⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

We generate one token for this connector and every API authenticates with it.

Token address *

api.vendor.example

Scope

Leave blank for everything the key allows

No token yet — save this connector, then generate one.

Generate token

One token. All 2 APIs use it. It is shown once and afterwards only regenerated.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-04 Bulk email service — the connection

`/:tenant/settings/connectors/email_bulk` **PROPOSED**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Bulk email service

No connector exists yet. This is what the screen shows once one does.

mail.ottumwa-water.example 2 APIs • 2 of 2 working

Email · We send · They send · Production · connected by Alex Reed

2 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Send messages `api.vendor.example`	We send	When queued	`120 / min`	Today 04:12
Delivery feedback `api.vendor.example`	They send	They call us	`300 / min`	Today 04:15

Token we hold

••••7Q2F generated 12 Aug 2026 expires 11 Nov 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One token. All 2 APIs use it.

Recent activity

Support reference | All | + Filter | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Send messages	1,240 of 1,240	8F2B9C1D
Today 04:15	Delivery feedback	1,240 of 1,240	8F2B9C1D

1–2 of 2 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Regenerate token | Disconnect

Runs are kept exactly as they happ

CFG-05 **Twilio — set up** `/:tenant/settings/connectors/sms_twilio/configure` PROPOSED FORM

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance

← Connectors

MESSAGING · WE SEND · THEY SEND

Twilio

Send text messages to customers.

> Coming soon · v1.1 — not built yet.

Connection details

Account SID *

Auth token *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 2 APIs.

Send from number *

Runbooks

Reports

Settings

Where it runs

Production | Sandbox

APIs on this connector 2

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T
Send texts `api.twilio.example`	We send	When queued	`120 / min`	
Delivery status `api.twilio.example`	They send	They call us	`300 / min`	

All 2 use the address above.

Outward the limit is what they allow us; inward it is what we allow them.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

We generate one token for this connector and every API authenticates with it.

Token address *

api.vendor.example

Scope

Leave blank for everything the key allows

No token yet — save this connector, then generate one.

Generate token

One token. All 2 APIs use it. It is shown once and afterwards only regenerated.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Savin

VIEW-05 **Twilio — the connection** `/:tenant/settings/connectors/sms_twilio` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

← Connectors

CONNECTOR

Twilio

No connector exists yet. This is what the screen shows once one does.

+1 641 555 0143 2 APIs • 2 of 2 working

Messaging · We send · They send · Production · connected by Alex Reed

2 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Send texts `api.twilio.example`	We send	When queued	`120 / min`	Today 04:12
Delivery status `api.twilio.example`	They send	They call us	`300 / min`	Today 04:15

Token we hold

••••7Q2F generated 12 Aug 2026 expires 11 Nov 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One token. All 2 APIs use it.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Send texts	1,240 of 1,240	`8F2B9C1D`
Today 04:15	Delivery status	1,240 of 1,240	`8F2B9C1D`

1–2 of 2 · Rows per page 50

Test all | Regenerate token | Disconnect

Runs are kept exactly as they happ

VB Vijay Bikkad
System Admin

Test all | Regenerate token | Disconnect

CFG-06 **Neptune 360 — set up** `/:tenant/settings/connectors/meters_neptune360/configure` RENAMED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

METERS · WE COLLECT

Neptune 360

Collect readings and keep the meter list in step.

> Renamed. Ships today under a generic name. The connector, its fields and its operations do not change.

Connection details

Site ID *

API key *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 16 APIs.

Where it runs

Production | Sandbox

APIs on this connector 16

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	TI
Meter readings `api.neptune360.example`	We collect	Nightly	10 / min	
Readings by route `api.neptune360.example`	We collect	Nightly	10 / min	
Billing export `api.neptune360.example`	We collect	Nightly	10 / min	
Hourly interval data `api.neptune360.example`	We collect	Nightly	10 / min	
Meter alerts `api.neptune360.example`	We collect	Nightly	10 / min	

Register replacements `api.neptune360.example`	We collect	Nightly	`10 / min`
Meter inventory `api.neptune360.example`	We collect	Nightly	`10 / min`
Endpoint inventory `api.neptune360.example`	We collect	Nightly	`10 / min`

…8 more

All 16 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

When it runs

13 of 16 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-06 Neptune 360 — the connection

`/:tenant/settings/connectors/meters_neptune360`

S142, FOR THIS CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard

Plans & Rates

Premises

Assets

← Connectors

CONNECTOR

Neptune 360

Site 4471 | 16 APIs | • Working

Meters · We collect · Production · connected by Alex Reed

Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

16 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN	RESULT
Meter readings `api.neptune360.example`	We collect	Nightly	`10 / min`	Today 04:12	• Worked
Readings by route `api.neptune360.example`	We collect	Nightly	`10 / min`	Today 04:15	• Worked
Billing export `api.neptune360.example`	We collect	Nightly	`10 / min`	Today 04:19	• Worked
Hourly interval data `api.neptune360.example`	We collect	Nightly	`10 / min`	Today 05:02	• Worked
Meter alerts `api.neptune360.example`	We collect	Nightly	`10 / min`	Today 05:40	• Refused Retry The site refused the k
Register replacements `api.neptune360.example`	We collect	Nightly	`10 / min`	Today 06:07	• Worked
Meter inventory `api.neptune360.example`	We collect	Nightly	`10 / min`	Today 23:50	• Worked
Endpoint inventory `api.neptune360.example`	We collect	Nightly	`10 / min`	Today 23:58	• Worked

…8 more

Key we hold

••••7Q2F 12 Aug 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One key. All 16 APIs use it.

⚠ No token on this one — a file goes to a folder or a download, and neither carries a token.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Meter readings	1,240 of 1,240	`8F2B9C1D`
Yesterday, 06:00	Readings by route	412 of 412	`4C71A0E5`

1–2 of 2 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Replace key | Disconnect

Runs are kept exactly as they happ

CFG-07 **Badger BEACON — set up** `/:tenant/settings/connectors/meters_badger/configure` NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

METERS · WE COLLECT

Badger BEACON

Collect readings from the BEACON head-end.

> New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Service address *
api.vendor.example

Site / account ID *

API key *
Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 4 APIs.

Where it runs

Production | Sandbox

APIs on this connector 4

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Collect readings `api.vendor.example` PROPOSED	We collect	Nightly	10 / min
Collect meter inventory `api.vendor.example` PROPOSED	We collect	Nightly	10 / min

Collect alerts `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Request billing export `api.vendor.example` PROPOSED	We collect	On demand	`60 / min`

All 4 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

When it runs

3 of 4 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-07 Badger BEACON — the connection

`/:tenant/settings/connectors/meters_badger` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads

← Connectors

CONNECTOR

Badger BEACON

No connector exists yet. This is what the screen shows once one does.

Site 4471 · BEACON 4 APIs • 3 of 4 working

Meters · We collect · Production · connected by Alex Reed

4 APIs on this connector

Billing
Payments
Work
Finance
Runbooks
Reports
Settings

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN	RESULT
Collect readings `api.vendor.example` PROPOSED	We collect	Nightly	10 / min	Today 04:12	• Worked
Collect meter inventory `api.vendor.example` PROPOSED	We collect	Nightly	10 / min	Today 04:15	• Worked
Collect alerts `api.vendor.example` PROPOSED	We collect	Nightly	10 / min	Today 04:19	• Worked
Request billing export `api.vendor.example` PROPOSED	We collect	On demand	60 / min	Today 04:19	• Refused Retry The site refused

Key we hold

••••7Q2F 12 Aug 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One key. All 4 APIs use it.

⚠ No token on this one — a file goes to a folder or a download, and neither carries a token.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Collect readings	1,240 of 1,240	8F2B9C1D
Today 04:15	Collect meter inventory	1,240 of 1,240	8F2B9C1D
Today 04:19	Collect alerts	1,240 of 1,240	8F2B9C1D

1–3 of 4 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Replace key | Disconnect

Runs are kept exactly as they happ

CFG-08 **Itron — set up** /:tenant/settings/connectors/meters_itron/configure NEW · SPLIT OUT

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

METERS · WE COLLECT

Itron

Collect readings from the Itron head-end.

> New · split out. Itron and Sensus share one card today. This gives each its own connector.

Connection details

Service address *

api.vendor.example

Site / account ID *

API key *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 4 APIs.

Where it runs

Production | Sandbox

APIs on this connector 4

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Collect readings `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Collect meter inventory `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Collect alerts `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Request billing export `api.vendor.example` PROPOSED	We collect	On demand	`60 / min`

All 4 use the address above.

These are the limits they allow us.

> ⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

When it runs

3 of 4 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

Save and test | Cancel | Saving

VB Vijay Bikkad
System Admin

VIEW-08 Itron — the connection

`/:tenant/settings/connectors/meters_itron` **PROPOSED**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Itron

No connector exists yet. This is what the screen shows once one does.

Agency 88214 | 4 APIs | • 4 of 4 working

Meters · We collect · Production · connected by Alex Reed

4 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Collect readings `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:12
Collect meter inventory `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:15
Collect alerts `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:19
Request billing export	We collect	On demand	`60 / min`	Today 05:02

api.vendor.example PROPOSED

Key we hold

••••7Q2F 12 Aug 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One key. All 4 APIs use it.

⚠ No token on this one — a file goes to a folder or a download, and neither carries a token.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Collect readings	1,240 of 1,240	8F2B9C1D
Today 04:15	Collect meter inventory	1,240 of 1,240	8F2B9C1D
Today 04:19	Collect alerts	1,240 of 1,240	8F2B9C1D

1–3 of 4 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Replace key | Disconnect

Runs are kept exactly as they happ

CFG-09 Sensus — set up

/:tenant/settings/connectors/meters_sensus/configure

NEW · SPLIT OUT

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard

Plans & Rates

Premises

← Connectors

METERS · WE COLLECT

Sensus

Collect readings from the Sensus head-end.

New · split out. Itron and Sensus share one card today. This gives each its own connector

Assets

Accounts

Reads

Billing

Payments

Work

Finance

Runbooks

Reports

Settings

Now "split cat. Itron and Sensus share one card today. This gives each its own connector.

Connection details

Service address *

api.vendor.example

Site / account ID *

API key *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 4 APIs.

Where it runs

Production | Sandbox

APIs on this connector 4

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Collect readings `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Collect meter inventory `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Collect alerts `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Request billing export `api.vendor.example` PROPOSED	We collect	On demand	`60 / min`

All 4 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

When it runs

3 of 4 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-09 Sensus — the connection

`/:tenant/settings/connectors/meters_sensus` **PROPOSED**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Sensus

No connector exists yet. This is what the screen shows once one does.

Site 4471 | 4 APIs | • 4 of 4 working

Meters · We collect · Production · connected by Alex Reed

4 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Collect readings `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:12
Collect meter inventory `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:15
Collect alerts `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:19
Request billing export `api.vendor.example` PROPOSED	We collect	On demand	`60 / min`	Today 05:02

Key we hold

••••7Q2F 12 Aug 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One key. All 4 APIs use it.

⚠ No token on this one — a file goes to a folder or a download, and neither carries a token.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Collect readings	1,240 of 1,240	8F2B9C1D
Today 04:15	Collect meter inventory	1,240 of 1,240	8F2B9C1D
Today 04:19	Collect alerts	1,240 of 1,240	8F2B9C1D

1–3 of 4 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Replace key | Disconnect

Runs are kept exactly as they happ

CFG-10 **InvoiceCloud — set up** /:tenant/settings/connectors/pay_invoicecloud/configure NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance

← Connectors

PAYMENTS · WE COLLECT · WE SEND

InvoiceCloud

Take card and bank payments through InvoiceCloud.

> New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Merchant ID *

Merchant secret *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 4 APIs.

Client ID *

Card page *

Runbooks

Reports

Settings

hosted by them | embedded

Where it runs

Production | Sandbox

APIs on this connector 4

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`

All 4 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

We generate one token for this connector and every API authenticates with it.

Token address *

api.vendor.example

Scope

Leave blank for everything the key allows

No token yet — save this connector, then generate one.

Generate token

One token. All 4 APIs use it. It is shown once and afterwards only regenerated.

When it runs

1 of 4 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Savin

VIEW-10 **InvoiceCloud — the connection** `/:tenant/settings/connectors/pay_invoicecloud` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

← Connectors

CONNECTOR

InvoiceCloud

No connector exists yet. This is what the screen shows once one does.

Merchant 11f0•••308d 4 APIs • 4 of 4 working

Payments · We collect · We send · Production · connected by Alex Reed

4 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 04:12
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`	Today 04:15
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:19
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 05:02

Token we hold

••••7Q2F generated 12 Aug 2026 expires 11 Nov 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One token. All 4 APIs use it.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Charge a card	1,240 of 1,240	8F2B9C1D
Today 04:15	Hosted card page	1,240 of 1,240	8F2B9C1D
Today 04:19	Settlement report	1,240 of 1,240	8F2B9C1D

1–3 of 4 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Regenerate token | Disconnect

Runs are kept exactly as they happ

CFG-11 **Paymentus — set up** /:tenant/settings/connectors/pay_paymentus/configure NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance

← Connectors

PAYMENTS · WE COLLECT · WE SEND

Paymentus

Take card and bank payments through Paymentus.

> New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Merchant ID *

Merchant secret *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 4 APIs.

Client ID *

Card page *

hosted by them | embedded

Runbooks

Reports

Settings

Where it runs

Production | Sandbox

APIs on this connector 4

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`

All 4 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

We generate one token for this connector and every API authenticates with it.

Token address *

api.vendor.example

Scope

Leave blank for everything the key allows

No token yet — save this connector, then generate one.

Generate token

One token. All 4 APIs use it. It is shown once and afterwards only regenerated.

When it runs

1 of 4 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-11 Paymentus — the connection

`/:tenant/settings/connectors/pay_paymentus` **PROPOSED**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Paymentus

> No connector exists yet. This is what the screen shows once one does.

Merchant 11f0••••308d | 4 APIs | • 4 of 4 working

Payments · We collect · We send · Production · connected by Alex Reed

4 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 04:12
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`	Today 04:15
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:19
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 05:02

Token we hold

••••7Q2F generated 12 Aug 2026 expires 11 Nov 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One token. All 4 APIs use it.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Charge a card	1,240 of 1,240	8F2B9C1D
Today 04:15	Hosted card page	1,240 of 1,240	8F2B9C1D
Today 04:19	Settlement report	1,240 of 1,240	8F2B9C1D

1–3 of 4 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Regenerate token | Disconnect

Runs are kept exactly as they happ

CFG-12 **US Payments — set up** `/:tenant/settings/connectors/pay_uspayments/configure` NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks

← Connectors

PAYMENTS · WE COLLECT · WE SEND

US Payments

Take card and bank payments through US Payments.

> New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Merchant ID *

Merchant secret *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 4 APIs.

Client ID *

Card page *

hosted by them | embedded

Reports

Settings

Where it runs

Production | Sandbox

APIs on this connector 4

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`

All 4 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

We generate one token for this connector and every API authenticates with it.

Token address *

api.vendor.example

Scope

Leave blank for everything the key allows

No token yet — save this connector, then generate one.

Generate token

One token. All 4 APIs use it. It is shown once and afterwards only regenerated.

When it runs

1 of 4 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-12 **US Payments — the connection** `/:tenant/settings/connectors/pay_uspayments` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

← Connectors

CONNECTOR

US Payments

No connector exists yet. This is what the screen shows once one does.

Merchant 11f0••••308d 4 APIs • 4 of 4 working

Payments · We collect · We send · Production · connected by Alex Reed

4 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 04:12
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`	Today 04:15
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:19
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 05:02

Token we hold

`••••7Q2F` generated 12 Aug 2026 expires 11 Nov 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One token. All 4 APIs use it.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Charge a card	1,240 of 1,240	8F2B9C1D
Today 04:15	Hosted card page	1,240 of 1,240	8F2B9C1D
Today 04:19	Settlement report	1,240 of 1,240	8F2B9C1D

1–3 of 4 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Regenerate token | Disconnect

Runs are kept exactly as they happ

CFG-13 **WorldPay — set up** `/:tenant/settings/connectors/pay_worldpay/configure` NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks

← Connectors

PAYMENTS · WE COLLECT · WE SEND

WorldPay

Take card and bank payments through WorldPay.

> New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Merchant ID *

Merchant secret *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 4 APIs.

Client ID *

Card page *

hosted by them | embedded

Reports

Settings

Where it runs

Production | Sandbox

APIs on this connector 4

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`

All 4 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

We generate one token for this connector and every API authenticates with it.

Token address *

api.vendor.example

Scope

Leave blank for everything the key allows

No token yet — save this connector, then generate one.

Generate token

One token. All 4 APIs use it. It is shown once and afterwards only regenerated.

When it runs

1 of 4 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Saving

VIEW-13 **WorldPay — the connection** `/:tenant/settings/connectors/pay_worldpay` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

WorldPay

No connector exists yet. This is what the screen shows once one does.

Merchant 11f0••••308d 4 APIs • 4 of 4 working

Payments · We collect · We send · Production · connected by Alex Reed

4 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 04:12
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`	Today 04:15
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:19
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 05:02

Token we hold

••••7Q2F generated 12 Aug 2026 expires 11 Nov 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One token. All 4 APIs use it.

Recent activity

Support reference | All | + Filter

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Charge a card	1,240 of 1,240	8F2B9C1D
Today 04:15	Hosted card page	1,240 of 1,240	8F2B9C1D
Today 04:19	Settlement report	1,240 of 1,240	8F2B9C1D

1–3 of 4 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Regenerate token | Disconnect

Runs are kept exactly as they happ

CFG-14 **Global Payments — set up** `/:tenant/settings/connectors/pay_globalpayments/configure` NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports

← Connectors

PAYMENTS · WE COLLECT · WE SEND

Global Payments

Take card and bank payments through Global Payments.

New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Merchant ID *

Merchant secret *

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 4 APIs.

Client ID *

Card page *

hosted by them | embedded

Where it runs

Settings

Production | Sandbox

APIs on this connector 4

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`

All 4 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

We generate one token for this connector and every API authenticates with it.

Token address *

api.vendor.example

Scope

Leave blank for everything the key allows

No token yet — save this connector, then generate one.

Generate token

One token. All 4 APIs use it. It is shown once and afterwards only regenerated.

When it runs

1 of 4 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

VB Vijay Bikkad
System Admin

Save and test | Cancel

Saving

VIEW-14 **Global Payments — the connection** `/:tenant/settings/connectors/pay_globalpayments` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

← Connectors

CONNECTOR

Global Payments

No connector exists yet. This is what the screen shows once one does.

Merchant 11f0••••308d 4 APIs • 4 of 4 working

Payments · We collect · We send · Production · connected by Alex Reed

4 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Charge a card `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 04:12
Hosted card page `api.vendor.example` PROPOSED	We send	While you wait	`600 / min`	Today 04:15
Settlement report `api.vendor.example` PROPOSED	We collect	Nightly	`10 / min`	Today 04:19
Refund a payment `api.vendor.example` PROPOSED	We send	On demand	`60 / min`	Today 05:02

Token we hold

••••7Q2F generated 12 Aug 2026 expires 11 Nov 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One token. All 4 APIs use it.

Recent activity

Support reference | All | + Filter | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Charge a card	1,240 of 1,240	8F2B9C1D
Today 04:15	Hosted card page	1,240 of 1,240	8F2B9C1D
Today 04:19	Settlement report	1,240 of 1,240	8F2B9C1D

1–3 of 4 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Regenerate token | Disconnect

Runs are kept exactly as they happ

CFG-15 **Payment kiosk network — set up** `/:tenant/settings/connectors/payments_kiosk/configure` PROPOSED FORM

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports

← Connectors

PAYMENTS · THEY SEND · THEY READ

Payment kiosk network

Take payments at a machine, and hear when one bounces.

Connection details

The key for them to use

No key yet — save this connector, then issue one.

Issue key

Allowed addresses

Where it runs

Production | Sandbox

APIs on this connector 3

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	TI

Settings

Payments `api.vendor.example`	They send	They call us	`300 / min`
Account lookup `api.vendor.example`	They read	They call us	`300 / min`
Returns `api.vendor.example`	They send	They call us	`300 / min`

All 3 use the address above.

These are the limits we allow them.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

They start this one. There is no schedule — the trigger is their call.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Ready. It starts wor

VIEW-15 Payment kiosk network — the connection

`/:tenant/settings/connectors/payments_kiosk`

S142, FOR THIS CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance

← Connectors

CONNECTOR

Payment kiosk network

••••7Q2F · 3 addresses allowed | 3 APIs | • Working

Payments · They send · They read · Production · connected by Alex Reed

3 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Payments `api.vendor.example`	They send	They call us	`300 / min`	Today 04:12
Account lookup `api.vendor.example`	They read	They call us	`300 / min`	Today 04:15
Returns `api.vendor.example`	They send	They call us	`300 / min`	Today 04:19

Runbooks

Reports

Settings

api.vendor.example

Keys we issue

KEY	MAY CALL	ISSUED	STATE
••••7Q2F	Payments, Account lookup +1	12 Aug 2026	In us

⚠ May call is new. Today one key opens every API on the connector — see CR-4.

Recent activity

Support reference | All | + Filter ▾

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Payments	1,240 of 1,240	8F2B9C1D
Yesterday, 06:00	Account lookup	412 of 412	4C71A0E5

1–2 of 2 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Keys we issue | Disconnect

Runs are kept exactly as they happ

CFG-16 **Test payments (sandbox) — set up** `/:tenant/settings/connectors/payments_sandbox/configure` PROPOSED FORM

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard

Plans & Rates

Premises

Assets

Accounts

← Connectors

PAYMENTS · WE SEND · THEY SEND

Test payments (sandbox)

A test processor for the whole payment journey.

Connection details

How every charge answers *

Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

approve | decline | timeout

Where it runs

Production | Sandbox

APIs on this connector 3

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T
Take a payment `api.vendor.example`	We send	On demand	`60 / min`	
Card page `api.vendor.example`	We send	While you wait	`600 / min`	
Payment confirmed `api.vendor.example`	They send	They call us	`300 / min`	

All 3 use the address above.

Outward the limit is what they allow us; inward it is what we allow them.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

No token. The sandbox answers without one.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Saving

VIEW-16 Test payments (sandbox) — the connection

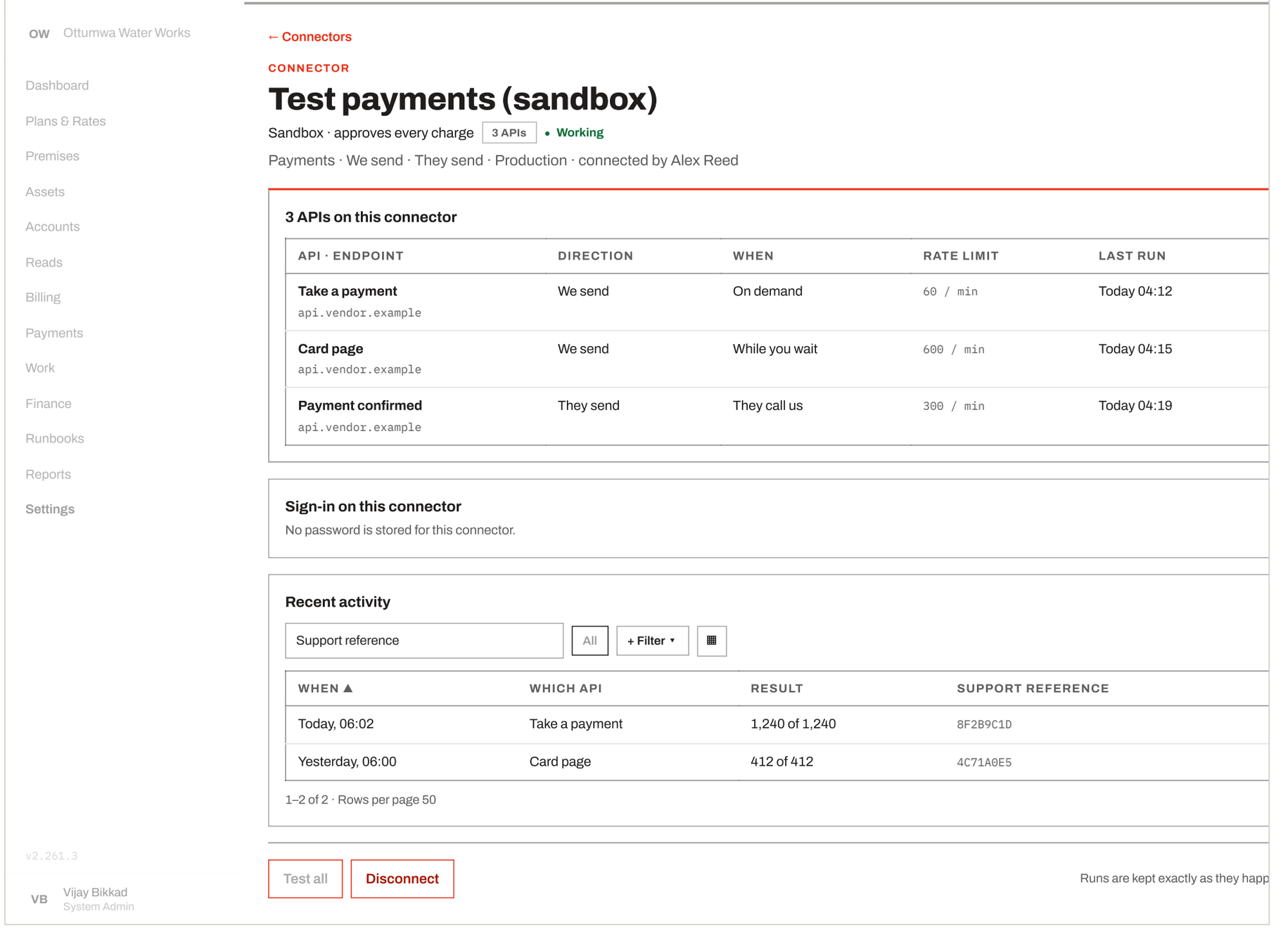
`/:tenant/settings/connectors/payments_sandbox`

S142, FOR THIS CONNECTOR

SMARTBillHub

Settings

Search



CFG-17 **Aclarian — set up** `/:tenant/settings/connectors/erp_aclarian/configure` NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

ERP · THEY READ

Aclarian

Your finance system collects payments and bills from here.

New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Their client id *

What they may read *

payments | bills | customers

Allowed addresses *

The key for them to use

No key yet — save this connector, then issue one.

Issue key

Where it runs

Production | Sandbox

APIs on this connector 3

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Read payments `api.vendor.example` PROPOSED	They read	They call us	`300 / min`
Read bills `api.vendor.example` PROPOSED	They read	They call us	`300 / min`
Read customers `api.vendor.example` PROPOSED	They read	They call us	`300 / min`

All 3 use the address above.

These are the limits we allow them.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

They start this one. There is no schedule — the trigger is their call.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Ready. It starts wor

VIEW-17 Aclarian — the connection

`/:tenant/settings/connectors/erp_aclarian` **PROPOSED**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Aclarian

No connector exists yet. This is what the screen shows once one does.

••••7Q2F · reads payments, bills 3 APIs • 2 of 3 working

ERP · They read · Production · connected by Alex Reed

3 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN	RESULT
Read payments `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:12	• Worked
Read bills `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:15	• Worked
Read customers `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:19	• Refused Retry The site refused

Keys we issue

KEY	MAY CALL	ISSUED	STATE
••••7Q2F	Read payments, Read bills +1	12 Aug 2026	In us

⚠ May call is new. Today one key opens every API on the connector — see CR-4.

Recent activity

Support reference | All | + Filter | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Read payments	1,240 of 1,240	8F2B9C1D
Today 04:15	Read bills	1,240 of 1,240	8F2B9C1D
Today 04:19	Read customers	Refused	4C71A0E5

1–3 of 3 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Keys we issue | Disconnect

Runs are kept exactly as they happ

CFG-18 **Workday — set up** /:tenant/settings/connectors/erp_workday/configure NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports

← Connectors

ERP · THEY READ

Workday

Your finance system collects payments and bills from here.

New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Their client id *

What they may read *

payments | bills | customers

Allowed addresses *

The key for them to use

No key yet — save this connector, then issue one.

Issue key

Where it runs

Settings

Production | Sandbox

APIs on this connector 3

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Read payments `api.vendor.example` PROPOSED	They read	They call us	`300 / min`
Read bills `api.vendor.example` PROPOSED	They read	They call us	`300 / min`
Read customers `api.vendor.example` PROPOSED	They read	They call us	`300 / min`

All 3 use the address above.

These are the limits we allow them.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

They start this one. There is no schedule — the trigger is their call.

v2.261.3

Save and test | Cancel

Ready. It starts wor

VB Vijay Bikkad
System Admin

VIEW-18 Workday — the connection

`/:tenant/settings/connectors/erp_workday` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts

← Connectors

CONNECTOR

Workday

No connector exists yet. This is what the screen shows once one does.

••••7Q2F · reads payments, bills 3 APIs • 2 of 3 working

ERP · They read · Production · connected by Alex Reed

Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

3 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Read payments `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:12
Read bills `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:15
Read customers `api.vendor.example` PROPOSED	They read	They call us	300 / min	—

Keys we issue

KEY	MAY CALL	ISSUED	STATE
••••7Q2F	Read payments, Read bills +1	12 Aug 2026	In use

⚠ May call is new. Today one key opens every API on the connector — see CR-4.

Recent activity

Support reference | All | + Filter ▾

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Read payments	1,240 of 1,240	8F2B9C1D
Today 04:15	Read bills	1,240 of 1,240	8F2B9C1D
—	Read customers	—	4C71A0E5

1–3 of 3 · Rows per page 50

Keys we issue | Disconnect

Runs are kept exactly as they happ

CFG-19 **SAP — set up** `/:tenant/settings/connectors/erp_sap/configure` NEW CONNECTOR

SMARTBillHub
Settings
Search

← Connectors

ERP · THEY READ

SAP

Your finance system collects payments and bills from here.

> New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Their client id *

What they may read *

payments | bills | customers

Allowed addresses *

The key for them to use

No key yet — save this connector, then issue one.

Issue key

Where it runs

Production | Sandbox

APIs on this connector 3

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Read payments `api.vendor.example` PROPOSED	They read	They call us	300 / min
Read bills `api.vendor.example` PROPOSED	They read	They call us	300 / min
Read customers `api.vendor.example` PROPOSED	They read	They call us	300 / min

All 3 use the address above.

These are the limits we allow them.

> ⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

They start this one. There is no schedule — the trigger is their call.

VB Vijay Bikkad
System Admin

Save and test | Cancel

Ready. It starts wor

VIEW-19 **SAP — the connection** `/:tenant/settings/connectors/erp_sap` **PROPOSED**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

SAP

No connector exists yet. This is what the screen shows once one does.

••••7Q2F · reads payments, bills 3 APIs • 2 of 3 working

ERP · They read · Production · connected by Alex Reed

3 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Read payments `api.vendor.example` PROPOSED	They read	They call us	300 / min	—
Read bills `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:15
Read customers `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:19

Keys we issue

KEY	MAY CALL	ISSUED	STATE
••••7Q2F	Read payments, Read bills +1	12 Aug 2026	In us

⚠ May call is new. Today one key opens every API on the connector — see CR-4.

Recent activity

Support reference | All | + Filter

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
—	Read payments	—	4C71A0E5
Today 04:15	Read bills	1,240 of 1,240	8F2B9C1D
Today 04:19	Read customers	1,240 of 1,240	8F2B9C1D

1–3 of 3 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Keys we issue | Disconnect

Runs are kept exactly as they happ

CFG-20 **Infor — set up** /:tenant/settings/connectors/erp_infor/configure NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

← Connectors

ERP · THEY READ

Infor

Your finance system collects payments and bills from here.

New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Their client id *

What they may read *

payments | bills | customers

Allowed addresses *

The key for them to use

No key yet — save this connector, then issue one.

Issue key

Where it runs

Production | Sandbox

APIs on this connector 3

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Read payments `api.vendor.example` PROPOSED	They read	They call us	300 / min
Read bills `api.vendor.example` PROPOSED	They read	They call us	300 / min
Read customers `api.vendor.example` PROPOSED	They read	They call us	300 / min

All 3 use the address above.

These are the limits we allow them.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

They start this one. There is no schedule — the trigger is their call.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Ready. It starts wor

VIEW-20 Infor — the connection

`/:tenant/settings/connectors/erp_infor` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing

← Connectors

CONNECTOR

Infor

No connector exists yet. This is what the screen shows once one does.

••••7Q2F · reads payments, bills 3 APIs • 3 of 3 working

ERP · They read · Production · connected by Alex Reed

3 APIs on this connector

Payments

Work

Finance

Runbooks

Reports

Settings

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Read payments `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:12
Read bills `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:15
Read customers `api.vendor.example` PROPOSED	They read	They call us	300 / min	Today 04:19

Keys we issue

KEY	MAY CALL	ISSUED	STATE
••••7Q2F	Read payments, Read bills +1	12 Aug 2026	In use

⚠ May call is new. Today one key opens every API on the connector — see CR-4.

Recent activity

Support reference | All | + Filter ▾

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Read payments	1,240 of 1,240	8F2B9C1D
Today 04:15	Read bills	1,240 of 1,240	8F2B9C1D
Today 04:19	Read customers	1,240 of 1,240	8F2B9C1D

1–3 of 3 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Keys we issue | Disconnect

Runs are kept exactly as they happ

CFG-21 **Microsoft Dynamics GP — set up** `/:tenant/settings/connectors/erp_msgp/configure` NEW CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

← Connectors

ERP · THEY READ

Microsoft Dynamics GP

Your finance system collects payments and bills from here.

New connector. Nothing is built for this vendor today — the fields below are the shape it will take.

Connection details

Their client id *

What they may read *

payments | bills | customers

Allowed addresses *

The key for them to use

No key yet — save this connector, then issue one.

Issue key

Where it runs

Production | Sandbox

APIs on this connector 3

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Read payments `api.vendor.example` PROPOSED	They read	They call us	300 / min
Read bills `api.vendor.example` PROPOSED	They read	They call us	300 / min
Read customers `api.vendor.example` PROPOSED	They read	They call us	300 / min

All 3 use the address above.

These are the limits we allow them.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

They start this one. There is no schedule — the trigger is their call.

Save and test | Cancel

Ready. It starts wor

VIEW-21 **Microsoft Dynamics GP — the connection** `/:tenant/settings/connectors/erp_msgp` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Microsoft Dynamics GP

No connector exists yet. This is what the screen shows once one does.

••••7Q2F · reads payments, bills 3 APIs • 3 of 3 working

ERP · They read · Production · connected by Alex Reed

3 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Read payments `api.vendor.example` PROPOSED	They read	They call us	`300 / min`	Today 04:12
Read bills `api.vendor.example` PROPOSED	They read	They call us	`300 / min`	Today 04:15
Read customers `api.vendor.example` PROPOSED	They read	They call us	`300 / min`	Today 04:19

Keys we issue

KEY	MAY CALL	ISSUED	STATE
••••7Q2F	Read payments, Read bills +1	12 Aug 2026	In us

⚠ May call is new. Today one key opens every API on the connector — see CR-4.

Recent activity

Support reference | All | + Filter ▾

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE

Today 04:12	Read payments	1,240 of 1,240	8F2B9C1D
Today 04:15	Read bills	1,240 of 1,240	8F2B9C1D
Today 04:19	Read customers	1,240 of 1,240	8F2B9C1D

1–3 of 3 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Keys we issue | Disconnect

Runs are kept exactly as they happ

CFG-22 **Accounting export file — set up** `/:tenant/settings/connectors/accounting_file/configure` **PROPOSED FORM**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

ACCOUNTING · WE SEND

Accounting export file

Hand a journal file to your accounts system.

Connection details

What to call it *

Where the file goes *

download | drop

File format *

qbo | xero | csv | iif

Folder host

api.vendor.example

Folder username

Folder path

Folder password

Typed once, never shown again

Stored encrypted. Never shown again.

Where it runs

Production | Sandbox

API on this connector 1

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	TI
Journal export `api.vendor.example`	We send	On demand	`60 / min`	

All 1 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Sign-in on this connector

No token here. This one signs in with the details above, on every call.

⚠ A token is generated on every connector we dial that speaks over an API. This one does not — a file goes to a folder or a download, and neither carries a token. Named rather tha nobody could press.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Saving

VIEW-22 **Accounting export file — the connection** `/:tenant/settings/connectors/accounting_file` **S142, FOR THIS CONNECTOR**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

← Connectors

CONNECTOR

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

Accounting export file

Ottumwa month-end · CSV 1 API • Working

Accounting · We send · Production · connected by Alex Reed

1 API on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Journal export `api.vendor.example`	We send	On demand	`60 / min`	Today 04:12

Key we hold

•••• 7Q2F 12 Aug 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One key. All 1 APIs use it.

⚠ No token on this one — a file goes to a folder or a download, and neither carries a token.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Journal export	1,240 of 1,240	`8F2B9C1D`
Yesterday, 06:00	Journal export	412 of 412	`4C71A0E5`

1–2 of 2 · Rows per page 50

Test all | Replace key | Disconnect

Runs are kept exactly as they happ

CFG-23 **Bank & lockbox file — set up** `/:tenant/settings/connectors/bank_lockbox_file/configure` PROPOSED FORM

SMARTBillHub

Settings

Search

← Connectors

BANKING · WE COLLECT

Bank & lockbox file

Import a statement or lockbox file.

Connection details

What to call it *

File format *

bai2 | camt053 | csv | ofx | qbo | qfx

Where it comes from *

upload | sftp

Folder host

api.vendor.example

Folder username

Folder path

Folder password

Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 2 APIs.

Which column is which

This part appears once a file has been seen.

Where it runs

Production | Sandbox

APIs on this connector 2

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	TI
Bank statement `api.vendor.example`	We collect	On demand	`60 / min`	
Lockbox file `api.vendor.example`	We collect	On demand	`60 / min`	

All 2 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-23 Bank & lockbox file — the connection

`/:tenant/settings/connectors/bank_lockbox_file`

S142, FOR THIS CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Bank & lockbox file

Chase operating · BAI2 | 2 APIs | • Working

Banking · We collect · Production · connected by Alex Reed

2 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Bank statement `api.vendor.example`	We collect	On demand	`60 / min`	Today 04:12
Lockbox file `api.vendor.example`	We collect	On demand	`60 / min`	Today 04:15

Key we hold

••••7Q2F | 12 Aug 2026 | In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One key. All 2 APIs use it.

⚠ No token on this one — a file goes to a folder or a download, and neither carries a token.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Bank statement	1,240 of 1,240	8F2B9C1D
Yesterday, 06:00	Lockbox file	412 of 412	4C71A0E5

1–2 of 2 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Replace key | Disconnect

Runs are kept exactly as they happ

CFG-24 **Print vendor — set up** /:tenant/settings/connectors/print_vendor/configure NEEDS A VENDOR NAME

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance

← Connectors

PRINT · WE SEND

Print vendor

Hand the printed batch to a mailing house.

Still generic. You have not named a vendor for this one — it keeps a category name until you do.

Connection details

API key *

Typed once, never shown again

Stored encrypted. Never shown again.

Your account with them *

Where it runs

Production | Sandbox

Runbooks

Reports

Settings

API on this connector 1

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T
Print hand-over `api.vendor.example`	We send	When queued	`120 / min`	

All 1 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

Token

We generate one token for this connector and every API authenticates with it.

Token address *

api.vendor.example

Scope

Leave blank for everything the key allows

No token yet — save this connector, then generate one.

Generate token

One token. All 1 API use it. It is shown once and afterwards only regenerated.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-24 **Print vendor — the connection** `/:tenant/settings/connectors/print_vendor` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

← Connectors

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

CONNECTOR

Print vendor

No connector exists yet. This is what the screen shows once one does.

Account 88214 1 API • Working

Print · We send · Production · connected by Alex Reed

1 API on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Print hand-over `api.vendor.example`	We send	When queued	`120 / min`	Today 04:12

Token we hold

••••7Q2F generated 12 Aug 2026 expires 11 Nov 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One token. All 1 API use it.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today 04:12	Print hand-over	1,240 of 1,240	`8F2B9C1D`

1–1 of 1 · Rows per page 50

Test all | Regenerate token | Disconnect

Runs are kept exactly as they happ

CFG-25 **Esri ArcGIS — set up** `/:tenant/settings/connectors/gis_esri/configure` RENAMED

SMARTBillHub

Settings

Search

← Connectors

MAPS · WE COLLECT

Esri ArcGIS

Find an address and keep parcels in step.

> Renamed. Ships today under a generic name. The connector, its fields and its operations do not change.

Connection details

Service address *
api.vendor.example

Map tile server
api.vendor.example

API key
Typed once, never shown again

Stored encrypted. Never shown again. This key is used by all 3 APIs.

Where it runs

Production | Sandbox

APIs on this connector 3

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Address search `gis.ottumwa.example`	We collect	While you wait	`600 / min`
Reconcile parcels `gis.ottumwa.example`	We collect	Nightly	`10 / min`
Map tiles `tile.arcgisonline.example` ITS OWN HOST	We collect	While you wait	`600 / min`

1 of these calls a different address. Only that one is editable.

These are the limits they allow us.

> ⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

When it runs

1 of 3 run on a clock. The rest are started by a person, by a queue, or by them.

Time

01:00

Your time zone. The only part you choose.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-25 **Esri ArcGIS — the connection** `/:tenant/settings/connectors/gis_esri` **S142, FOR THIS CONNECTOR**

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Connectors

CONNECTOR

Esri ArcGIS

gis.ottumwa.example 3 APIs • Working

Maps · We collect · Production · connected by Alex Reed

3 APIs on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN	RESULT
Address search `gis.ottumwa.example`	We collect	While you wait	`600 / min`	Today 04:12	• Refused Retry The site refu
Reconcile parcels `gis.ottumwa.example`	We collect	Nightly	`10 / min`	Today 04:15	• Worked
Map tiles `tile.arcgisonline.example` 2ND HOST	We collect	While you wait	`600 / min`	Today 04:19	• Worked

Key we hold

••••7Q2F 12 Aug 2026 In use

Only the last four characters are ever shown. It cannot be read back, by anyone, including us.

One key. All 3 APIs use it.

⚠ No token on this one — a file goes to a folder or a download, and neither carries a token.

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Address search	1,240 of 1,240	8F2B9C1D
Yesterday, 06:00	Reconcile parcels	412 of 412	4C71A0E5

1–2 of 2 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Replace key | Disconnect

Runs are kept exactly as they happ

CFG-26 **Census geocoder — set up** `/:tenant/settings/connectors/geocoder_census/configure` PROPOSED FORM

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks

← Connectors

MAPS · WE COLLECT

Census geocoder

Turn an address into a point.

Connection details

Service address *

api.vendor.example

Where it runs

Production | Sandbox

API on this connector 1

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T

Reports

Settings

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT
Geocode addresses `api.vendor.example`	We collect	On demand	`60 / min`

All 1 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel | Saving

VIEW-26 Census geocoder — the connection

`/:tenant/settings/connectors/geocoder_census`

S142, FOR THIS CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance

← Connectors

CONNECTOR

Census geocoder

geocoding.geo.census.gov | 1 API | • Working

Maps · We collect · Production · connected by Alex Reed

1 API on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Geocode addresses `api.vendor.example`	We collect	On demand	`60 / min`	Today 04:12

Sign-in on this connector

No password is stored for this connector.

Runbooks

Reports

Settings

Recent activity

Support reference | All | + Filter

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Geocode addresses	1,240 of 1,240	8F2B9C1D
Yesterday, 06:00	Geocode addresses	412 of 412	4C71A0E5

1–2 of 2 · Rows per page 50

v2.261.3

VB Vijay Bikkad
System Admin

Test all | Disconnect

Runs are kept exactly as they happ

CFG-27 **Public map tiles — set up** `/:tenant/settings/connectors/map_tiles_public/configure` PROPOSED FORM

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard

Plans & Rates

Premises

Assets

Accounts

Reads

Billing

Payments

Work

Finance

Runbooks

← Connectors

MAPS · WE COLLECT

Public map tiles

The background picture under the map.

Connection details

Tile server *

api.vendor.example

Where it runs

Production | Sandbox

API on this connector 1

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T

Reports

Settings

Map tiles `api.vendor.example`	We collect	While you wait	`600 / min`

All 1 use the address above.

These are the limits they allow us.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

When it runs

Nothing here runs on a clock. Every API on this connector is started by a person, by a queue, or by them.

⚠ Whether an API is scheduled is the vendor's, not yours — it ships with the connector. And nothing acts on the time today, see RAN-2.

v2.261.3

Save and test | Cancel | Saving

VB Vijay Bikkad
System Admin

VIEW-27 Public map tiles — the connection

`/:tenant/settings/connectors/map_tiles_public`

S142, FOR THIS CONNECTOR

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks

← Connectors

CONNECTOR

Public map tiles

server.arcgisonline.com | 1 API | • Working

Maps · We collect · Production · connected by Alex Reed

1 API on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN
Map tiles `api.vendor.example`	We collect	While you wait	`600 / min`	Today 04:12

Sign-in on this connector

No password is stored for this connector.

Reports

Settings

v2.261.3

VB Vijay Bikkad
System Admin

Recent activity

Support reference | All | + Filter ▾ | ▦

WHEN ▲	WHICH API	RESULT	SUPPORT REFERENCE
Today, 06:02	Map tiles	1,240 of 1,240	8F2B9C1D
Yesterday, 06:00	Map tiles	412 of 412	4C71A0E5

1–2 of 2 · Rows per page 50

Test all | Disconnect

Runs are kept exactly as they happ

CFG-28 **Single sign-on — set up** `/:tenant/settings/connectors/sso/configure` NEEDS A VENDOR NAME

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports

← Connectors

SIGN-IN · THEY SEND

Single sign-on

Staff sign in with your own identity provider.

Still generic. You have not named a vendor for this one — it keeps a category name until you do.

Coming soon · v2 — not built yet.

Connection details

Identity provider address *

api.vendor.example

You will be sent to your identity provider and brought back here. We never see or store your password.

Sign in with your identity provider

Settings

Where it runs

Production | Sandbox

API on this connector 1

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	T
Sign in `api.vendor.example`	They send	While you wait	`600 / min`	

All 1 use the address above.

These are the limits we allow them.

⚠ The API list and its rate limits are set when we configure this connector — neither is a clerk control. See CR-3 and CR-11.

They start this one. There is no schedule — the trigger is their call.

v2.261.3

VB Vijay Bikkad
System Admin

Save and test | Cancel

Signing in is all ther

VIEW-28 Single sign-on — the connection `/:tenant/settings/connectors/sso` PROPOSED

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing

← Connectors

CONNECTOR

Single sign-on

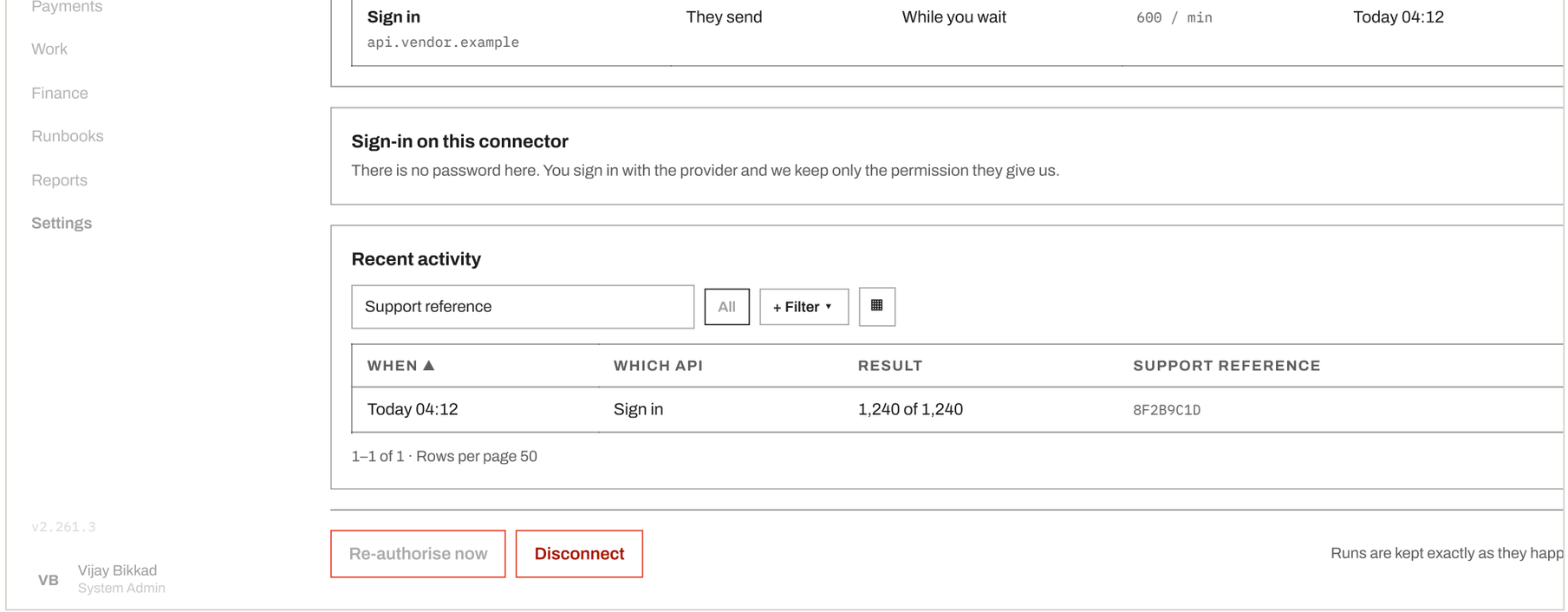
No connector exists yet. This is what the screen shows once one does.

idp.ottumwa-water.example 1 API • Working

Sign-in · They send · Production · connected by Alex Reed

1 API on this connector

API · ENDPOINT	DIRECTION	WHEN	RATE LIMIT	LAST RUN



S141-1 **The "Nightly" a utility picks here is the cadence nothing fires** THE GAP, ON SCREEN

How often — Nightly · Hourly · Manual is the third row of every connect modal, defaulted to Nightly. It is accepted, validated, saved to integration_connection.cadence, and nothing will ever read it (RAN-2).

This is where the scheduler gap becomes a promise to a customer. A utility connects its bank file on Monday, chooses Nightly, and waits. Nothing on this screen, the card, or the connection detail says the choice is inert.

It is one page, not a wizard — and I checked rather than assumed

The specification describes a row stack and I had read that as steps. There is no step index, no Next, no Back. Four draw states — connect, connect-with-redirect, manage, manage-replacing — plus a disconnect confirm rendered as a second body in the same shell. So R6 asks for one drawing here, not six, and the field slot is the only part that varies.

Two slots the pack draws are deliberately empty here

Template I3 has a WarningBox (a sign-in about to lapse) and a BudgetLine (Calls used this week — 4,180 of 10,000). Neither renders. The layout document stamps it: waves H and G own them, "and a box that never fills is law 2 wearing a layout". The budget line is the one that bites — when an allowance runs out, collection stops, and the pack's own note says that without it "the connection simply looks quiet, and the first sign is an estimated bill three weeks later." That is the same silence as the scheduler gap, one screen down.

S142-1 The sort arrow never appears until you click

SMALL, REAL, CHEAP

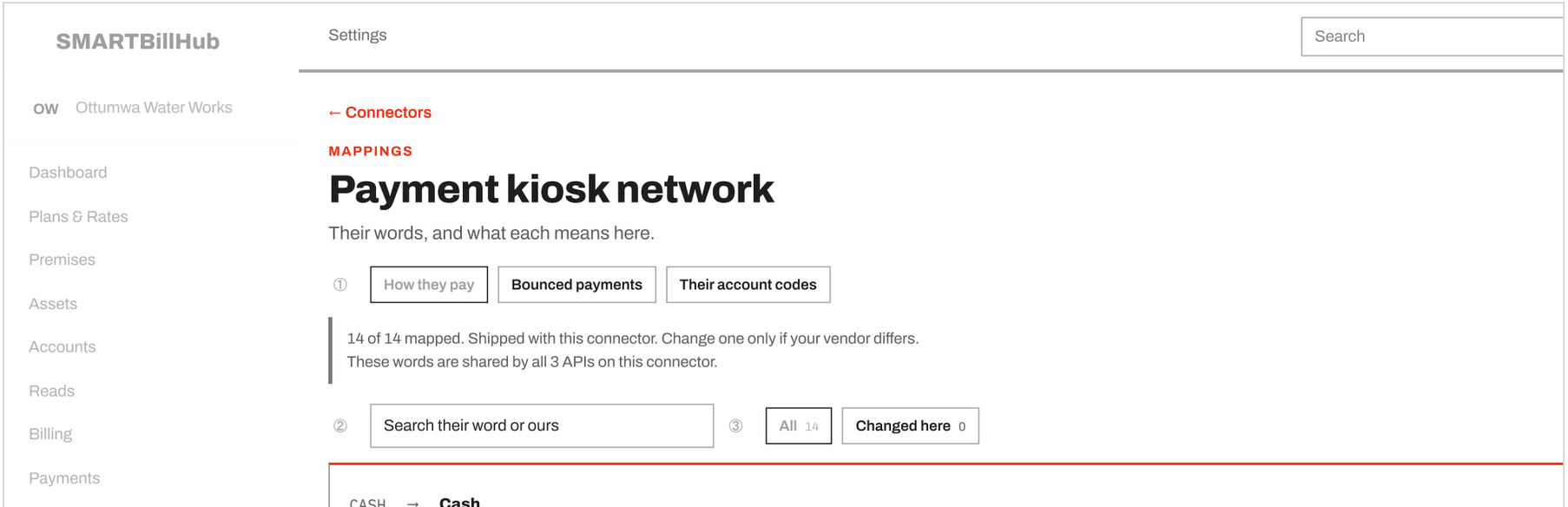
The runs table declares `defaultSort: {sort:"started"}` but the column's key is `"when"`. On first paint the two do not match, so no ▲/▼ is drawn on the only sortable column — the table is sorted, and does not say so.

> `ConnectionDetailPage.tsx:515 — defaultSort: { sort: "started", dir: "desc" }`
>
> `:497-528 — the column key is when`

MAP-01 Mappings — their words for our things

`/:tenant/settings/connectors/:key/mappings`

DRAWN TO THE RULES



Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

CASH → **Cash**
CHECK → **Check**
ACHCHECK → **Bank transfer**
ACHSAVINGS → **Bank transfer**
VISA → **Card**

⑤ 1–5 of 14 · Rows per page 50

A word they send that is not on this list still lands — it appears here unmapped, in red.

⚠ The answers ship with the connector. Our seven tenders are seeded identically into every tenant, so ACHCHECK → Bank transfer is the same at every utility. See CR-6.

MAP-02 Mappings — the ones only you can answer

`/:tenant/settings/connectors/erp_aclarian/mappings`

DRAWN TO THE RULES

SMARTBillHub

Settings

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

← Connectors

MAPPINGS

Aclarian

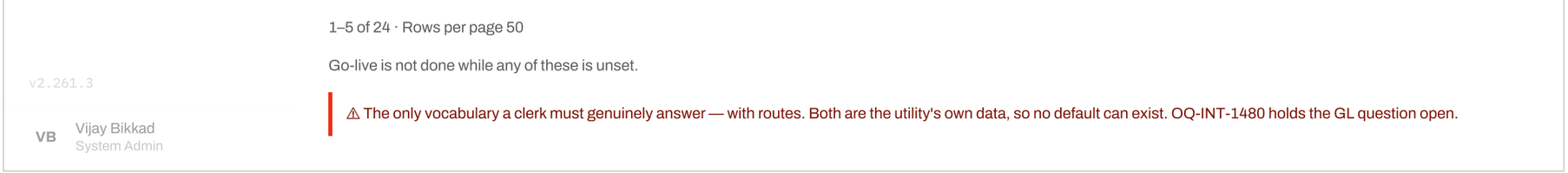
Their words, and what each means here.

Their account codes | How they pay

9 of 24 not mapped. Each posts to Suspense until mapped.
These are your ledger codes, not theirs. Nobody can ship these for you.

Search their code or ours | All 24 | Unmapped 9

001.100.41100 → **Water revenue**
001.100.41200 → **Sewer revenue**
001.100.41900 → **Not set · posts to Suspense**
001.200.22100 → **Not set · posts to Suspense**
001.200.22300 → **Not set · posts to Suspense**



Each legend line carries four things and no more: **what it is · where its data comes from · which rule it satisfies · new, changed or unchanged.**

#	WHAT IT IS	DATA	RULE	FLAG
①	Kind selector, square — the kit's ChipToggle	payload.types, unchanged	--radius-*: 0px · B14 rule 36 (one kit)	Changed
②	Search box, the kit's .input	client filter over the loaded page, unchanged	UI-RULES 12 (toolbar)	Changed
③	Unmapped as a chip, not a checkbox, with its count	count from payload.counts.unmapped — already served	UI-RULES 12 (filter chips)	Changed
④	Column selector	local, per-viewer	UI-RULES 14	New
⑤	Pager	**NOT SERVED** — the endpoint returns every row today	UI-RULES 16 · NFR-3	New

> **⑤ makes this a back-end change wearing a wireframe**
>
> The pager is drawn, and nothing serves it. Adding it means the mappings endpoint learns `page` and `per` and orders in SQL. Saying so here is the point — a drawn field with no source named is how a drawing becomes a promise nobody costed.

Carry-over. A remedy for an existing screen never loses anything — the subtraction proves it.

AS-BUILT ELEMENT	DISPOSITION
Back link · Kicker · Title · Intro sentence	Unchanged ×4
Kind selector (pills)	Relabelled — same words, square control
Unmapped alert box	Unchanged

Search input	Moved — into the toolbar
"Unmapped only" checkbox	Relabelled — becomes a chip with a count
Mapping rows · row edit control	Unchanged ×2
Footer note	Unchanged
Column selector · Pager	New ×2

```
As-built  10 = unchanged 7 + moved 1 + relabelled 2 + removed 0
Remedy    12 = 10 − 0 removed + 2 new          reconciles
```

Nothing is removed, deliberately

The checkbox and the pills both survive as controls, changed in form and not in function. A removal with no quoted ruling would be a defect in this document, so the disposition stays relabelled even where a tidier screen might have dropped one.

THE PACK SPECIFIES (FRAME 45.1, TEMPLATE I5)

- A tab strip: How they pay · Why a payment bounced · Routes
- A full red AlertBox with three sentences of reasoning
- Each row carries a [Change] / [Map it] button with a word on it
- A closing NoteBox in a bordered panel
- A second tab, Waiting for a decision — the GIS queue (template I10)

WHAT SHIPS

- Rounded pills, not tabs — and only when more than one kind exists
- A one-line left-ruled box: "2 of 14 not mapped. Each records as Other until mapped."
- Each row carries an icon only (✎), named Map it / Change to a screen reader
- A plain footnote, no panel
- The second tab is NOT PRESENT — a stated absence in the file's own header

S143-1 **This screen draws rounded pills in a product with no rounded corners** TOKEN VIOLATION

tokens.css sets --radius-sm / md / lg: 0px — every corner in this product is square on purpose. This screen's kind selector is the one place that breaks it, with a hand-rolled rounded-full rather than the kit's ChipToggle that every other screen uses.

Radius is the fastest tell that a screen was built to its own taste rather than to the kit — and it is a one-line fix that also deletes a bespoke component.

> `ValueMappingPage.tsx:339-340 — rounded-full border … ×2`
>
> `Only 2 rounded-* uses in all of features/integrations/, both here`
>
> `IntegrationsTab.tsx:580 uses the kit's ChipRow / ChipToggle for the same job`

S143-2 One sentence does duty for "nothing exists" and "your filter found nothing"

ROUTE: DEFECT

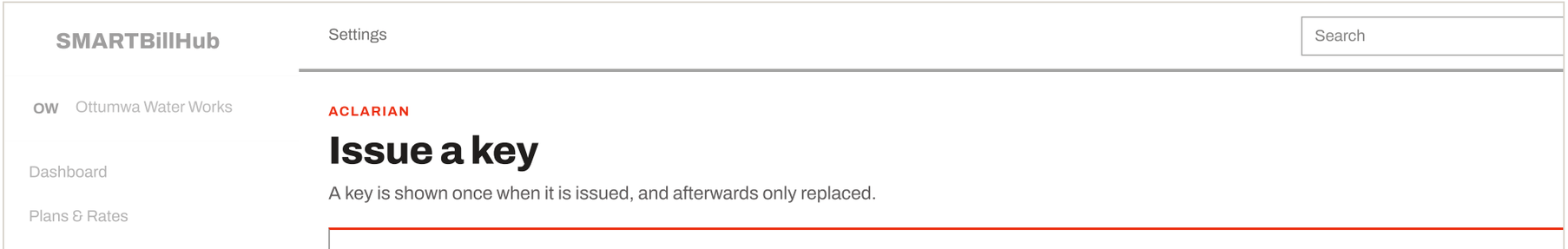
Type a word that matches nothing and the screen says "Nothing recorded yet." — the same string it shows when the vendor has genuinely sent no values. There is no zero-results state and no Clear-filters affordance on this screen, unlike every other list in the product.

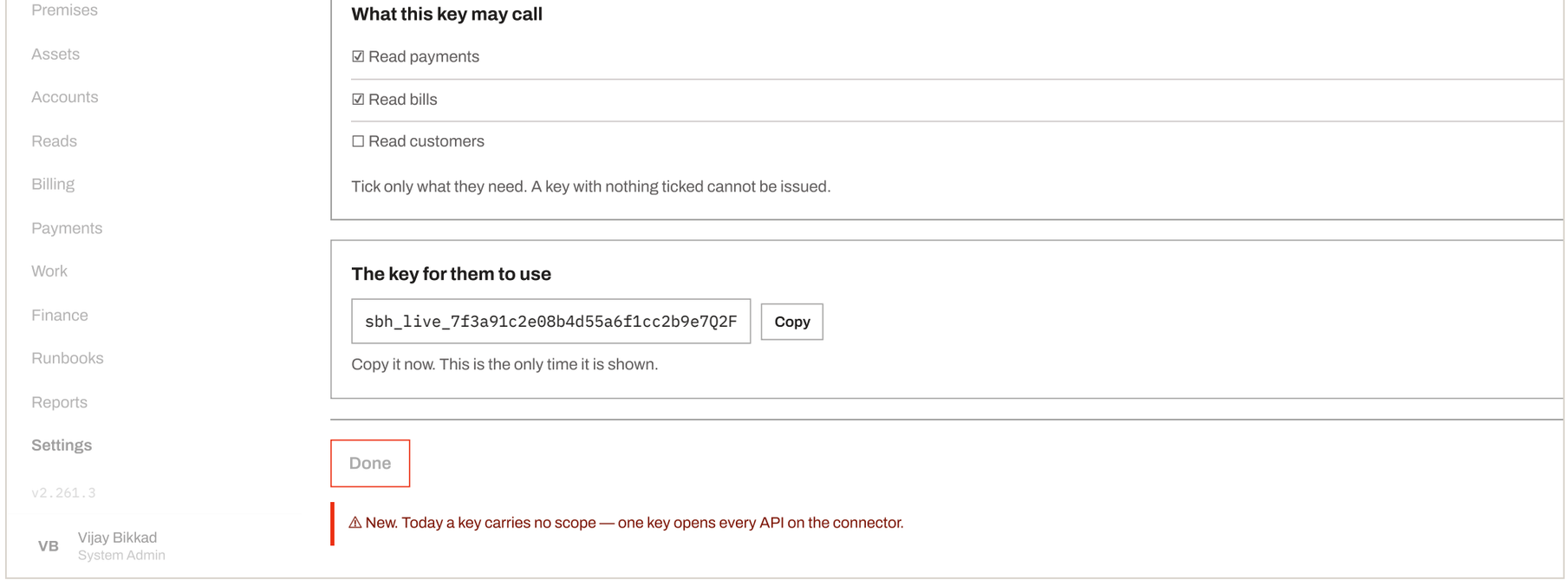
> `ValueMappingPage.tsx:398 — integrations.mappings.empty for both branches`

And this screen is the overlap itself

Deciding that ACHCHECK means Bank transfer is business logic by your own ruling — and it lives in apps/integrations/mappings.py, 537 lines, with this clerk screen on top. About three people-weeks sit on the wrong side of the line. It is OQ-INT-1727, it offers three options, and it is still open eight days on. Option (c) — "Runbooks reads them across a seam" — is not what is built: runbooks does not reference the mappings at all.

KEY-01 Issue a key `/:tenant/settings/connectors/:key/access` DRAWN TO THE RULES





The one item on this list with a security shape. Today issue_key() takes a connection and nothing else, and one credential is active per connection.

> S144-1 **My earlier drawing put the access log on this screen. It is not here.** MY ERROR, CORRECTED
>
> Passes 1 and 2 drew "Who has been reading" as the screen at /access, because frame 46.2 titles it S119 WHO HAS BEEN READING and the pack maps S119 → S144. That is the pack's intent, not the build.
>
> In the product AccessLogPanel is mounted only by ConnectionDetailPage, as a panel near the bottom of S142. The keys screen is a key table and nothing else. The two halves of one drawn screen shipped to two different addresses, and the pack's own note says the access-log half was wave S and landed on the connection page instead.
>
> > `ConnectionDetailPage.tsx:77, :1183 — the only non-test import and mount`
> >
> > `MachineKeysPage.tsx:41-43 — records that the access-log half went elsewhere`

ACC-01 **Who has been reading** `a panel inside the connection view` DRAWN TO THE RULES

Who has been reading

An access log, not a run history

① **LOADING**

Loading…

② **EMPTY**

Nobody has read anything

This partner has a key and has not used it.

③ **FAILED**

The access log did not load

We cannot say whether anybody has read anything.

Retry

⑤ **LOADED — AND IT NOW SAYS WHICH API**

WHEN	WHICH API	ROWS	RESULT
Today, 06:14	Account lookup	1,240	• Worked
Today, 06:02	Payments	318	• Worked
Yesterday, 22:40	Returns	—	• Refused

⚠ Today this panel says who read, never what. One connector, three APIs, one line.

WATCH · Why this screen exists

Nothing else in the product would show a partner taking two years of history at two in the morning.

④ Allowed addresses | Revoke the key

#	WHAT IT IS	DATA	RULE	FLAG
①	LoadingSkeleton while the query is in flight	logQuery.isLoading — already available, unread	UI-RULES 17 · the kit's own state set	New
②	An empty state that claims nothing about failure	items.length === 0 && !isError	B14 rule 32 (a stated absence is closed)	Changed

	failure		is closed)	
③	A failure state that says we do not know, with Retry	logQuery.isError — available, unread	UI-RULES 17 · B14 rule 33 (six words)	New
④	The pack's two buttons, built	POST …/access/allowlist, POST …/keys/revoke — the revoke endpoint exists; the allowlist one does not	frame 46.2 · template I6	New

```
As-built   6 = unchanged 5 + relabelled 1 + removed 0
Remedy    10 = 6 − 0 removed + 4 new            reconciles
```

Why ② is the whole fix

Today one sentence covers three states, and the sentence is the reassuring one. A security screen that says "all clear" when its request failed is worse than one that shows nothing — a reader who sees a blank table goes looking; a reader who sees "Nobody has collected anything yet." stops. That is the difference the remedy buys, and it needs no new data at all.

S142-2 **The access log cannot tell "still loading" from "nobody has read anything"**

ROUTE: DEFECT

This panel has no loading state and no error state. While the request is in flight it renders "Nobody has collected anything yet." — and it renders the same sentence if the request fails. On a security screen whose entire purpose is to show unusual access, a failed fetch is indistinguishable from a clean record.

The pack also draws two buttons here — Allowed addresses and Revoke the key — and neither is built; the file records that as deliberate. So the screen that exists to raise an alarm has no way to act on one.

> `AccessLogPanel.tsx:178 — logQuery.data?.items ?? [], no isLoading branch`
>
> `:181 — no isError branch anywhere in the component`

RB The runbook screens

Twenty-three drawings on four tabs — and they are a different product from the sixty

Twenty-three drawings on four tabs — and they are a different product from the sixty above. A connector holds a connection and a key. A runbook holds the business logic: what to check, in what order, what a person must approve, what gets written. That is the line you ruled, and this is the other side of it.

> **This is a change to the pack, not a fix to a miss — raise it as a CR**
>
> The runbook screens ship today as buttons and back-links, and the blueprint pack draws them that way too. Putting them on tabs is therefore a change to a signed drawing, and merging the home screen into the list is part of the same change rather than a second one. It needs a change request before it is built, and nothing here should be read as agreed.
>
> **What the change removes:**
>
> RBS-001, the runbooks home — a real blueprint screen with three frames. Its four counters move to the top of the list.

> **Four tabs, and running is deliberately not one of them**
>
> **Runbooks · Templates · Work list · Run history.**
>
> The lifecycle is five stages and four of them are a place: pick a template, set it up, run it, act on what it found, look back. Running is a verb, not a place — giving it a tab is what produced the old home screen, whose only job was to link to four siblings.
>
> Templates nearly lost its tab — you add a runbook once, and a tab is for somewhere you return to. It keeps one because your own flow starts there and 28 templates with filters and a detail page is a place. Below the tabs: the setup screen is drawn once per sub-tab — all seven, the run wizard frame by frame — all eight, plus the template path and the run result.

> **The setup screen is seven screens, and the Templates tab is a path**
>
> RBS-004 carries seven sub-tabs and each one is drawn. A sub-tab has no address of its own, which is exactly why nothing checks it and why five of the seven had never been drawn at all.
>
> Five are drawn on Can we bill? — it is the runbook the seed script populates, so its Runs and Activity tabs hold real rows. Guards and Limits are drawn on Collections ladder.

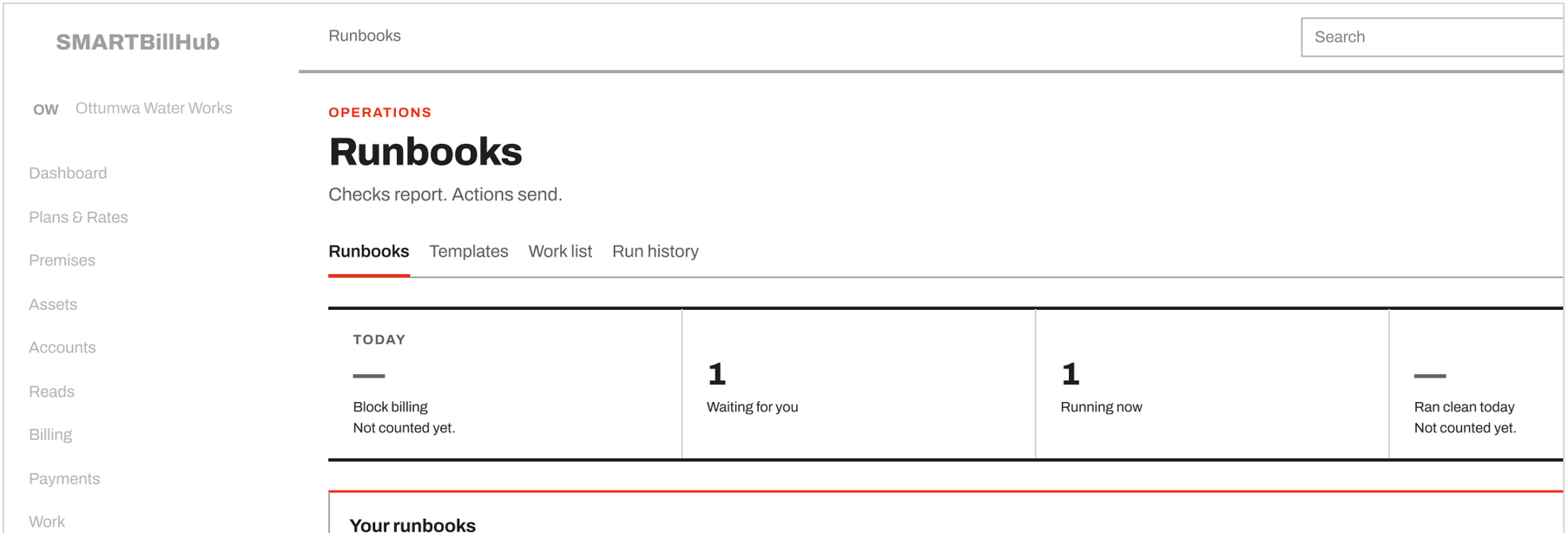
and Activity tabs hold real rows. Guards and Limits are drawn on Collections ladder, because a Check has no guards and never waits for approval: drawing those two tabs on a Check would have produced two empty screens that teach nothing. Each of the seven is drawn once.

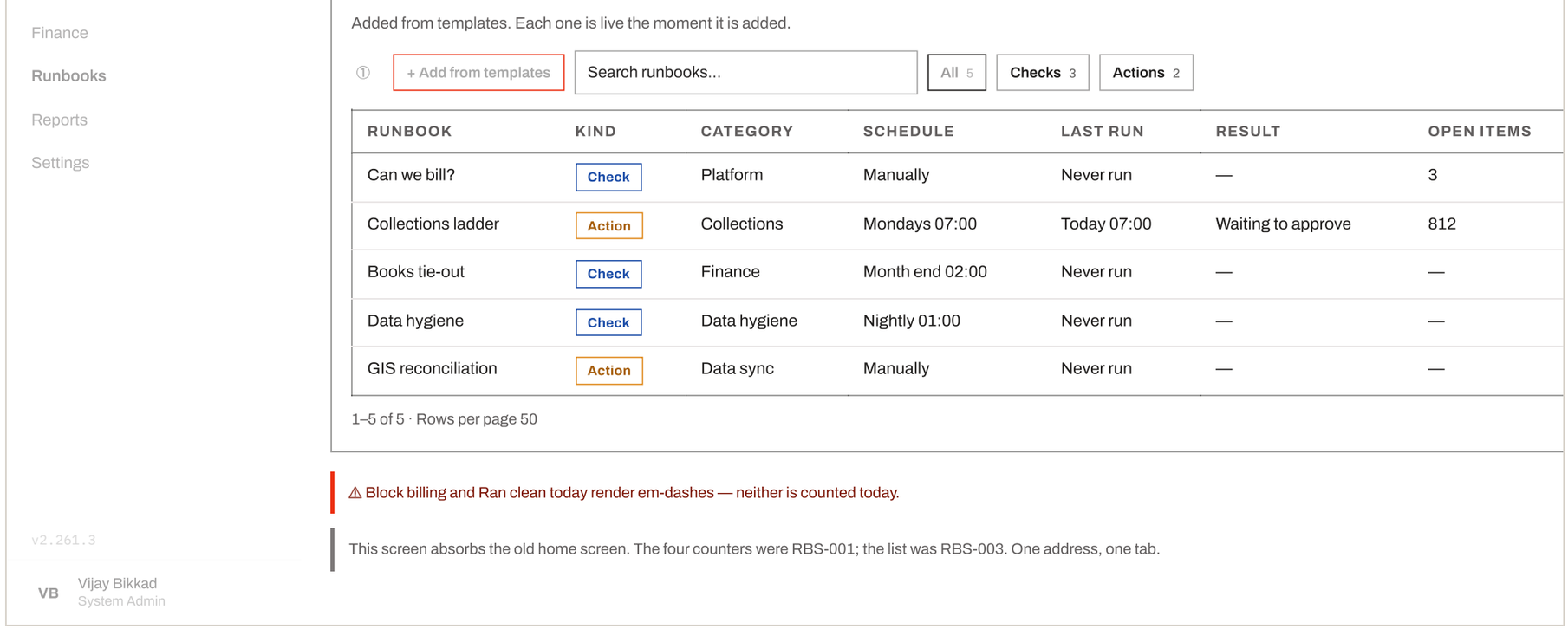
And the Templates tab now draws all four steps of the path: Templates → Review → Add → Set up. Step 4 is a moment rather than a route — the product posts and redirects in the same breath — so it is drawn as what the call makes and who is told, which is nobody. Step 5 is the set-up screen in its day-one state: v1, never run, nothing scheduled. Reviewing a template step by step before adding it is the half that is not built — the shipped detail page lists step names and nothing else.

The ids are the register's, not this document's

RBS-00n is the register id, ruled 2026-09-06 — every build sheet in the pack says so in the same sentence: "The document numbers this screen S151.1; this repo numbers it RBS-005.1." So these are real ids, unlike the connector screens above.

RBS-003 **Runbooks — the list, and today** `/:tenant/runbooks` DRAWN TO THE RULES





THE PACK SPECIFIES (RBS-001.1)

- KPIs read 3 · 1 · 1 · 6, coloured red / amber / ink / green
- Work list → drawn as a live, emphasised button
- Outcome column shows 3 blockers · Priced · Balanced · 248 queued
- Detail column reads 812 planned · cap is 400
- A sort chevron on Finished; row actions 👁 ▶ ⋯

WHAT SHIPS

- KPIs 1 and 4 are em-dashes with "Not counted yet." beneath
- Work list → is disabled, tooltip "Not built yet."
- Outcome shows the run's state word — Done / Failed / Stopped — not its finding
- Detail reads "Records 812"
- No sorting anywhere; one row action (👁 View)

RBS-1 Half the dashboard is an em-dash, and the two that work are the two that matter least

ROUTE: DEFECT

Block billing and Ran clean today both render — with "Not counted yet." They are the two a clerk actually opens this screen for: is anything stopping the bill run and did last night go well. The two that do carry numbers — Waiting for you and Running now — are the ones you can also see from the table beneath.

The dash is honest, not broken: a zero would claim nothing is blocking billing, and nobody has counted. But the screen's whole promise is "a clerk presses one button and sees exactly what is stopping billing", and today it says it does not know.

> `runbooksHomeModel.ts:73-84 — blockBilling and ranCleanToday resolve to null`

> `RunbooksHome.tsx:236-250 — null renders Dash + runbooks.home.kpi.notCounted`

RBS-5

Three of the eight columns say "Never run" on every row, and the eye icon is disabled on all of them

ROUTE: DEFECT

Last run, Result and Open items are the columns a clerk scans this table for. Today Last run and Result read "Never run" on every row — nothing has ever run — and the View last run row action is disabled on all of them with the same words.

This is honest and it is also three-eighths of a table carrying no information. It resolves itself the day anything runs, which is the day the scheduler gap (RAN-2) is closed — so it is not a separate fix, it is a symptom of the same one.

RBS-6 **This screen has exactly one inbound link, and it is on an empty state** DOOR: CONDITIONAL

The only link to /runbooks/mine anywhere in the front end is the empty-state CTA on Run history. Once a tenant has any run history at all, that empty state never renders — and My runbooks has no door.

Worse, the back link on Runbook setup is labelled "My runbooks" and navigates to the Runbooks home — so the one control that names this screen does not go to it.

> `RunHistoryPage.tsx:90 → RunHistory.tsx:289 — onCta, empty state only`

> `Not in the nav · not on the Runbooks home · not in the template library`

> `RunbookSetup.tsx:172 — label runbooks.setup.back = "My runbooks", target tenantPathFrom(tenant, "runbooks")`

RBS-004 **Runbook setup — Overview** `/:tenant/runbooks/:key` **DRAWN TO THE RULES**

SMARTBillHub

Runbooks

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

← Runbooks

Can we bill?

Check | Platform

Pau

Overview | Steps 10 | Scope | Guards | Limits | Runs 51 | Activity

Last run

When	Today 06:02
Result	• Worked
Took	12s

This can

Write anything	No
Raise work items	Yes
Send anything	No

Version

Current	
Last edited	
Dry run first	

Schedule ②

Saved as a new version

Runs

Manually | On a schedule | On an event

Time zone

America/Chicago

From the utility. Not typed here.

When

Daily | Weekly | Monthly

Mondays ∨ 07:00

Your time zone, not the server's.

Next

Mon 21 Sep, 07:00

Computed. Never typed.

⚠ New. Today this panel renders three facts and has no form — so Next can only read Not set. And the time zone defaults to UTC rather than the utility's, so a job set for 01:00 fires at 7 — see CR-7.

Can we bill? is the exemplar for five of the seven tabs because it is the one the seed script populates — 51 real runs, written through the product's own open_run.

The seven tabs of RBS-004, enumerated from the tab configuration rather than the router — a tab has no address of its own, which is exactly why nothing checks it.

TAB	URL	WHAT ITS PANEL RENDERS TODAY	STATE
Overview	`#overview`	Three blocks — Schedule · Last run · This can. Next is always "Not set"; This can says "Nothing is written."	**Mostly absent**
Steps	`#steps`	A numbered list of the runbook's steps, each tagged writes or read. Every marker is "waiting".	**Real**
Scope	`#scope`	Parameter rows + Change scope (live, opens a modal). "Matches {{total}} accounts today" is never drawn — the count is always null.	**A fact is missing**
Guards	`#guards`	One row per guard, all reading Never touched, and a permanently disabled Edit with "Guards cannot be switched off here."	Deliberate
Limits	`#limits`	Most per run · Approval · Fees · Who can approve, plus Change limits (live).	**Real**
Runs	`#runs`	A table — When · Outcome · Acted. Empty on every runbook today; its CTA is an enabled button that does nothing.	**Dead control**
Activity	`#activity`	A version history in sentences — Cap raised to 400, Paused, Scope changed.	**Real**

RBS-2 Run ▶ is enabled, and answers "Not connected yet."

ROUTE: DEFECT

The primary action on every runbook is live, styled as the main thing to do, and pressing it raises a toast reading "Not connected yet." Nothing on the screen says so beforehand.

This is the mirror of the work-list button, and the worse half: a disabled control tells you before you invest anything; an enabled one takes the click and then refuses. The Runs tab has the same shape one level down — an empty-state CTA with no reason attached, so it draws as an ordinary enabled button wired to a no-op.

> `RunbookSetupPage.tsx:178, :210 — onRun = toast runbooks.setup.notConnected`

> `RunbookSetup.tsx:597-606 — Runs-tab CTA has no ctaReason, so EmptyList draws it enabled`

RBS-004 ▸ Steps

Runbook setup — Steps

`/:tenant/runbooks/:key#steps`

DRAWN TO THE RULES

SMARTBillHub

Runbooks

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

← Runbooks

Can we bill?

Check | Platform

Overview | Steps 10 | Scope | Guards | Limits | Runs 51 | Activity

What it does, in order

A step either looks at the whole set or one record at a time. Only a step that writes can change anything.

#	STEP	LOOKS AT	WRITES
1	Services and units	Every account	No
2	Reference data	Every account	No
3	Tax	Every account	No
4	Plans in force	Every account	No
5	Rate coverage	Every account	No
6	Account chain	Every account	No
7	Meters	Every account	No
8	Reads and windows	Every account	No
9	Documents	Every account	No
10	Books	Every account	No

Ten steps, none of them a writer. That is what makes this a Check rather than an Action.

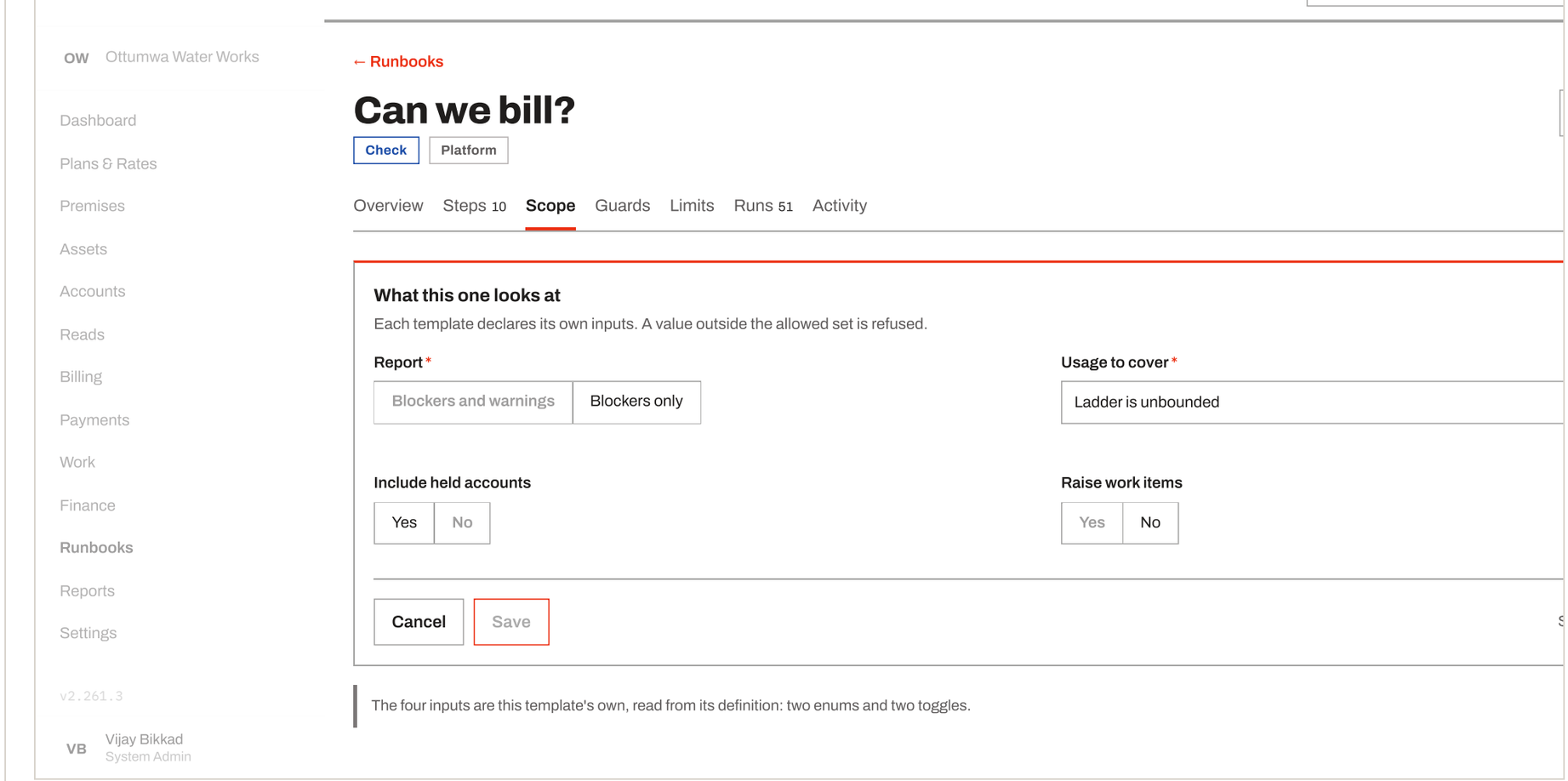
> Steps are declared on the template, in code. A tenant does not reorder or remove one — that is what cloning is for.

RBS-004 ▸ Scope **Runbook setup — Scope, the inputs you set** `/:tenant/runbooks/:key#scope` DRAWN TO THE RULES

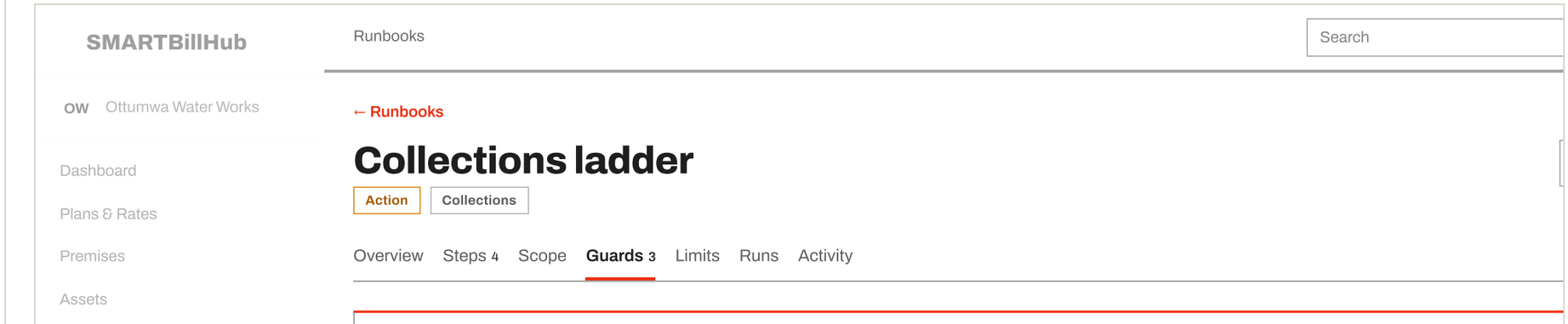
SMARTBillHub

Runbooks

Search



RBS-004 ▸ Guards **Runbook setup — Guards** `/:tenant/runbooks/:key#guards` DRAWN TO THE RULES



Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings
v2.261.3
VB Vijay Bikkad
System Admin

Who this never touches

A guard removes a subject before any step runs. It is not a filter you can override.

GUARD	WHEN	LEFT OUT LAST RUN
Never include: Sample	Never	12
Never include: Medical	Never	3
Never include: Dispute	Never	7

Guards come from the template and cannot be switched off here. To change them, clone it.

⚠ This tab is drawn on Collections ladder, not Can we bill? — a Check with no guards would draw an empty tab and teach nothing.

Three guards, read from the definition. An excluded subject is not reported as skipped — it is never a subject.

RBS-004 ▸ Limits **Runbook setup — Limits and approval** `/:tenant/runbooks/:key#limits` DRAWN TO THE RULES

SMARTBillHub
Runbooks
Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks

← Runbooks

Collections ladder

Action Collections

Overview Steps 4 Scope Guards 3 Limits Runs Activity

How far it may go

Two settings, and both are saved as a new version.

Most per run

400

Leave it blank for no limit. Over it, the run fails rather than doing half the work.

Approval

Approve each run | Approved up front

An Action waits here. A Check never does.

Kind

Edited since its last clean dry run

Reports

Settings

v2.261.3

VB Vijay Bikkad
System Admin

So it may run

Dry only, until o

Cancel | Save

This tab is live today — cap and approval are two of the four things the setup screen can already save. The dry-run ratchet below them is enforced by the server, not by this screen.

RBS-004 ▸ Runs

Runbook setup — Runs

`/:tenant/runbooks/:key#runs`

DRAWN TO THE RULES

SMARTBillHub

Runbooks

Search

OW Ottumwa Water Works

Dashboard

Plans & Rates

Premises

Assets

Accounts

Reads

Billing

Payments

Work

Finance

Runbooks

Reports

Settings

v2.261.3

VB Vijay Bikkad
System Admin

← Runbooks

Can we bill?

Check | Platform

Overview | Steps 10 | Scope | Guards | Limits | Runs 51 | Activity

Every run of this one

Newest first. A run is kept exactly as it happened and is never edited.

Search runs... | All 51 | Failed 2

STARTED	TRIGGER	MODE	OUTCOME	CHECKED	TOOK
Today, 06:02	A schedule	Dry	• Worked	1,204	12s
Yesterday, 06:02	A schedule	Dry	• Worked	1,198	11s
12 Sep, 06:02	A schedule	Dry	• Failed	—	3s

1–3 of 51 · Rows per page 50

⚠ These 51 runs exist because a seed command wrote them through the product's own open_run. Nothing on any screen can start one.

The one place in the pack where runs are real. The seed script exists so a walkthrough catches a broken open_run — which is where DEF-RBS-4291 was hiding.

RBS-004 ▸ Activity

Runbook setup — Activity

`/:tenant/runbooks/:key#activity`

DRAWN TO THE RULES

SMARTBillHub

Runbooks

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

← Runbooks

Can we bill?

Check | Platform

Overview | Steps 10 | Scope | Guards | Limits | Runs 51 | **Activity**

Every edit, and what it changed

Each edit makes a version. A run records the version it used, so a later edit never rewrites history.

VERSION	WHAT CHANGED	WHO	WHEN
v3	Cap raised to 400	A. Reed	Today, 09:14
v2	Schedule set to Mondays	A. Reed	12 Sep, 16:40
v2	Paused	P. Menon	11 Sep, 08:02
v1	Added from templates	A. Reed	10 Sep, 11:25

The sentence is composed here from the facts the server serves — a field, a value before and a value after.

⚠ v3 is a version number, not a database id — which is why it may be shown (B14 rule 31).

An unrecognised field falls back to Settings changed rather than printing its column name at a clerk.

RBS-005.1

Run wizard — 1 Scope

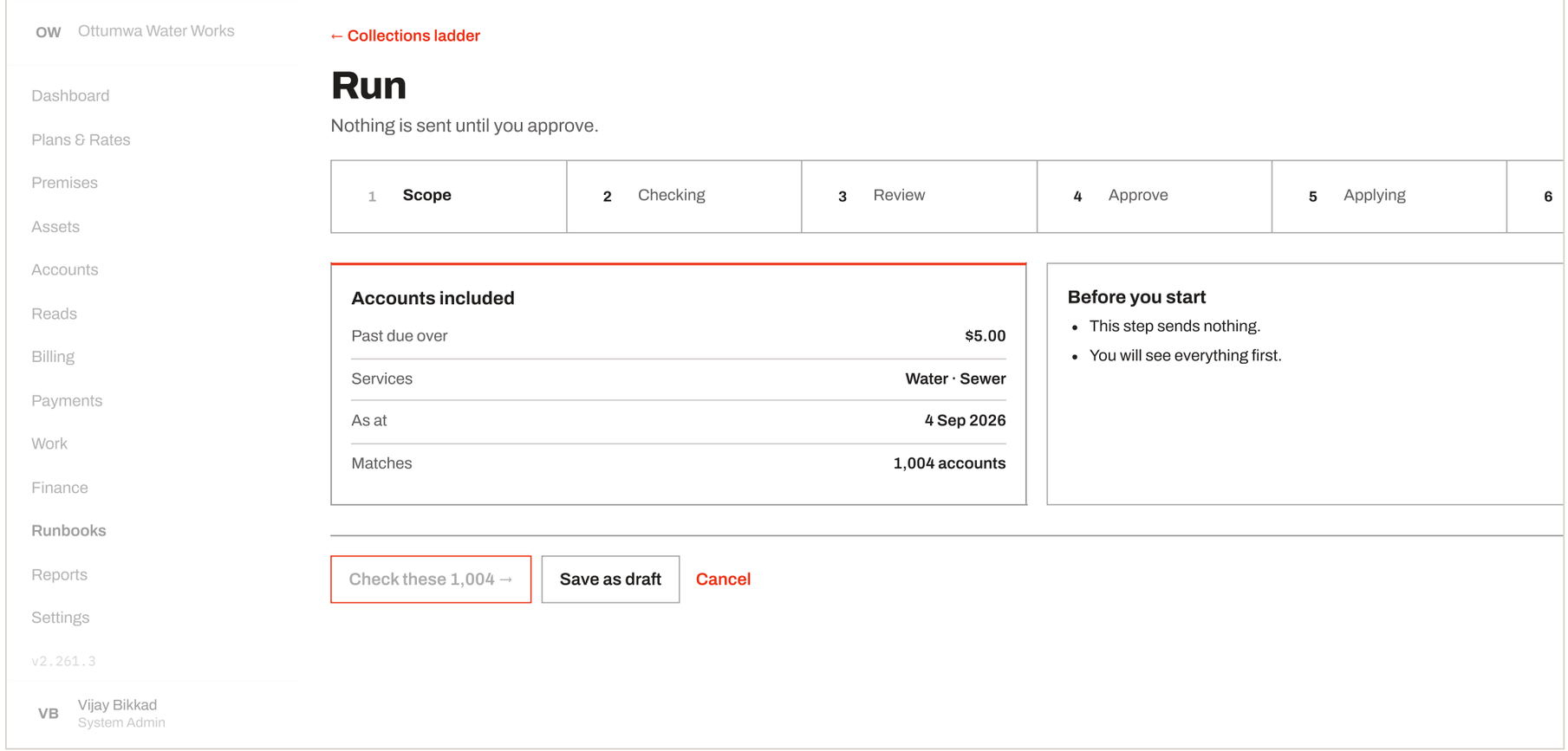
`/:tenant/runbooks/:key/run`

DRAWN TO THE RULES

SMARTBillHub

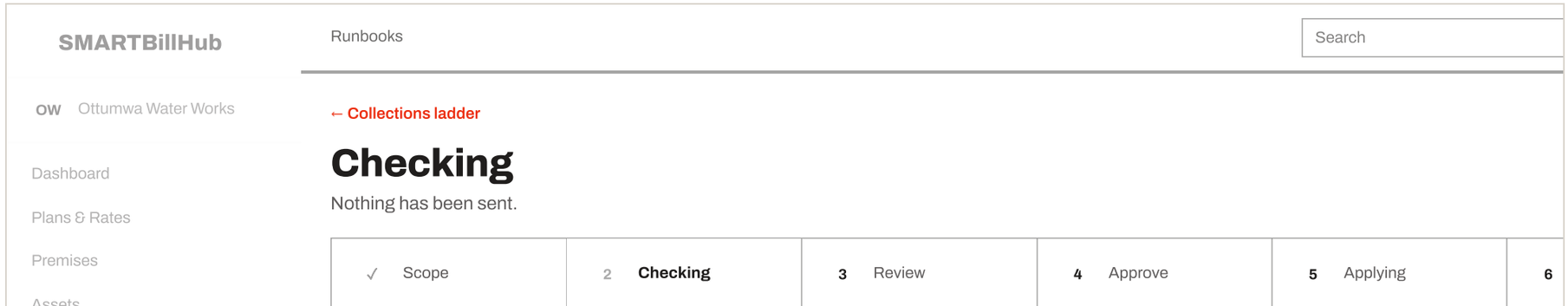
Runbooks

Search



Five regions, in this order. The consequence lines sit above the commit button, never after it — B14 rule 11.

RBS-005.2 **Run wizard — 2 Checking** `/:tenant/runbooks/:key/run` DRAWN TO THE RULES



Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings
v2.261.3
VB Vijay Bikkad
System Admin

582 of 1,004 accounts

About 25 seconds left

Find accounts past due
Remove protected accounts
Work out the tier due
Check the address works
Queue the notice
Runs only after you approve
Record what was sent
Runs only after you approve

> The two steps that write are listed and marked Runs only after you approve — visible, and visibly not running.

RBS-005.3 Run wizard — 3 Review

`/:tenant/runbooks/:key/run`

DRAWN TO THE RULES

SMARTBillHub

Runbooks

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance

← Collections ladder

Review

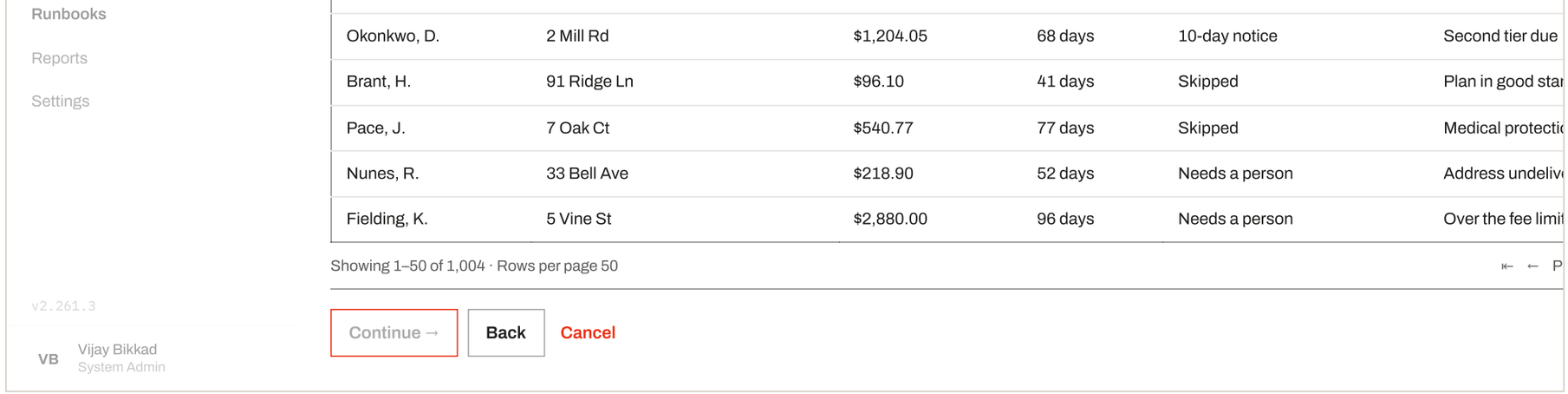
Nothing is sent until you approve.

✓ Scope	✓ Checking	3 **Review**	4 Approve	5 Applying	6

1,004 Accounts looked at
812 Notices to send
97 Skipped
95 Needs a person

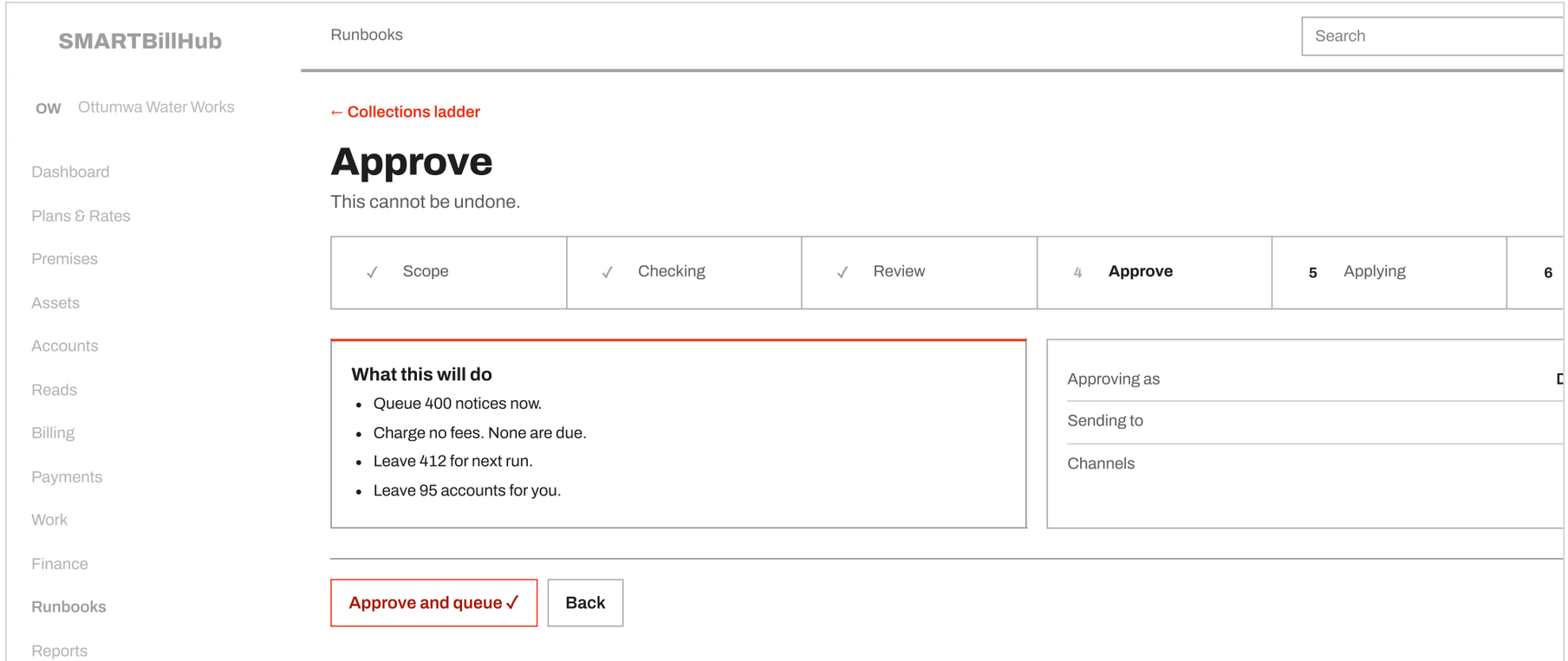
Search accounts… | All | To send 812 | Skipped 97 | Needs a person 95 | + Filter ▾ | ▦ Columns

ACCOUNT	SERVICE ADDRESS	PAST DUE	AGE	WHAT HAPPENS	WHY
Alvarez, M.	14 Cedar St	$182.40	34 days	Reminder	First tier due



Seven regions. Every row says what happens and why — a skipped account names the guard that skipped it.

RBS-005.4 **Run wizard — 4 Approve** `/:tenant/runbooks/:key/run` DRAWN TO THE RULES



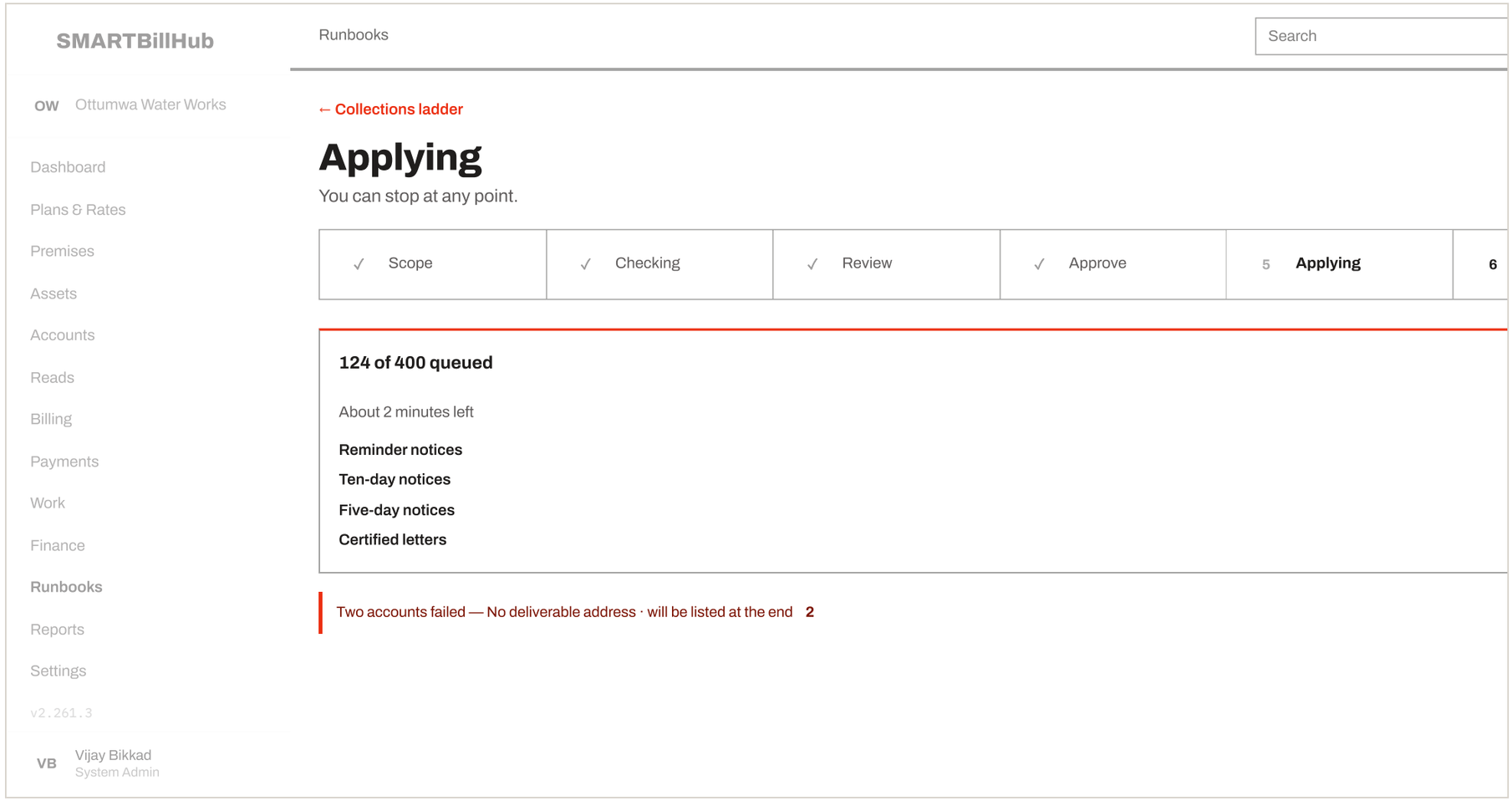
Settings

v2.261.3

VB Vijay Bikkad
System Admin

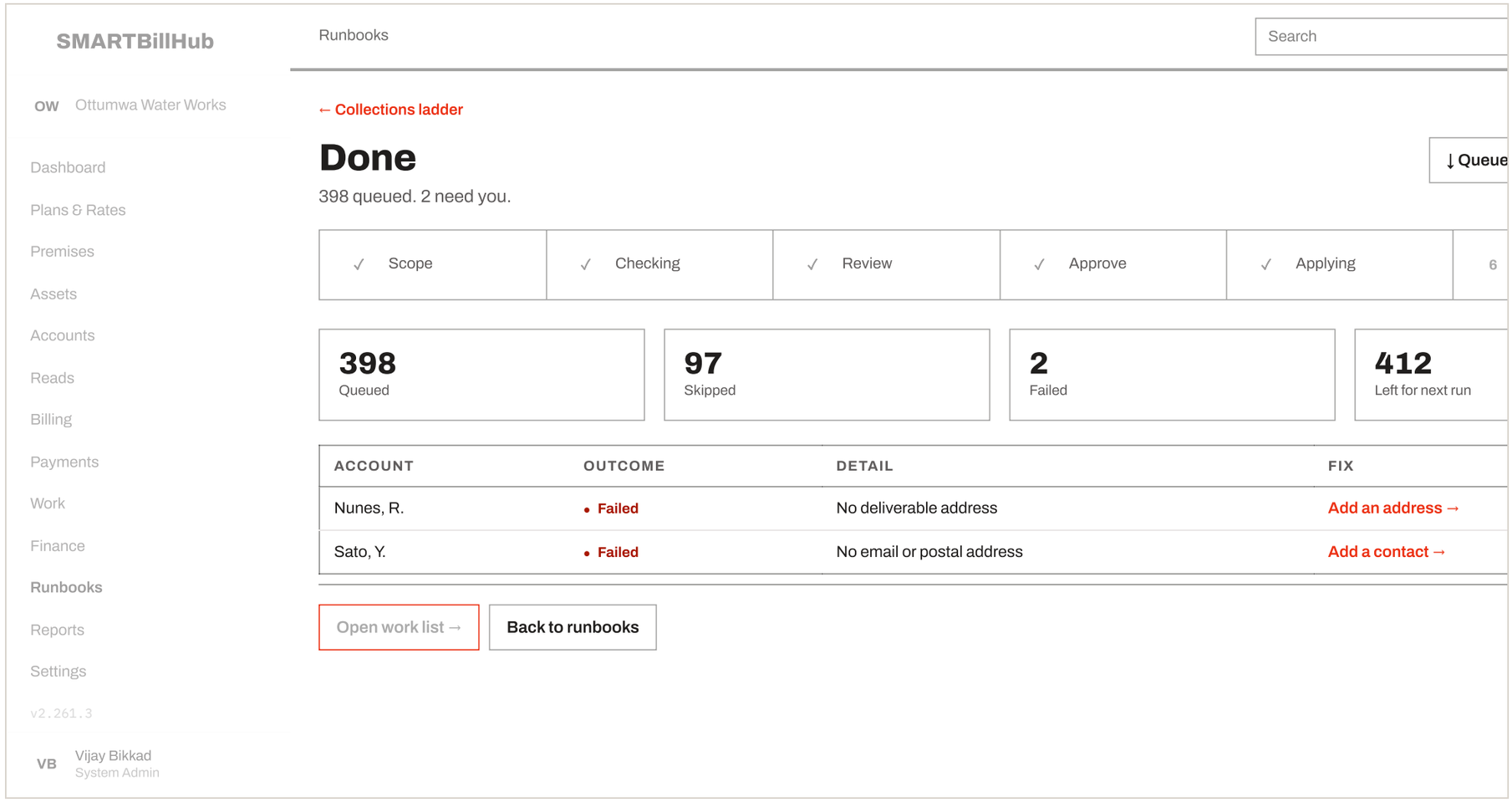
The commit is the danger tier, not the primary — B14 rule 6, because it sends. The note beside it is a fact, not a control.

RBS-005.5 **Run wizard — 5 Applying** `/:tenant/runbooks/:key/run` **DRAWN TO THE RULES**



Stop stays available the whole way through. A failure does not halt the run — it is collected and listed at the end.

RBS-005.6 **Run wizard — 6 Done** `/:tenant/runbooks/:key/run` DRAWN TO THE RULES



Every failure carries its fix as a link. A run that ends with two problems and no route to either is what BR-RBS-011 calls a defect in the runbook.

RBS-005.7 **Run wizard — the cap refuses it** `a state of step 3, Review` A STATE, NOT A PAGE

✓ Checking	3 **Review**	4 Approve

Too many for one run

812 planned. Your limit is 400.

Narrow the scope, or raise the limit.

Narrow scope | Raise limit | Send first 400 →

The refusal names the number, the limit and two ways out — and a third that is neither: send the first 400. One sentence of what happened, one of what to do.

RBS-005.8 **Run wizard — Checking failed** a state of step 2, Checking A STATE, NOT A PAGE

✓ Scope	2 **Checking**	3 Review

Find accounts past due 1,004

Remove protected accounts stopped

Could not read protections. Nothing was changed.

Work out the tier due –

Never reached

Check the address works –

Never reached

Try again ▶ | Tell support

Checking stays current rather than being ticked. A tick on a step that broke would be a lie, so the rail keeps the accent on it and the steps below say Never reached.

Checking stays current rather than being ticked. A tick on a step that broke would be a lie, so the rail keeps the accent on it and the steps below say Never reached.

RBS-8 **A four-step wizard, fully built, with no address at all** DEAD UI

RunWizard.tsx draws the check path — Scope · Checking · Review · Done, plus the failure state of step 2. It has a step rail, a pinned scope, a review body handed in from the result renderer, and its own walkthrough script. Nothing imports it except its own test, and it has no route.

Its own test is pointing at an address that does not exist. The fixture sets workListTo: /{tenant}/runbooks/work-list and asserts an anchor to it — while the product mounts the work list at /runbooks/work. A fixture that invents its own address cannot tell a working link from a broken one, which is exactly how the wizard's unreachability stayed invisible.

Four screens, by R6's count — Scope, Checking, Review and Done are each a step and each a drawing. All four are unreachable, so I have drawn step 1 and enumerated the rest rather than drawing four views of a screen nobody can open.

> RunWizard.test.tsx:83, :408 — runbooks/work-list

> App.tsx:452 — the product mounts runbooks/work

> Its own header names DEF-RBS-4289 (Save as draft has no draft to save) and DEF-CWP-3925 (a live screen behind a dead control)

The run wizard is eight frames, and two of them are refusals

Six steps and two failure states, drawn from the pack's own frames — every word of copy above is verbatim from the build sheets, which are themselves derived from the drawings. Scope · Checking · Review · Approve · Applying · Done is the Action rail; a Check runs the same wizard without Approve and Applying.

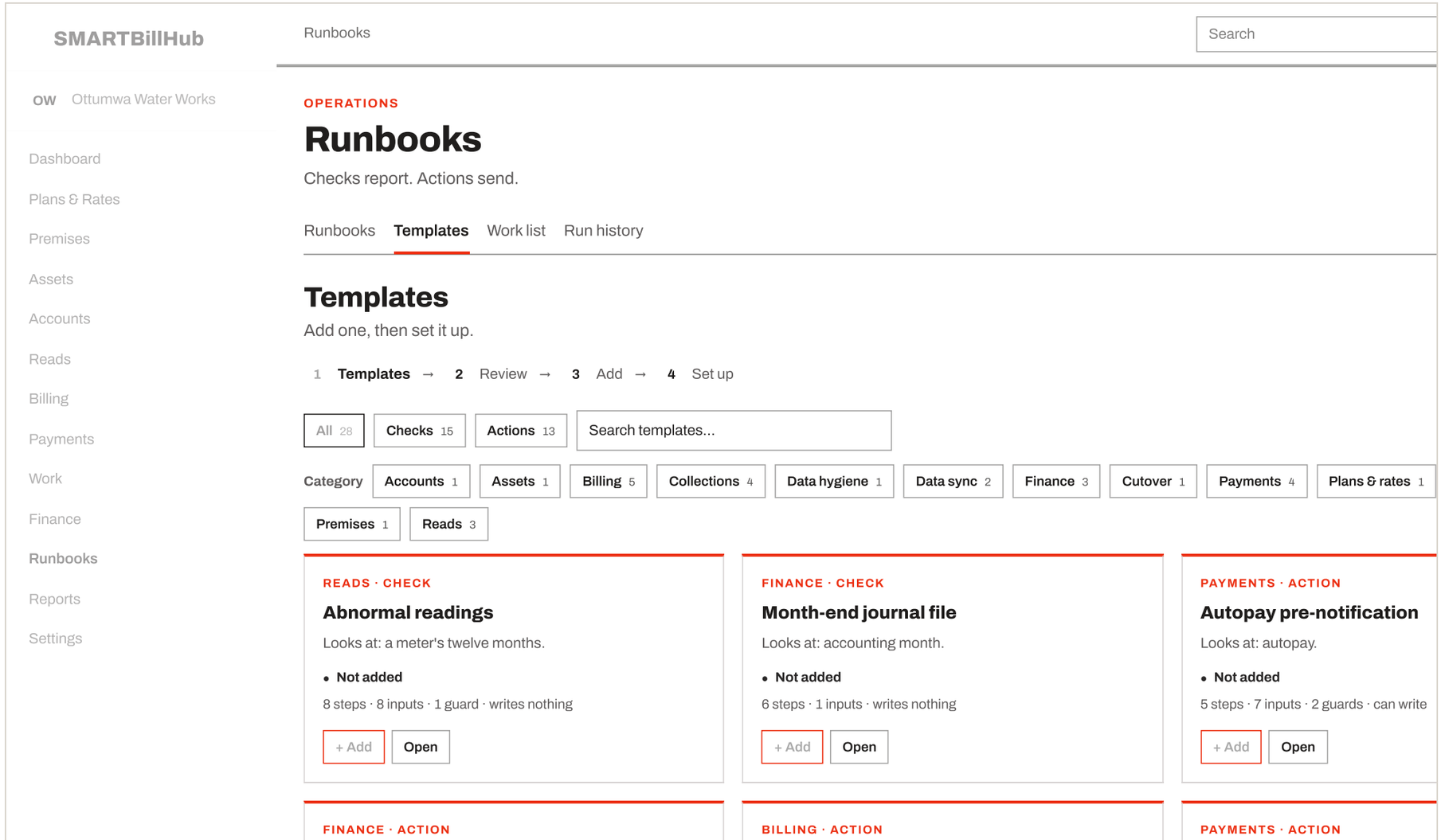
The two refusals are where the design earns its keep. RBS-005.7 is the cap being hit — it names the number, the limit, and three ways out rather than two. RBS-005.8 is a step failing: Checking stays current rather than being ticked, because a tick on a step that broke would be a lie, and everything below it reads Never reached.

All eight are drawn on Collections ladder, which is what the pack draws them on — the route in every frame is …/collections-ladder/run. Nothing in this wizard is reachable in the product: it is mounted nowhere.

Eight frames, one route.

The wizard is a single routed page holding six step components and two refusal states — and the seven setup sub-tabs are one page holding seven panels, because a sub-tab has no address. Section B counts every drawing in this pack against the file it lands in.

RBS-002.1 **Template library** `/:tenant/runbooks/templates` DRAWN TO THE RULES



Bank reconciliation

Looks at: a bank statement line.

- **Not added**

6 steps · 6 inputs · can write

+ Add | Open

Budget billing re-level

Looks at: a budget-billing account.

- **Not added**

4 steps · 6 inputs · 1 guard · can write

+ Add | Open

Cash application

Looks at: a landed payment.

- **Not added**

7 steps · 7 inputs · can write

+ Add | Open

COLLECTIONS · ACTION

Collections ladder

Looks at: past due.

- **Added**

4 steps · 8 inputs · 3 guards · can write

Open | Added

PLATFORM · CHECK

Can we bill?

Looks at: config graph.

- **Added**

10 steps · 4 inputs · writes nothing

Open | Added

BILLING · CHECK

Coverage grid

Looks at: account-period.

- **Not added**

3 steps · 6 inputs · writes nothing

+ Add | Open

1–9 of 28 · Rows per page 24

v2.261.3

28 templates, counted from the registry — 15 Checks and 13 Actions across 13 categories.

VB Vijay Bikkad
System Admin

RBS-4 Every template card has a description slot and none of the 35 has a description

ROUTE: DEFECT

The card renders a description paragraph only if the wire carries one — and it never does. So a clerk choosing between Books tie-out and Month-end completion gets two names, a category word and nothing else to choose on.

> `TemplateLibrary.tsx:371-373 — the paragraph is conditional and the condition is never met`

RBS-002.2 Template detail — read it, then add it `/:tenant/runbooks/templates/:key` DRAWN TO THE RULES

SMARTBillHub

Runbooks

Search

OW Ottumwa Water Works

← Templates

PLATFORM

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

Can we bill?

Check | Not added

✓ Templates → 2 **Review** → 3 Add → 4 Set up

What it does, in order

Ten steps. A step either looks at the whole set or one record at a time, and only a step that writes can change anything.

#	STEP	LOOKS AT	WRITES
1	Services and units	Every account	No
2	Reference data	Every account	No
3	Tax	Every account	No
4	Plans in force	Every account	No
5	Rate coverage	Every account	No
6	Account chain	Every account	No
7	Meters	Every account	No
8	Reads and windows	Every account	No
9	Documents	Every account	No
10	Books	Every account	No

What you can set

You choose these after you add it.

INPUT	NEEDED	STARTS AT
Report	Required	Blockers and warnings
Usage to cover	Required	Ladder is unbounded
Include held accounts	Optional	False
Raise work items	Optional	True

Who it never touches

Guards remove a subject before any step runs.

Nobody. This one excludes no accounts.

What it needs

A connector

Looks at

Raises work items

Add this to my runbooks

It is live at once. You set its schedule next.

⑤ + Add to my runbooks

The add is live today — one call, and the runbook exists. The step-by-step review is not: the shipped detail page lists step names only, with no Looks at, no Writes and no guards. Every
served by the endpoint already.

v2.261.3

VB Vijay Bikkad
System Admin

RBS-002.3 **Add — the one call, and what it makes** `/:tenant/runbooks/templates/:key → POST` **DRAWN TO THE RULES**

SMARTBillHub

Runbooks

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

← Templates

PLATFORM

Can we bill?

Check | Not added

✓ Templates → ✓ Review → 3 **Add** → 4 Set up

What happens when you press Add

One call. The runbook exists, switched on, with nothing scheduled.

It is created as	Your ow
Switched on	
Scheduled	
Version	
Can it run yet	Onl

Adding “Can we bill?”…
This takes you to its set-up screen.

⑥ Adding…

⚠ Nothing confirms it worked. Today the page simply changes to another screen — no banner, no toast, no sentence. A refusal gets a toast; a success gets silence.

Adding is reversible — pause it or remove it — so it asks nothing before it acts. That part is right.

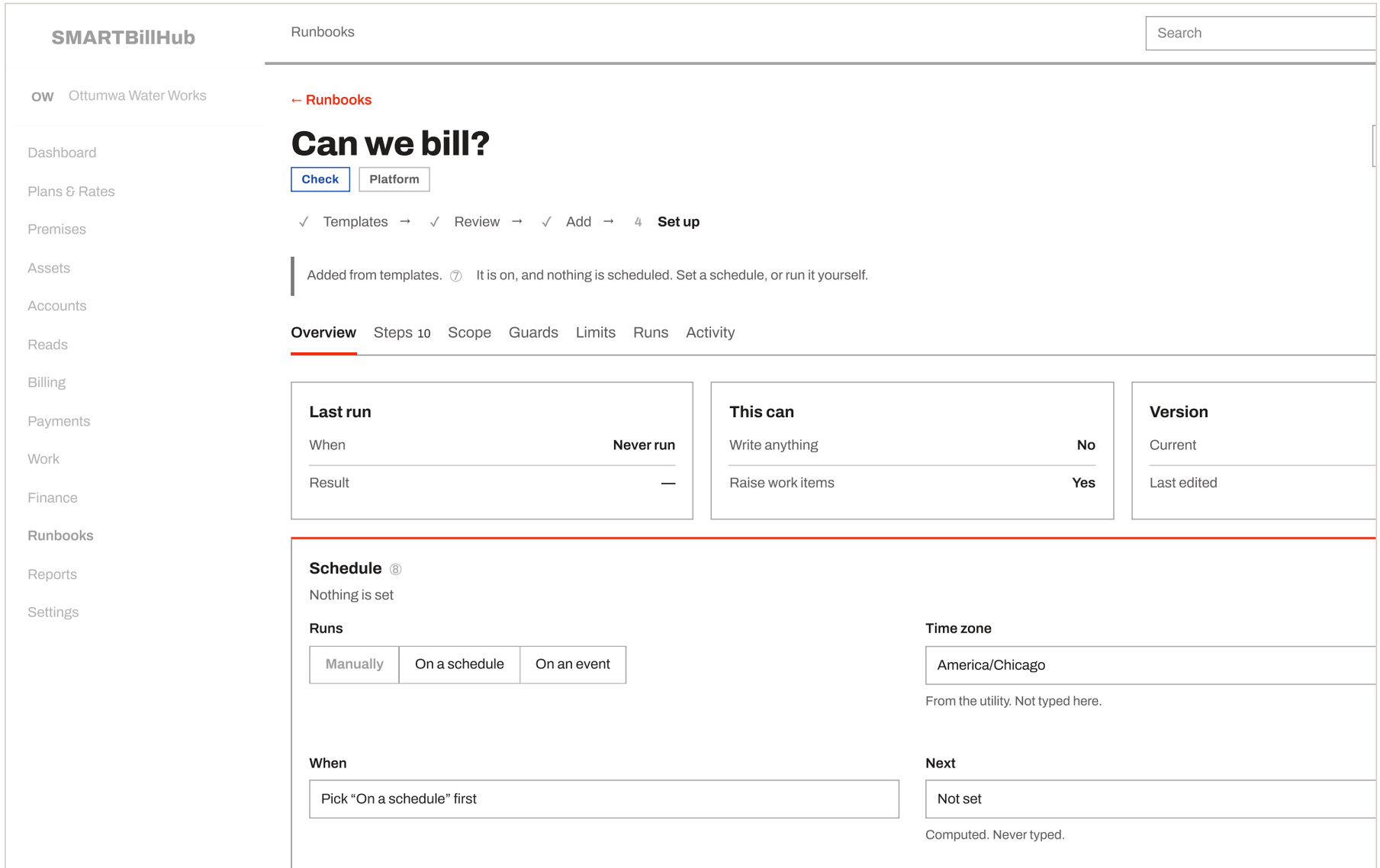
v2.261.3

Vijay Bikkad

VB
System Admin

This is a moment, not a route. The product posts and redirects in the same breath, so the only thing a builder can draw here is what the call creates — and the fact that nobody is told.

RBS-004 ▸ Just added **Set up — the screen you land on** `/:tenant/runbooks/:key (first open)` DRAWN TO THE RULES



v2.261.3

VB Vijay Bikkad
System Admin

> The same screen as RBS-004, in the only state it has on day one: v1, never run, nothing scheduled, trigger Manually. Drawn because a builder given only the settled state builds the w

RBS-9 Adding a runbook succeeds in silence, and the screen changes under you

REAL GAP · FOUND BY WALKING YOUR FLOW

Press + Add and the product posts, throws away the page you were on, and drops you on a different screen with no sentence anywhere saying what just happened. A refusal gets a toast. A success gets nothing at all.

It is the asymmetry that makes it a defect rather than a preference. The same handler already knows how to speak — it calls showToast on error, two lines below the redirect. Somebody decided the failure was worth a sentence and the success was not, and the clerk who pressed the button is left inferring from the address bar.

Drawn as an arrival banner rather than a toast, on RBS-004 ▸ Just added: "Added from templates. It is on, and nothing is scheduled." A toast disappears while you are still reading the screen it arrived on; this one is the first thing on the page you landed on, and it names the one thing left to do.

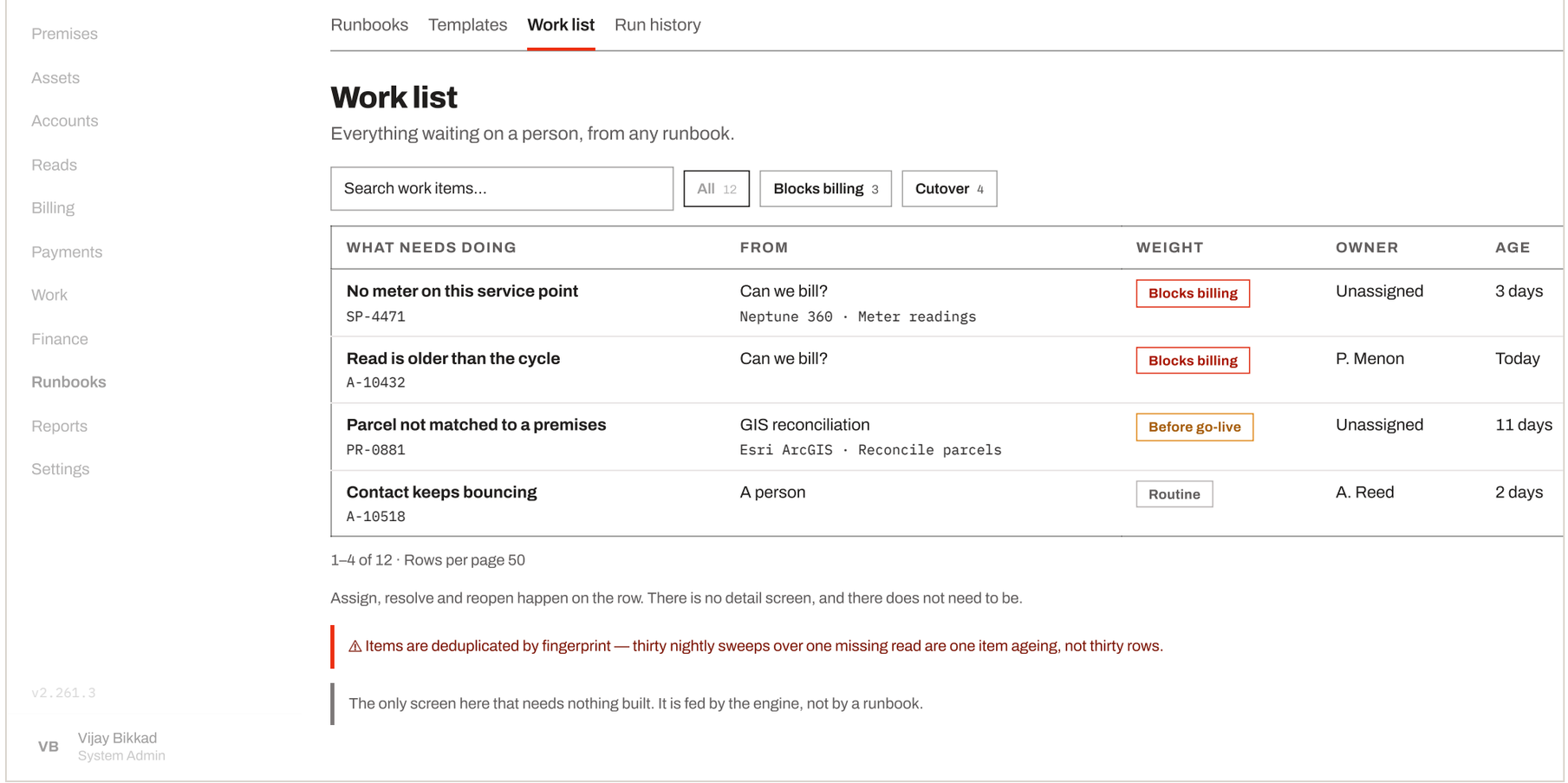
> `TemplateLibraryPage.tsx — onSuccess invalidates the cache and calls navigate; there is no success message`

> `Same handler — onError: showToast(refusalText(...))`

> `Both doors — the library card and the template detail share this one mutation`

RBS-007 **Work list** `/:tenant/runbooks/work` **DRAWN TO THE RULES**





RBS-3 This screen works, and nothing in the product links to it

LIVE SCREEN, DEAD CONTROL

Mounted, rendered, searchable, filtered, paginated, tested — and its only pointer is the disabled button on the Runbooks home reading "Not built yet."

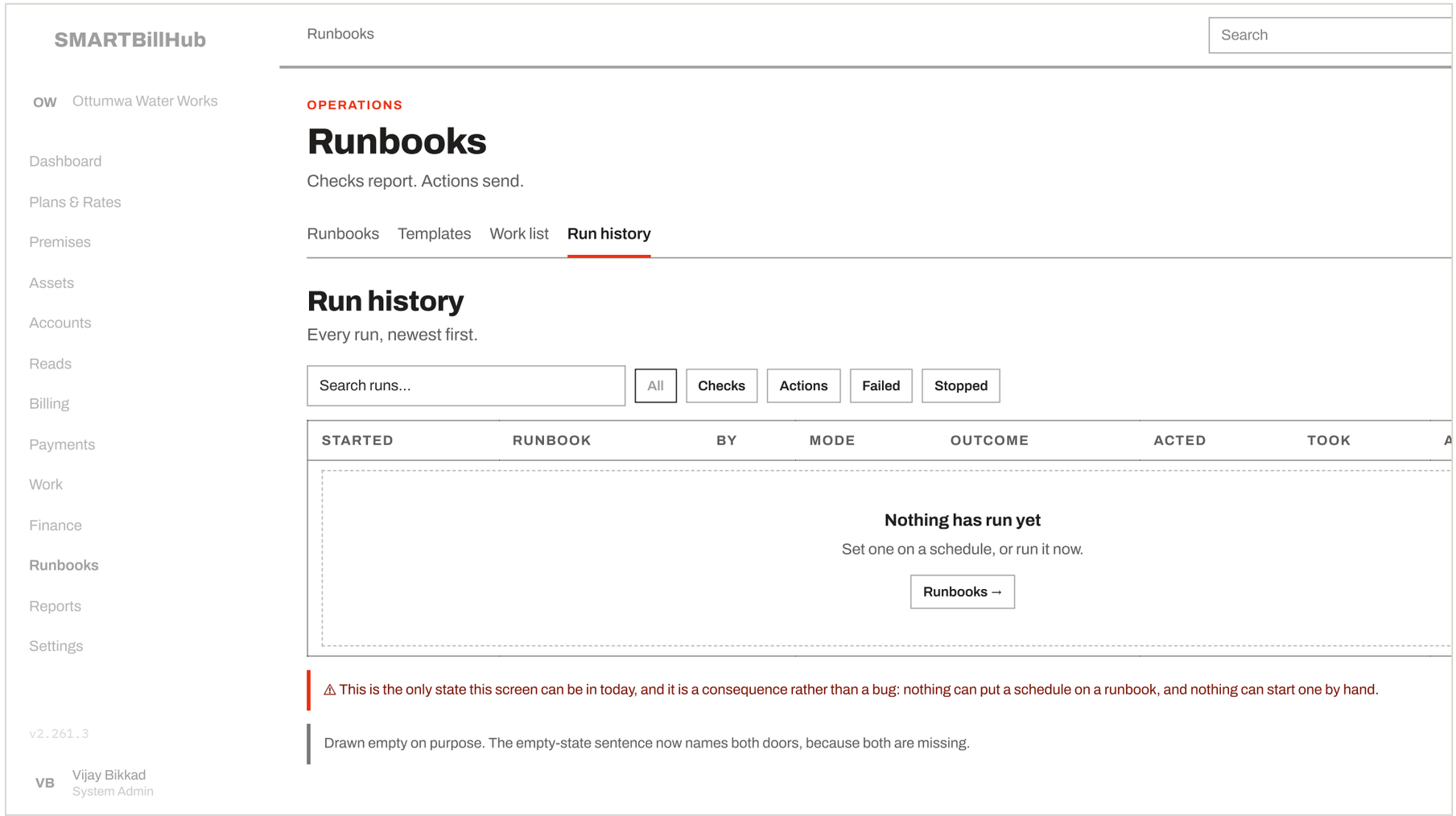
This is the failure that cost the customer portal six of its eight front-page doors on 5 September. Every check passes over it, because each wave tests its own screen at its own address and "has the door started pointing at it yet" belongs to nobody.

> `App.tsx:452 — { path: "runbooks/work", element: <WorkListPage /> }`

> `runbooksHomeModel.ts:247-253 — still in HELD_DESTINATIONS`

> `Every mention of runbooks/work in src/: the route, the register, the held entry, its tests. No link.`

RBS-008 **Run history** `/:tenant/runbooks/runs` DRAWN TO THE RULES



This is the state every tenant sees today

Run history is drawn empty on purpose here, because that is the only state it can be in: nothing has run, because nothing fires. Its empty-state helper says "Set one on a schedule." — advice that cannot be followed, since a runbook's Next is always "Not set". The empty state is pointing at the gap.

RBS-006 **Run result** `/:tenant/runbooks/:key/runs/:runId` **DRAWN TO THE RULES**

SMARTBillHub

Runbooks

Search

OW Ottumwa Water Works

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.261.3

VB Vijay Bikkad
System Admin

← Run history

Can we bill?

Run · 14 Sep 2026, 08:12

3 blockers | Version 3 | Dry

1,204	3	6	12s
Accounts checked	Blockers	Warnings	Took

Which APIs it called

Neptune 360 · Meter readings

Esri ArcGIS · Address search

What stops billing

All 9 | Blockers 3 | Warnings 6

ACCOUNT	WHAT	WHY	ACT
A-10432	No meter on this service point	SP-4471 has no active meter	
A-10518	Read is older than the cycle	Last read 47 days ago	
A-10604	No plan on the unit	Unit 2B carries no plan	

1–3 of 9 · Rows per page 50

The report is frozen as it closed. Reopening it a year from now draws exactly this.

Version 3 and Dry are on the header deliberately — a run pins the version it used, and dry is not live.

RBS-7 **All four actions on the result screen are enabled and all four refuse** ROUTE: DEFECT

Pressing any of them raises a toast and does nothing:

- Run again → "Starting a run arrives with the acting half."
- ↓ download → "Result files arrive with the run surfaces."
- a row decision → "Deciding a row arrives with the acting half."
- an action block's button → "This action arrives with the acting half."

Four live-looking controls, four refusals, on the screen a clerk lands on after every run. Each sentence is honest and well-written; the problem is that nothing on the screen says so until the click. This is the Run ▶ pattern (RBS-2) repeated four times.

Two more screens, drawn and not built — and I am not filing them

S145 (consent, staff) and S146 (consent, customer) exist as frames 57.1 and 56.1. Neither has a component, a route or a row in the screen register. Not a defect: the layout document marks both "shape only · wording not settled". Recorded so their absence is not read as an oversight. The same for S141, the connect modal — it is built, as a modal over the grid, so it correctly has no address of its own.

The five callouts, across the four tabs — what each one is, and whether anything answers it today.

#	SCREEN	WHAT IT IS	ANSWERED TODAY?
①	RBS-003	+ Add from templates — the one door into the Templates tab from the list, and the empty state's only instruction	**Yes** — POST /runbooks is live and the button is wired
②	RBS-004	The schedule form — Runs · When · Time zone, with Next computed and never typed	**No** — the panel renders three facts and has no form, so Next can only read Not set
③	RBS-004	Run ▶	**No** — it shows a toast. Of the twelve runbook routes the app serves, none starts a run
④	RBS-005	Start the check →, at an address the wizard does not have today	**No** — same missing route as ③, plus the wizard is mounted nowhere

⑤	RBS-002.2	+ Add to my runbooks from the template detail	**Yes** — wired on both the card and the detail page
⑥	RBS-002.3	The add itself — one call that creates the runbook switched on, at v1, with nothing scheduled	**Half** — the call is live; nothing tells the clerk it worked. A refusal gets a toast, a success gets silence and a different screen
⑦	RBS-004 ▸ Just added	Added from templates — the arrival banner that says it is on and unscheduled	**No** — there is no banner. This is ⑥'s missing half, drawn where it belongs
⑧	RBS-004 ▸ Just added	The schedule form, empty — Manually, and Next reading Not set truthfully for once	**No** — same missing form as ②

Two controls on one screen — that is the whole build

Templates, Work list and Run history need nothing. Both missing doors are on the same screen: the schedule form and the Run button on RBS-004.

And the engine behind them is finished. I probed it at runtime — both scheduler job handlers are bound and the sweep task is registered. What is missing is not the machinery, it is the two ways in.

B What a builder actually builds

Eighty-four drawings are not eighty-four files. A drawing is a state; a file is a route. Read this before opening an editor, because the most expensive mistake available here is to read CFG-01…28 as twenty-eight screens to write.

Twenty-eight connector files would be a rule breach, not just waste

BR-INT-037, in the PO's own words: "every time such integration comes we dont want to change our integration framework." A new vendor is a manifest and nothing else — no new screen, no new route, no new file. If adding Badger means writing BadgerSetupPage.tsx, the framework has already failed.

The 28 set-up forms are one page rendering a field list from data, and the 28 connection views are one page rendering an operation list from data. They are drawn 28 times to prove

that one page renders all 28 correctly — which is what a drawing is for. That is the whole reason Manifest carries the fields and Operation carries 23 attributes.

Every drawing in this pack, and the file it lands in. Counted against what is on disk today.

DRAWINGS	#	WHAT GETS BUILT	FILES
CFG-01…28 — the set-up forms	28	**ONE** page, fields from the manifest	1
VIEW-01…28 — the connection views	28	**ONE** page, APIs from the manifest	1
CON-01 · MAP-01 · KEY-01 · ACC-01	4	A settings tab, two pages and a panel	4
Section 03 — connectors		`IntegrationsTab · ConnectModal · ConnectionDetailPage · ValueMappingPage · MachineKeysPage · AccessLogPanel`	≈6
RBS-004 + its 7 sub-tabs + Just added	9	**ONE** page and seven panels. A sub-tab has no address — tabs are state, not routes	1 + 7 panels
RBS-005.1…8 — the wizard	8	**ONE** routed wizard: six step components and two refusal states	1 + 8 parts
RBS-002.1 · .2 · .3	3	Two pages — .3 is a mutation, not a page	2
RBS-003 · 006 · 007 · 008	4	Four list-and-detail pages	4
Section RB — runbooks		Eight page files, one per route	≈8
The whole pack — 84 drawings		≈14 routed pages, plus the panels and steps inside them	**≈14**

The product already proves this, and the number is on disk

The eighteen connectors that ship today are served by six files and 3,785 lines. Not eighteen files — six. ConnectModal.tsx already renders its inputs from field.name and field.required, so a nineteenth vendor costs it nothing.

So the arithmetic that matters is not 28 → 28. It is 18 → 28 with the file count unchanged.

What IS one per thing, and it is not a screen

One manifest per connector

— data, in apps/integrations/manifests/, which is why 28 connectors are 28 files there and one file in the front end. **One definition per runbook** — code, in apps/runbooks/definitions/, 28 of them today.

The pattern is the same on both sides: the thing that varies per vendor is a declaration, and the thing that renders it is a single screen. Every drawing in this pack is a picture of that one screen with different data in it.

F

Field tables — where every pixel's data comes from

A drawing with no field table is half a drawing: the picture says what is on the screen, the table says whether any of it is real. ⇄ marks a fact that crosses a module boundary, with both modules named — those rows are what the seam grading in section 07 is built from.

S8 · the provider card

One endpoint: `GET /integrations → api.py:181 → selectors.py:428 grid()`

FIELD ON SCREEN	PAYLOAD KEY	OWNER	HEALTH
Kicker — EMAIL · WE SEND	`category + kicker.{initiative,direction}`	integrations	**Healthy**
Provider name	`display_name`	integrations	**Untranslatable**
Summary sentence	`summary`	integrations	**Untranslatable**
Status word	`display_state`	integrations	**Healthy**
Per-operation status lines	`operations[].label / .state`	integrations	**Healthy**
Failure reason	`connection.last_test_text`	integrations	**Healthy**

Environment · address	`connection.environment / .describe`	integrations	**Healthy**
Button label and tier	none — a literal ACTION map in the component	browser	**Hardcoded**
"Not ready to connect yet."	none — providerIsConnectable() re-derives a server rule in the browser	browser	**Rule in two places**
Search and category filter	none — no ?q= or ?category= is ever sent	browser	**Client-side**
Footnote count	`counts.providers`	integrations	**Healthy**
⇄ (carried, drawn on S142) messages waiting	`connection.disconnect.messages_waiting`	integrations ← notifications	**Healthy**
⇄ Connected by	`connection.connected_by`	integrations ← tenancy	**Healthy**

S142 · connection detail

Five endpoints. The head, operations, credentials and runs all arrive in one: `GET /integrations/{key}/runs → runs.py:259 list_view()`

FIELD ON SCREEN	PAYLOAD KEY	OWNER	HEALTH
Describe line	`describe`	integrations	**Healthy**
Operations list	`operations[].label / .state / .executor`	integrations	**Healthy**
Credential history	`credentials[].{version,hint,created_at,is_active}`	integrations	**Healthy**
The Result cell — 86 of 93 · 7 bad addresses	composed in the browser from six numeric keys; the server sends no rendered string	browser	**Composed**
…its "skipped" phrasing	`integrations.runs.skipped.<operation>` — an empty namespace, always falls back	browser	**Generic**
…its "failed" phrasing	exists for one operation only (send_message)	browser	**Generic**
Run detail counts, items, provider text	`items[], provider_error, asked_for[], content_hash`	integrations	**Healthy**
Budget line — 4,180 of 10,000	**NOT SERVED** — the slot is drawn in the pack and never rendered	—	**Absent**
Rate limit, per operation	**NOT SERVED** — ruled 16 Sep, display only. No table holds one; it needs a row per connection and operation	—	**Absent**

Access-log rows	`items[].{when,what,rows,hint,unusual}`	integrations	Healthy
..."and 42 more reads"	computed in the browser; SHOWN = 20 is a literal while the server caps at 100 / 25	browser	Three magic numbers
⇄ Disconnect consequence	`disconnect.{channel, messages_waiting}`	integrations ← notifications	Healthy
⇄ What a partner took	`items[].what / .rows`	integrations ← payments · billing	Healthy
⇄ Every date and time on the screen	`timeZone / dateFormat`	integrations → metering → tenancy	Three-module hop

S143 · mappings

One path, two verbs: `GET and PUT /integrations/{key}/mappings → mappings.py:225 view()`

FIELD ON SCREEN	PAYLOAD KEY	OWNER	HEALTH
Their word	`rows[].external_value`	integrations	Healthy
Our word	`rows[].local_label / .local_value`	integrations	Healthy
"2 of 14 not mapped"	`types[].counts.{unmapped,rows}`	integrations	Healthy
Kind pills	`types[].entity_type`	integrations	Healthy
Search · "Unmapped only" · kind filter	all browser-side — the whole row set arrives in one GET	browser	Client-side
⇄ How they pay choices	`PaymentMethod`	integrations ← tenancy	Healthy
⇄ Bounced payments choices	`REVERSAL_REASONS`	integrations ← payments	Second copy
⇄ Their account codes choices	`LedgerAccount`	integrations ← ledger	Healthy
⇄ Routes choices	`Route`	integrations ← metering	Healthy
⇄ Meter types choices	`MeterType`	integrations ← assets	Healthy
⇄ Services choices	`Service`	integrations ← tenancy	Healthy

This one screen reaches into five other modules

Its option lists come from tenancy, payments, ledger, metering and assets. That is the widest cross-module surface anywhere in either module — and it is on the screen your own ruling says is Runbooks' business logic. Whatever OQ-INT-1727 decides, these six reads move with it, and that is a materially bigger piece of work than "537 lines and a screen".

One of the six is already a second copy: the four bounced-payment reasons are restated as English literals inside the integrations module, duplicating payments' own vocabulary across a boundary.

S144 · keys we issue

⚠ There is no dedicated read endpoint — the table reads the whole S8 grid and picks one provider out of it.

COLUMN	PAYLOAD KEY	OWNER	HEALTH
Key	`issued_keys[].hint + .version`	integrations	**Healthy**
What it reaches — "This utility only"	**NOT SERVED** — a literal in the component. Nothing on the server says a key's scope.	browser	**Hardcoded**
Issued	`issued_keys[].issued_at`	integrations	**Healthy**
Last used	`issued_keys[].last_used_at`	integrations	**Healthy**
State	`issued_keys[].state`	integrations	**Healthy**
Pagination	client array length — there is no server pagination for keys at all	browser	**Client-side**
⇄ Last used, how it gets set	partner traffic recorded through the serve runner	integrations ← payments · billing	**Healthy**

FLD-1 **Three whole i18n namespaces are declared, referenced nine ways, and empty**

ROUTE: DEFECT

en.json holds 319 keys under integrations. and zero under integrations.provider. Every one of the nine call sites that asks for a provider's name, summary, field label or operation label falls back to the

server's manifest English, always.

Two more namespaces are empty the same way: integrations.option.*, so the connect modal renders raw enum values as segment labels — a clerk choosing a bank file format reads camt053 and bai2; and integrations.modal.segmentLabel.*, so three segmented controls carry aria-label="sign-in".

The consequence is concrete: translating a provider name is impossible today without first adding keys — in a product that ships eleven locale catalogues.

```
0 keys under integrations.provider; 9 call sites read it
```

```
0 keys under integrations.option; 6 manifests declare select fields
```

```
integrations.modal.fieldSlot exists and is rendered by nothing — dead copy
```

FLD-2 Every search and filter in this module is client-side

NFR-3

S8: no ?q=, no ?category= — the whole grid arrives and the browser filters it. S143: search, the unmapped checkbox and the kind tabs all filter an array already in memory. S144: the state filter and the pager both count a client-side array.

Only S142's run table sends anything to the server — and it is the one list here whose rows a tenant actually accumulates. So the pattern is right where it matters most and absent everywhere else, which is exactly the shape that looks fine on twenty rows and is silently wrong on twenty thousand.

S8 and S144 are defensible — 18 manifests and a handful of keys are bounded by the code, not by tenant data. S143 is not: the vendor decides how many rows there are, and the screen says so in its own footnote.

FLD-3 Eleven fields are served and read by nothing

ROUTE: DEFECT

Each is computed on the server, sent over the wire, and dropped:

- S8 — counts.connected
- access log — unusual_reads, total_records (neither is even in the TypeScript interface)
- mappings — types[].summary, types[].open, rows[].shipped, root counts
- keys — issued_keys[].revoked_at
- subprocessor register — six of seven fields; only sentence is drawn

Two of them are the interesting ones. unusual_reads is the server's own count of suspicious access — computed, sent, and never shown on the screen whose entire purpose is to surface suspicious access. And types[].summary would have given the mappings tabs the explanatory sentence they currently lack.

04 Every table, and its four controls

A toolbar is four independent mechanisms that fail independently. Auditing "the table" as one thing is how three of the four go unchecked. Twelve lists across the two modules, each control read from its config rather than its markup.

Read from each screen's TableConfig. "—" means the control is not rendered at all, not that it is empty.

LIST	SEARCH	FILTERS	COLUMNS ▦	SORT	PAGES
S8 card grid	name + summary	registry categories	—	—	—
S142 recent activity	trace id only	3 dimensions + Filter ▾	yes	When — arrow hidden	50
S143 mappings	their word + ours	a checkbox	—	—	none
S144 keys	none	state, client-side	yes	—	client-side
S142 access log	—	—	—	—	"and 42 more reads"
RBS-001 needs you	—	—	yes	none	yes
RBS-001 finished	—	—	yes	none	yes
RBS-002 templates	yes	kind + category	—	none	conditional
RBS-003 my runbooks	name + key	kind	yes	none	yes
RBS-004 runs tab	—	—	yes	none	yes

RBS-006 records	—	server chips	yes	none	yes
RBS-007 work list	title + ref	3 chips with counts	yes	none	yes
RBS-008 run history	runbook name	5 chips	yes	none	yes

TBL-1 Not one column in the whole runbooks module is sortable ROUTE: DEFECT

Eight lists, and no column anywhere in features/runbooks/ declares sortable. So no header is a button and no ▲/▼ ever appears — on run history, on the work list, on my runbooks. A clerk cannot put the oldest work item at the top.

Each list has a sensible default order — newest first, or weight first — so nothing looks broken. That is why it has gone unnoticed: the absence of sorting is invisible until somebody wants a different order, and then there is no affordance at all.

> `grep sortable in features/runbooks/ — zero hits`

> `r10 UI-RULES rule 13 — a list table is sortable by its meaningful columns`

TBL-2 The mappings list has no pagination, and the vendor controls its length NFR-3

S143 renders every value as a bordered row, with no pager and no cap. That is fine at fourteen. The screen's own closing sentence is the problem: "Their new words appear here unmapped." — the list grows when the vendor decides, not when we do.

NFR-3 is universal server pagination and NFR-5 is no hard caps. This is the same shape as the four breaches CLAUDE.md already names, on a list nobody has measured because today it is short.

05 Behind every row: detail, add, edit, update

Four surfaces hang off each list, and each is its own screen with its own door. "Add" is where this most often stops.

LIST	DETAIL	ADD	EDIT	UPDATE GOES
S8 cards	Recent activity → S142	n/a — manifests are code	Connect modal (S141)	`POST connect / test`
S142 runs	RunCard, inline below the table	n/a — runs are made by running	none — append-only	—
S143 mappings	none — the row is the detail	none — rows arrive from the vendor	inline select, in the row	`PUT mapping`
S144 keys	none	Issue key (header)	n/a — replace, never edit	`POST issue / revoke`
RBS-003 mine	row → RBS-004	from Templates	RBS-004's tabs	`PUT enabled / scope / limits`
RBS-007 work	**Open the fix — often absent**	n/a — raised by runs	Resolve / Reopen	`PUT status`

The one that will bite a clerk

On the work list,
Open the fix
is the whole point of the row — and it is disabled with "No screen fixes this" whenever the raising runbook did not name a destination. A work list whose items cannot be opened is a report, not a queue. I could not measure how often that happens without running the product against real data; it is the first thing to check in a browser pass.

And financial immutability holds

Nothing in either module offers an edit on an issued invoice, a payment or a journal line. The runs table is explicitly append-only — "Runs are kept exactly as they happened and are never edited." No finding here, and that is worth saying, because an edit control on one of those surfaces would be a rule breach rather than a feature.

06 Against the UI rules

One row per violation, never per screen. Most of B14 is ratified and unenforced — no

gate in this repo reads rendered output — so expect real violations with a green suite over every one.

SCREEN	RULE	WHAT IT SAYS	WHAT THE SCREEN DOES
S143 mappings	tokens	--radius-sm/md/lg: 0px — every corner square	Two rounded-full pills, hand-rolled, where every other screen uses the kit's ChipToggle
All 8 runbooks lists	UI-RULES 13	A list table sorts by its meaningful columns	No column declares sortable; no header is a button
S143 mappings	UI-RULES 16	Pagination on every list table	No pager; the list length is the vendor's to decide
S143 mappings	UI-RULES 17	An empty state on every list table	One string — "Nothing recorded yet." — for both "nothing exists" and "your search matched nothing"
S142 runs	UI-RULES 13	The sorted column shows which way it is sorted	defaultSort names started, the column is when — no arrow until clicked
S142 access log	B14 rule 32	A stated absence is closed by default	A failed fetch renders as "Nobody has collected anything yet." — an absence claimed, not stated

What passes, and is worth naming

Rule 21

(Tailwind ledger): both features/integrations (8 entries) and features/runbooks (4) are on route-b-migrated.txt — so utilities are checked, not silently unchecked. **Rule 31** (no database id on screen): keys show a four-character hint, never a row id; support references are trace ids and are meant to be read aloud to support. **Rule 33** (copy budget): every refusal I read is at or under six words — "Not built yet.", "Runs cannot start yet.", "No screen fixes this".

07 Every crossing, graded

Cross-module writes are forbidden and cross-module FKs are the reviewed interface

set. A seam that hands back a raw queryset is leaky even though it only reads — grade on what it returns, not what its docstring says.

CROSSING	THROUGH	WHAT IT RETURNS	GRADE
integrations → runbooks	`seams/runbooks.project_readings`	an empty tuple, by ruling	**Healthy**
runbooks → integrations	`seams/integrations_seam.live_connections`	dicts of key, name, status — no credential, no connection id	**Healthy**
notifications → integrations	`provider_seam → services.dispatch_outbound_batch`	one entry per message id; resolved by name at call time	**Healthy**
integrations → notifications	`seams/notifications.deferred_backlog`	a count, never a list	**Healthy**
integrations → metering	`seams/metering`	the premises write on the GIS path	**Healthy**
metering → the US Census	nothing	a live urllib call to geocoding.geo.census.gov	**Absent**
runbooks → vendor vocabulary	string literals in definitions	METER_PROVIDER_KEYS, DELIVERY_PROVIDER_KEYS typed by hand	**Duplicated**

SEAM-1 A third party is dialled from the metering module, with no connector and no seam

ALREADY FILED

apps/metering/geocode.py:74 holds a hard-coded census URL and calls it directly. Meanwhile a geocoder_census manifest exists, is offered to utilities as live, and has no connector behind it. So the product both advertises the connection it does not have and makes the call it says it does not make.

It is DEF-INT-4251, already on the register. Not re-filed — but it is the clearest single example of connector work outside the connector module.

Vendor names in two vocabularies

Two runbook definitions type "meters_mdm" and "meters_neptune360" as string literals rather than importing them — deliberately, to avoid a cross-module import that a gate would refuse. The instinct is right and the result is a second copy of the vendor list that nothing keeps in step. A published constant on the integrations seam would settle it.

08 Your seven, one at a time

Two of the seven names do not mean what they look like, and one needs a decision from you.

Connector = can we reach the vendor. Logic = is anything done with what comes back.

YOU NAMED	WHAT IT IS HERE	CONNECTOR	WHERE IT STANDS
Email	Four cards — SMTP, Microsoft 365, Gmail, bulk sender	**3 of 4 real**	**Proven working — I ran it.** The SMTP connector authenticated against a real mail server and delivered real messages. Email does not send because a per-tenant switch ships off, held for W6. A decision, not a gap. The bulk-sender card is offered as live with nothing behind it.
SMS	sms_twilio, coming soon · v1.1	None	A card and two declared operations. No connector, and no Twilio client anywhere. Honestly marked, so this is on plan.
Aclarian	⚠ A partner name we invented, used as test data in our own plan	**Ask first**	Here "Aclarian" is the make-believe finance partner in worked examples for the *they read from us* direction — it is even drawn on the access log as "Aclarian — reads payments and bills". Aclara, the meter vendor, appears nowhere.
Spurse	A payment processor — your word "spruse", from your list of five	None	Not a meter vendor. No card, no connector, nothing.
Invoice Cloud	A payment processor from the same list	None	The decision log says these were "registered NOW". They were not. And a known defect says our model cannot hold a partner of this shape — a schema change, not a settings entry.
GIS / Esri	Three cards: City GIS, map tiles, census lookup	**1 of 3**	City GIS is real and shipped. Esri is only the map picture, hard-coded in the browser. The census lookup dials out from metering, not integrations. The parcel-reconciliation logic is correctly in Runbooks — the one place the split worked end to end.
Neptune	meters_neptune360 — sixteen operations	**Shape only**	The most complete build in the module. No HTTP client is bound — the call that would reach Neptune raises rather than dialling, and the test honestly answers "not ok". Its sixteen operations are also what make its card sixteen rows tall.

The one question only you can answer

When you said "Aclarian", did you mean Aclara, the meter vendor? Here that word is a partner we made up for worked examples. If a fixture name has drifted into the build list, better caught now. If you did mean Aclara, it is an eighth integration and we have nothing.

09 What I ran, with the numbers

A figure counted by reading is a guess wearing a number's clothes. Every number here has its command beside it.

RAN-1 The email connector works — six real messages

PROVEN

I stood up a real SMTP server and drove the shipped connector against it — not the recorded stand-in the test suite uses.

```
Correct password        ok=True   "Sent one test message to alex@brazosbend.example."
Wrong password          ok=False  "The mail server rejected the password."  (535 …AUTHENTICATION FAILED
Address server refuses  ok=False  "The mail server refused that address."   (550 5.1.1 No such user her
Nothing listening       ok=False  "The mail server did not answer."         ([Errno 111] Connection ref

Batch of 5 notices      5 of 5 delivered, per-item results, confirmed=True
MESSAGES THE SERVER ACTUALLY RECEIVED: 6
```

So "can we connect and test an integration?" — yes. Eight of eighteen cards have a connector bound like this one.

RAN-2 Nothing fires a connection on its schedule

BLOCKS LIVE USE

A utility can choose nightly on a connection. It is accepted, saved and shown. Nothing will ever act on it. I booted the app and asked which job kinds have a body:

```
integration.test            body=run_test_job          queue=integrations
integration.run             body=run_run_job           queue=integrations
```

```
integration.sweep_due        body= -- NONE --          queue=integrations
integration.sweep_expiry     body=run_sweep_expiry_job  queue=integrations
integration.inbound_process  body= -- NONE --          queue=integrations

Runbooks' repeating schedule, in full:
  runbooks.schedule_sweep — every 60s — sweeps runbooks, never connections
```

The cause is clean. The connection scheduler was going to be built in integrations; on 6 September you moved scheduling to Runbooks and that work — two people-weeks — was correctly cancelled here. Runbooks then built a scheduler that schedules runbooks and does not know connections exist. The two people-weeks left one module and never arrived at the other.

This is also why the RBS-004 Overview tab always says "Next — Not set". The screen is telling the truth.

> next_run_at is never once assigned a value anywhere in the backend

> Nothing enqueues sweep_due — full grep, no truncation

> The only trigger="scheduled" runs in the product are demo fixtures

RAN-3 Nothing tells a runbook that a batch landed

BLOCKS LIVE USE

Runbooks knows three events — a batch landed, feedback landed, a bill run finished. The first two are integrations' to announce. Neither is announced by anything; there is no event bus. All 31 occurrences in app code are declarations, or comments saying so.

Six runbooks name one as their trigger — bank reconciliation, cash application, GIS reconciliation, leak alerts, returned payments, undeliverable contacts. Every one falls back to being started by hand. Seven files record it independently; nobody has counted it as one finding.

RAN-4 The seven integration journeys have never been executed

STANDS

You asked whether I ran the journeys. They cannot be run — they are written documents with no test behind them.

```
J48 connect day                  spec=NONE   NOT ACTIVATED
J49 wrong key, Friday            spec=NONE   NOT ACTIVATED
J51 the same bank file twice     spec=NONE   NOT ACTIVATED
J54 the partner's morning        spec=NONE   NOT ACTIVATED
```

```
J55 backfill Tuesday                spec=NONE   NOT ACTIVATED
J56 the Thursday it went quiet      spec=NONE   NOT ACTIVATED
J57 the size that unblocked a bill  spec=NONE   NOT ACTIVATED

The repo's own wiring index: 349 journeys, 53 activated
```

There are browser walkthrough scripts, and they are serious — the connect-day one stands up a real mail server and asserts on what it received. They need PostgreSQL, a front-end build and a browser, none of which exist here.

---

10 Every finding, routed

A finding that is not routed is a finding that is lost. Nothing here has been filed — this is analysis only, so these are proposals with their destination named.

No id is cited because none has been allocated — citing an id before the allocator answers is its own defect class.

FINDING	ROUTE	WHY THAT ROUTE
A connection's cadence never fires	**Defect**	The product does not do what it promised. Ambiguous between defect and change → defect, by rule.
batch.landed emitted by nothing	**Defect**	Six runbooks name a trigger that cannot fire.
Work list unreachable behind a disabled button	**Defect**	A live screen behind a dead control.
Run ▶ enabled, refuses on click	**Defect**	Worse than a disabled control — it takes the click first.
Access log cannot tell loading from empty from failed	**Defect**	A security screen that reports "all clear" when it failed.
No sorting in eight runbooks lists	**Defect**	Against UI-RULES 13; invisible until somebody wants another order.
Mappings: rounded pills, no pager, one empty string for two states	**Defect**	Three separate rule breaches on one screen.
Settings tabs vs. the specified left nav	**Defect**	Two signed sources disagree; ambiguous → defect, likely closed by amending the

		document.
Bulk email, print vendor, census geocoder offered as live with no connector	**Defect**	A card a utility can press that cannot work.
Decision log records five processors as "registered NOW"; none exists	**Defect**	A record of work never done.
A sixteen-operation card in a three-across grid	**Change request**	The product does what it promised; the promise did not anticipate Neptune.
Census geocoder dials out from apps/metering	Already filed	DEF-INT-4251. Not re-filed.
Where the value mappings live	Open question	OQ-INT-1727, already open. Do not re-ask.
"Aclarian" — fixture name or real vendor?	**To the PO**	A fact only you hold.
Invoice Cloud / Spurse — now or with the payments wave?	**To the PO**	Priority is yours; the schema change it implies is ours to size.

11 Stated absences — and whether each is still true

Every one was true when written. A share is false now, because landing a screen and releasing the control that pointed at it are two acts and only the first feels like the work.

WHAT THE PRODUCT SAYS	WHERE	STILL TRUE?
"Not built yet."	Runbooks home → Work list	**FALSE** — the screen is live
"Runs cannot start yet."	Runbooks home, run CTAs	True — nothing is live-enabled
"Stopping is not ready."	In-flight banner	True
"Not connected yet."	RBS-004 Run ▶	**True, but said too late**
"Guards cannot be switched off here."	RBS-004 Guards tab	True, permanently, by rule
"Starting a run arrives with the acting half."	Run result	True

"No screen fixes this"	Work list row action	**Depends on data — unmeasured**
"Not ready to connect yet."	Cards with no adapter	True for 7 cards
"Coming soon · v2 / · v1.1"	Itron/Sensus, Twilio, card, SSO	True
"This one cannot be tested until first use."	Connect modal	True — an honest gap
`integrations.hold.signInNotSetUp`	Registered as a hold; no English string, rendered by nothing	**Dead copy**
`integrations.disconnect.typeName`	Exists in all catalogues; the kit uses confirm.typeToConfirm instead	**Dead copy**
"apps/integrations holds three files"	The email seam's docstring	**FALSE** — 23,050 lines, 80+ files

> **The stale one that misleads a builder**
>
> The file that routes email, SMS and print states that "apps/integrations contains three docstrings and no models, no services and no selectors" and uses that as the stated reason for where a function was placed. It was true when written. A lane following it today puts code in the wrong place.

12 What I deliberately did not file

A judgment not to file something is a finding. Six of them, each with its reason.

WHAT I SAW	WHY I LEFT IT
The grid draws six category chips; the product has ten	The layout document already decided the chips are "generated from the registry's categories, never a typed list". Ten is correct; six was the stale illustration. A figure in a wireframe is an illustration.
RBS-001 draws "Showing 1–5 of 34" beside "Page 1 of 1"	Two sample figures contradicting each other inside one drawing. Same rule — build the shape they agree on, ignore both numbers.
S145 and S146 are drawn and unbuilt	The layout document marks both "shape only · wording not settled". An honest deferral.
S142 shows customer addresses whole where frame 51.1 masks them	Already stamped as a deliberate deviation with its reason: the rows are the tenant's own customers and "a clerk cannot go and fix j.doe@•••".

The pack's chrome — mono header bars, PROPOSED badges, a category switch	The pack says itself these are the drawing's own furniture and not the product's. Not a missing feature.
Three things I am not certain about	The stale "Not ready to connect" holds, the second button on the SMTP card, and how often "No screen fixes this" fires — all read at a version newer than your screenshot, and the last needs real data. A register holding only certainties is one somebody is tidying.

---

13 What step 2 should do first, and why that order

1. **Decide who fires a connection's schedule**
   Everything unattended is blocked on this. Until it is answered, every fetch, delivery and collection is a person pressing a button — and the Overview tab will go on saying "Next — Not set" for ever.
2. **Emit batch.landed and feedback.landed**
   The other half of the same handover. It unblocks bank reconciliation and cash application together, and it is the difference between a runbook module and a runbook screen.
3. **Release the work-list door; fix the access log's three states; fix the stale seam docstring**
   All three are hours, not weeks. One hides a finished screen, one makes a security screen lie by omission, and one is actively misdirecting the next builder.
4. **One sweep of the UI-rule breaches**
   Sorting across the eight runbooks lists, and the mappings screen's pills, pager and empty state. They are the same kind of work and cheaper together than apart.
5. **Answer "Aclarian", then size Invoice Cloud**
   Last because it is the only item that cannot start without you — and because Invoice Cloud needs a schema change, so knowing whether it is wanted now changes the shape of the next two months.

> **And the answer to the question you actually asked**
>
> **Yes — integrations is a connector, and almost purely one.**
>
> Of ~11,000 code lines, the parts on the wrong side are the value mappings, the per-row run results, and one stray outbound call in another module. Everything else — authenticating, holding keys, testing, switching a vendor on and off, answering the door, rate-limiting a partner — is connector work sitting where it belongs. The boundary is not your problem.

What crosses it is.

S Specification register

One row per commitment, no exceptions. This is the register that separates a description from a plan.

COMMITMENT	CLOSES	OWNER	STATUS
The register is empty, and that is the finding	—	—	—

The register is empty, and that is the finding

This was a measuring pass. Nothing in this document has been undertaken by anybody — not the scheduler, not the events, not the work-list door, not one drawing in this pack.

Read this section before quoting any other

Every drawing in this pack is a description. It is a thing a builder could work from and a reviewer could disagree with — it is not a thing anybody has agreed to build. If it is not in a card here, nobody has committed to it. The next step is yours: pick from section 13 and the rows there become commitments.

T What this pass changed

Line by line against the previous version, so nothing is quietly different.

CHANGE	WHY
Every API row shows its rate limit	Your ruling: held in the database, display only, the same standing as the API's own name. 78 rows across all 56 connector screens. The column

— CR-11	means opposite things on the two sides of the line, so each screen states which way it points: outward it is what they allow us, inward what we allow them. Nothing stores it today and every figure drawn is an illustration under B14 rule 37 — the ceiling cannot live on the manifest, because a quota is bought per contract and a manifest is the same for every utility.
A nomenclature section — NM — for the forty operations we own	At your direction, and recorded rather than left in chat. Four artefacts: the operation key is `<runner>_<noun>` so the verb names which of the four runners executes it, the clerk label is three words or fewer, the partner endpoint is `/partner/v1/<plural noun>`, and an outbound payload is `<noun>.v<n>`. Eight partner endpoints are listed in full, plus the one exception that is a browser redirect rather than a machine call. Your question about the tenant is answered there in full: key, then connection, then tenant, three hops all on the server, with what that costs and the escape hatch if a vendor insists on their own base URL.
One token per connection on the thirteen We send connectors — CR-10	Your ruling: generated once, used by every API on the connector, which is the shape the key already had. Eight get the exchange fields and a Regenerate token action, two are already OAuth and now show the token's state, one is the sandbox. **Two cannot have one** and say so: SMTP authenticates with a username and a password, and a file drop carries no token. Eight cards moved from Needs keys to Needs a token.
Every API row carries its provenance, and seven invented hostnames became stand-ins	At your direction. 47 rows on the 13 connectors that do not ship today are marked PROPOSED and draw `api.vendor.example`; the 31 rows on the 15 that do ship keep their own host and carry no mark. A plausible hostname reads as a fact, and `api.sap.example` claimed knowledge of SAP's API that nobody has. The real endpoints arrive with the manifest in the database, which is where they belong.
Every direction tag on all 56 connector screens re-read from its operations — CR-9	At your direction. Two more were wrong in opposite ways: Settlement report on the five processors is a file *we fetch*, not one we push, and Account lookup on the kiosk is them reading from us rather than sending to us. Nine of the 28 span two doors now, against three before. The tag is computed from the operation list rather than declared beside it, so a connector and its own APIs can no longer disagree.
The document is named Integration Blueprint — in the tab, in the gallery and on the page	At your direction. It was The Four Doors in all three. The phrase is not lost: section 02 still carries it as its own heading, which is where it belongs, because it names one finding rather than the whole pass. The standfirst beneath the headline is unchanged.
CON-01 carries the whole toolbar — search · chips · + Filter ▾ · ▦ Columns	At your direction. The screen shipped with two of UI-RULES 12's four parts. The two added are cheap: every dimension + Filter ▾ opens — direction, state, environment — is already in the grid payload, and ▦ Columns is per-viewer and asks the server nothing. On a card grid the column selector chooses which facts each card shows, which is the one control that makes Neptune's sixteen-line card scannable beside four-line neighbours (S8-1).
A connector that goes both ways carries two direction words, never a fifth	Your question. BR-INT-028 puts direction and initiative on the operation, so a connector is the union of its calls rather than a member of one quadrant. **Both ways** would be one word standing where two facts were. Three of the 28 span two doors — bulk email, Twilio, the sandbox processor — and the doors table stays four rows, because those rows are runners and a vendor is not a door.
Three API rows moved from We send to They send — CR-8	Delivery feedback, Delivery status and Payment confirmed each read *We send* beside a trigger of *They call us*, which is one row disagreeing with itself. A webhook the vendor posts to us is in · them. Corrected on six screens; the five Settlement report rows are left alone and named, because that one is a judgment for whoever writes the manifest.
The runbooks section is redrawn on four tabs — Runbooks · Templates · Work list · Run history	At your direction, and it is a CR: the pack draws these screens with buttons and back-links, so tabs are a change to a signed drawing rather than a fix. RBS-001 is removed and its four counters move onto the list. Template detail is added — routed and live, and I had never drawn it. The run wizard gets an address, /runbooks/:key/run. The strip is the one Settings already ships (B14 rule 36), so no new pattern.
The Settings title dropped from 40px to 32px	My own breach, found while building the tabs. B14 rule 40 says the page-head title is 32px on every screen; CON-01 was the only drawing in the pack using the 40px variant.

40px to 32px	pack using the 40px variant.
Runbooks moved to their own section — RB, eight screens	At your direction. A connector holds a connection and a key; a runbook holds the business logic. Mixing them in one section made the line you ruled hard to see.
The set-up form stopped asking the clerk when an API runs — 21 forms	Whether an API is scheduled is the vendor's, declared on the manifest. My form offered Nightly / Hourly / Manual, which is a choice the model does not have and which contradicts BR-INT-037. It now states the fact and offers a time. The other 7 forms have no clock at all — they call us.
Every API row now shows its real trigger — 156 rows across 56 screens	My generator stamped Nightly on every we send connector, so InvoiceCloud's Charge a card read Nightly — a card charge happens when a customer pays. Read from the manifest now: On demand · While you wait · When queued · They call us · Nightly.
MAP-01 opens mapped, not blank	14 of 14 mapped, shipped with the connector. It is a review screen now, not a form. Our seven tenders are seeded identically into every tenant, so ACHCHECK → Bank transfer is a fact about InvoiceCloud, not a utility's opinion — CR-6.
MAP-02 added — the mappings a clerk must genuinely answer	Drawn on an ERP connector, for GL account codes. With routes, these are the only two vocabularies that are the utility's own data, so no default can ever exist for them. The unmapped state belongs here and nowhere else.
The grid says “Needs keys”, not “Not set up” — 15 cards	Under the confirmed model there is no set-up to do — the connector is configured, the APIs are the vendor's, the words are mapped. Only the keys are the utility's. Not set up was false.
Time zone shows as inherited	And the defect behind it is filed: the tenant holds an IANA name, add_template ignores it, so a job set for 01:00 fires at 7pm the evening before — CR-7.
The run wizard drawn frame by frame — all eight	At your direction. Copy is verbatim from the build sheets, which are derived from the pack's own drawings (B14 rule 42 — build from the drawing as an image; I read the frames). Six steps plus two refusal states: the cap being hit, and a step failing mid-run. Drawn on Collections ladder, which is what the pack draws them on.
The path lost a step: five became four	Your catch. Browse and Open were the same step — browsing happens on the template list, and the card's own + Add means you need never open a template at all. Templates is now one step covering both.
Add and Set up drawn as screens — RBS-002.3 and RBS-004 ▸ Just added	At your direction. Step 4 is a moment, not a route — the product posts and redirects in one breath — so it is drawn as what the call creates. Step 5 is the set-up screen on day one: v1, never run, nothing scheduled, trigger Manually. A builder given only the settled state builds the wrong empty one.
All seven setup sub-tabs drawn — Overview · Steps · Scope · Guards · Limits · Runs · Activity	At your direction. Five of the seven had never been drawn. A sub-tab has no address of its own, which is why nothing checks one and why they went missing. Five are drawn on Can we bill? (the runbook the seed script populates, so Runs and Activity hold real rows); Guards and Limits on Collections ladder, because a Check has no guards and never waits for approval.
The Templates tab now shows the path — Templates → Review → Add → Set up, on all four of its screens	At your direction: the flow was not visible. The strip is drawn on all four screens in the path. The template detail became a review screen — every step with what it looks at and whether it writes, the inputs with their starting values, the guards, and what it needs — then one add bar that says what happens next: "It is live at once. You set its schedule next."

Reviewing a template step by step is named as not built	The add is one live call. The shipped detail page lists step names only — no Looks at, no Writes, no guards — although the endpoint already serves every one of those columns.
The runbook panels rebuilt on the kit's Panel — they were hand-rolled boxes	Your catch, and it is B14 rule 36 (compose the kit, don't fork it) — the same rule S8-3 files against the product's own connector cards. 8 titled blocks and 4 list boxes became Panel, and the template card is now the connector card one-for-one: kicker · title · one sentence · state · meta · buttons.
A template card now says what it looks at, and whether it is already added	The old card had a stats line where the description belongs and a floating pill. Every value here is served by the endpoint today — unit_of_work, writes_nothing, and state: added | not_added, which nothing was drawing. Also "1 guards" → "1 guard", and two raw True/False values became Yes and No.
The schedule form and Run ▶ are drawn on RBS-004	They are the two missing doors, and they are on the same screen. Everything else on the four tabs already works. The scheduler behind them is finished — both job handlers are bound and the sweep task is registered, probed at runtime.
The templates count corrected: 35 → 28	Counted from the registry, not from my drawing: 15 Checks and 13 Actions across 13 categories. My earlier 35 was invented.
Every duplicate screen removed — 77 drawings became 68	At your direction, and for a build reason: this pack is going to Claude Code. Two pictures of one screen is how a builder builds the wrong one. The as-built connectors grid, the connect modal, the generic connection detail, the old mappings screen, the old keys screen and the old access-log panel are gone; the revised drawing is the only one left.
The remedy section is gone — its drawings are now the screens	A drawing labelled remedy sits next to the thing it remedies, which is exactly the duplication above. R1→MAP-01, R2→ACC-01, R3→RBS-001, R6→KEY-01. R4 and R5 were dropped outright — the grid already draws with Panel, and R5 duplicated VIEW-15.
What ships today is still recorded — in words, not in a second picture	Every removed drawing keeps its finding, its file and its line number. A builder gets one picture; a reviewer asking *what does it do now* gets a sentence and a citation.
Five screens added — template library, my runbooks, run history, run result, the run wizard	R4/R5/R6: every screen, every flow, every step. The previous pass drew 8 of 14.
The run wizard is drawn and named unreachable	R6 counts a wizard step as a screen. It has four — Scope, Checking, Review, Done — and no route.
Remedy drawings added, with numbered callouts and carry-over arithmetic	Step 8 was missing entirely. Every carry-over reconciles and removes nothing.
A specification register added, and it is empty	Without it a reader cannot tell a description from a plan — which is the failure the register exists to prevent.
S8-3 added — the provider cards fork the kit's Panel	Found by your question about the accent rule. It is a real B14 rule 36 breach and I had not looked.

Integrations renamed to Connectors, and the list grown from 18 to 28 named vendors	Your change request. Recorded in section CR with what it reverses, what it breaks, and what it costs — the as-built grid is kept beside the proposed one rather than overwritten.
Fifty-seven connector screens — the new grid, then every connector twice	One set-up form and one connection view per vendor, in section 03 as you asked.
Thirty-six provider screens added inside section 03 — a set-up form and a connection view for each of the eighteen	You asked for them in section 03 rather than a section of their own, and for a view screen per connection. Every field and operation is read from the manifest.
The S8 cards now draw with the kit's accent Panel	At your direction. The product still hand-rolls this card — S8-3 is the finding; CON-01 is the drawing.
Field tables added for the four integrations screens, with ⇄ on every cross-module fact	Step 6: a drawing with no field table is half a drawing. They immediately produced three findings reading the components had not.
S141, the connect modal, drawn	It was missing entirely, and it is where a utility picks the cadence nothing fires.
A side navigation added	You asked. The document is now 20 sections and 15 drawings; it needed one.
Section order deviates from the skill's template — remedies and registers sit after the findings rather than between them	Reported rather than repaired. Re-sorting would change what every anchor above points at.

Still missing, and named rather than hidden

No screen was clicked, so Step 5 produced no journey rows at all — every finding here is reasoned from source, not seen. Field tables cover the four integrations screens only; the ten runbooks screens have none, because their fields are largely the frozen report payload and tracing that is a pass of its own. And the runbooks module has no cross-module field audit — its ports declare thirteen writers and none is bound, so there is nothing yet to trace.

OP

Many APIs on one connector

A vendor is rarely one endpoint. The product already models this, and the unit is the operation — not the connector. Every set-up form and every view screen in section 03 now carries the API list — one row per API with its endpoint, its direction, its schedule, its own Test and, on the view screens, its own last run and result. Naming and adding an API is ours at onboarding, not a clerk control (ruled), which is why the list is shown and not edited — and why this costs no schema change at all (CR-3). Here is what actually varies, read from the registry.

Three rulings, 14 Sep 2026 — these decided the drawings above

QUESTION	RULED	WHAT IT CHANGED	COSTS A SCHEMA CHANGE?
Can a clerk rename an API we declared?	**No.** "no clark will not do it — we will do it during initial configuration"	The set-up form lists APIs and does not edit them. The disabled + Add an API control is removed.	**No** — the name stays on the manifest
Are mappings per connector or per API?	**Per connector.**	MAP-01 says so on the screen: "These words are shared by all 3 APIs on this connector." No split.	**No** — this is what ships today
Should an issued key name the APIs it may call?	**Yes.**	KEY-01 draws it; six view screens gain a May call column.	**Yes** — a scope column and a check at the door

Two of the three cost nothing but drawing. The one that costs a migration is the one with a security shape, which is the right way round.

> **Already solved, and proven in shipped code**
>
> A connector declares a list of operations, and twenty-three things vary per operation — direction, initiative, transport, trigger, idempotency key, completion style, codec, executor, latency budget, fetch plan, poll schedule, safety parameters, and **host_field**, which is the one that answers your question directly: each operation names which configured host it calls.
>
> Two live examples, neither hypothetical:

- **Esri ArcGIS runs two different hosts today.**

  Its map-tile operation carries host_field="tile_url" while the rest use base_url — one connector, two APIs, two addresses.
- **Neptune 360 runs sixteen operations and two call patterns.**

  Thirteen scheduled, three manual; fourteen answer immediately, two are submit-poll-collect — ask them to build a file, wait, then collect it.

And BR-INT-028 puts direction and initiative on the operation, so one connector can sit in two quadrants at once: an email provider is *we send* for the messages we hand it and *they send* for the bounce it reports back. Never classify a vendor; classify a call.

The real limit is the credential, not the API

Many APIs sharing one credential: solved. Many APIs needing different credentials: not. Manifest.auth is a single Auth, and the database enforces one active credential per connection. So a vendor whose read API uses an API key and whose payment API uses OAuth cannot be one connector today.

⚠

And the token ruling of 16 September makes this bite harder, not softer.

A connector that holds a client secret AND a generated token is holding two live credentials, and the table allows one — so the thing that blocked five payment processors now stands in front of thirteen connectors.

That is DEF-INT-4118, and it is the thing blocking the five payment processors — each holds our key and has an operation we start. It is why the schema is the first item in section 13 and the names are the cheap half.

Three ways out, and the choice is a design decision rather than a guess: make auth a tuple and let an operation name which one it uses (biggest, most general) · allow a second credential row per connection with a role (smaller, fits the existing table) · or split the vendor into two connectors and accept two cards for one logo (no code at all, and it is what gis_city already does in spirit). None is chosen here.

NM Nomenclature — what we call our own APIs

Forty of the seventy-eight operations drawn here are ours to define, because data leaving us is in our shape. A name is the cheapest thing to get wrong and the most expensive to change, so it is settled before anything is built rather than after five vendors are live on it.

Four artefacts, and the first one is the useful idea

The verb is the runner that executes it, so an operation key tells you which of the four doors it goes through before you open a single file.

ARTEFACT	SHAPE	EXAMPLES
Operation key, on the manifest	`<runner>_<noun>`	`deliver_message · fetch_reading · inbound_payment · serve_bill`
Clerk label, on screen	verb first, three words or fewer	Send messages · Collect readings · Read bills
Partner endpoint, for the two doors where they dial us	`/partner/v1/<plural noun>`	`GET /partner/v1/bills · POST /partner/v1/payments`
Payload contract, for the two doors where we dial out	`<noun>.v<n>`	`message.v1 · journal.v1 · charge.v1`

The four verbs are the four runners, which already exist

VERB	DOOR	RUNNER IN THE REPO	READS AS
`fetch`	We collect · in · us	`runners/fetch.py`	we dial them, data comes in
`deliver`	We send · out · us	`runners/deliver.py`	we dial them, data goes out
`inbound`	They send · in · them	`inbound.py`	they dial us, data comes in
`serve`	They read · out · them	`runners/serve.py`	they dial us, data goes out

Singular in a key, plural in a URL. A key names one record — `serve_bill` — because that is what a step handles. A path names a collection — `/partner/v1/bills` —

because that is what a partner asks for.

Worked, against the twenty-eight

OPERATION KEY	CLERK LABEL	DOOR	ON WHICH CONNECTORS
`deliver_message`	Send messages	We send	Microsoft 365 · Google Workspace · Custom SMTP · Bulk email
`deliver_text`	Send texts	We send	Twilio
`deliver_charge`	Charge a card	We send	the five processors
`deliver_refund`	Refund a payment	We send	the five processors
`deliver_journal`	Journal export	We send	Accounting export file
`deliver_print_batch`	Print hand-over	We send	Print vendor
`fetch_reading`	Meter readings	We collect	Neptune · Badger · Itron · Sensus
`fetch_settlement`	Settlement report	We collect	the five processors
`fetch_statement`	Bank statement	We collect	Bank & lockbox file
`fetch_parcel`	Reconcile parcels	We collect	Esri ArcGIS
`inbound_payment`	Payments	They send	Payment kiosk network
`inbound_payment_return`	Returns	They send	Payment kiosk network
`inbound_delivery_receipt`	Delivery feedback	They send	Bulk email · Twilio
`serve_payment`	Read payments	They read	the five ERPs
`serve_bill`	Read bills	They read	the five ERPs
`serve_customer`	Read customers	They read	the five ERPs
`serve_account`	Account lookup	They read	Payment kiosk network

The partner surface, in full — everything a third party can call

Eight endpoints, and they are the whole of what we publish outward today. One base URL, the same for every partner.

METHOD AND PATH	DOOR	WHO CALLS IT	OPERATION KEY
`GET /partner/v1/payments`	They read	the five ERPs	`serve_payment`
`GET /partner/v1/bills`	They read	the five ERPs	`serve_bill`
`GET /partner/v1/customers`	They read	the five ERPs	`serve_customer`
`GET /partner/v1/accounts/{account_no}`	They read	the kiosk network	`serve_account`
`POST /partner/v1/payments`	They send	the kiosk network	`inbound_payment`
`POST /partner/v1/payment-returns`	They send	the kiosk network	`inbound_payment_return`
`POST /partner/v1/delivery-receipts`	They send	Bulk email · Twilio	`inbound_delivery_receipt`
`/auth/saml/acs` — the one exception	They send	the identity provider	`inbound_sign_in`

> **Why the kiosk is the one that proves the scheme**
>
> It is the only connector using both inbound doors, and the same resource noun carries both: `POST /partner/v1/payments` is them sending a payment, `GET /partner/v1/accounts/{account_no}` is them reading what is owed. The method and the door agree, every time, which is the property that makes a scheme worth having rather than merely tidy.
>
> And sign-in is deliberately outside `/partner`. It is a browser redirect carrying an assertion, not a machine calling an API with a key, so putting it on the partner surface would be a shape it does not have.

Rule 1 · the tenant never appears in a partner URL

You asked how a call finds the right utility if the path does not say. **Three hops, all on the server, none of them from the request:** the partner presents a key, the key belongs to one connection, and the connection belongs to one tenant.

```
key  ⟶  connection  ⟶  tenant  ⟶  app.tenant_id set  ⟶  RLS does the isolatin
```

WHY NOT PUT IT IN THE PATH	THE REASON
A path segment is a claim; a key is proof	If both exist, somebody must compare them on every endpoint. That is a line of code on every handler, and the day it is missed a partner reads another utility's bills. Deriving it means there is nothing to check, because there is nothing that can disagree.
Hard rule 1 already works this way	Resolve the key, set `app.tenant_id` from the connection, then query, and RLS FORCE does the rest. A tenant in the URL tempts somebody to set `app.tenant_id` from user input, which is the one thing that rule exists to prevent.
B14 rule 31	No database id is ever a visible value. A tenant id published to a third party is permanent the moment they write it into their config.
A vendor serving two utilities	holds **two keys** and makes two calls. Two connections, two keys, two tenants, no shared session. That is the correct model rather than a limitation of it.

What it costs, said plainly

A support person reading `GET /partner/v1/bills` in a log line cannot tell which utility it was without resolving the key. So the log line carries the connection, and the access log already draws it — ACC-01 is that screen. The URL is not the place that answers it.

And if a vendor insists on a distinct base URL per client

, the escape hatch is an opaque per-connection handle — `/partner/v1/c/{handle}/bills` — random, never the tenant id, and *still* checked against the key. The handle is a convenience for their config; the key remains the only thing that decides anything.

This is what makes CR-4 load-bearing rather than tidy

If the key is the tenant selector, a leaked key is a leaked tenant. Today one key opens every API on a connector, so a key handed over to push meter readings would also read customers. The scope column drawn on KEY-01 is the difference between a leaked key costing one operation and costing all of them.

Rule 2 · nouns are our vocabulary, never the vendor's

`ACHCHECK` never reaches a key, a label or a path. It is a mapping value and it stays on

the mappings screen, where MAP-01 turns it into **Bank transfer**. The same holds for a Nacha return code, a kiosk tender word and an ERP's account string: the vendor's language is data, ours is structure, and the mapping table is the one place they are allowed to meet.

B14 rule 34 already settles one pair of words for the whole product — **Recorded** and **In the books**, in all eleven catalogues — and it is a rule for exactly this reason: two names for one thing drift apart in wording while staying identical in the data.

CR The change: Integrations → Connectors, 18 → 28

A change request is read the moment a change arrives, and step 1 is to classify it. This is a change, not a defect — the product does what it promised; the promise is moving. Below is what it reverses, what it touches, and what is already built and now wrong.

> **What signed statement is being reversed**
>
> The manifests choose generic names deliberately — meters_neptune360 carries display_name="Water meter cloud" with a note that the vendor name is not the display name. That was a decision; this reverses it. The reasoning behind it should be recovered before the rename lands, because whoever wrote it had one.

> **The word changes on screen; the module does not**
>
> Renaming what a clerk reads is not renaming the code. The module stays apps/integrations, the route prefix and the settings tab id stay integrations unless you want a redirect, and the 319 i18n keys under integrations.* stay where they are. Moving those is a separate, larger change that touches eleven locale catalogues — and the rule ids (BR-INT-054, OQ-INT-1727) never move at all.

28 connectors. Ships today = a manifest exists at 2.261.3 and its name does not change.

CATEGORY	CONNECTOR	KEY	STATUS	SHAPE
Email	Microsoft 365	`email_microsoft`	**Ships today**	2 fields · 1 ops
	Google Workspace	`email_google`	**Ships today**	2 fields · 1 ops
	Custom SMTP	`email_smtp`	**Ships today**	5 fields · 1 ops
	Bulk email service	`email_bulk`	Needs a name	3 fields · 2 ops
Messaging	Twilio	`sms_twilio`	**Ships today**	3 fields · 2 ops
Meters	Neptune 360	`meters_neptune360`	**Renamed**	2 fields · 16 ops
	Badger BEACON	`meters_badger`	**New**	3 fields · 4 ops
	Itron	`meters_itron`	**New · split out**	3 fields · 4 ops
	Sensus	`meters_sensus`	**New · split out**	3 fields · 4 ops
Payments	InvoiceCloud	`pay_invoicecloud`	**New**	4 fields · 4 ops
	Paymentus	`pay_paymentus`	**New**	4 fields · 4 ops
	US Payments	`pay_uspayments`	**New**	4 fields · 4 ops
	WorldPay	`pay_worldpay`	**New**	4 fields · 4 ops
	Global Payments	`pay_globalpayments`	**New**	4 fields · 4 ops
	Payment kiosk network	`payments_kiosk`	**Ships today**	2 fields · 3 ops
	Test payments (sandbox)	`payments_sandbox`	**Ships today**	1 fields · 3 ops
ERP	Aclarian	`erp_aclarian`	**New**	4 fields · 3 ops
	Workday	`erp_workday`	**New**	4 fields · 3 ops
	SAP	`erp_sap`	**New**	4 fields · 3 ops
	Infor	`erp_infor`	**New**	4 fields · 3 ops

	Microsoft Dynamics GP	`erp_msgp`	**New**	4 fields · 3 ops
Accounting	Accounting export file	`accounting_file`	**Ships today**	7 fields · 1 ops
Banking	Bank & lockbox file	`bank_lockbox_file`	**Ships today**	8 fields · 2 ops
Print	Print vendor	`print_vendor`	Needs a name	2 fields · 1 ops
Maps	Esri ArcGIS	`gis_esri`	**Renamed**	3 fields · 3 ops
	Census geocoder	`geocoder_census`	**Ships today**	1 fields · 1 ops
	Public map tiles	`map_tiles_public`	**Ships today**	1 fields · 1 ops
Sign-in	Single sign-on	`sso`	Needs a name	2 fields · 1 ops

What is already built and is now wrong — change process step 4.

BUILT TODAY	WHAT THIS CHANGE DOES TO IT
meters_mdm — one card reading "Itron / Sensus"	Splits into two connectors. One manifest becomes two, and a tenant who connected it has to be migrated to whichever vendor they actually use — the row cannot say which.
payments_card — "Card processor", coming soon W7	Replaced by five named processors. Nothing is connected to it, so nothing migrates — but the wave that was going to build it now has five to build.
gl_partner — "Third-party GL", live, issues keys	Replaced by five named ERPs. Any partner already holding an issued key is on a connector that no longer exists by that name.
meters_neptune360, gis_city	Renamed only. Fields, operations and adapters are untouched — the safest two in the list.
The settings tab, its route, and 319 i18n keys	The visible word changes; the identifiers need not. Decide that separately — it is the difference between a day and a fortnight.
DEF-INT-4118 — the model cannot hold InvoiceCloud	Now blocks five connectors instead of one. A manifest holds a single Auth and the database enforces one active credential per connection; a processor that holds our key and has an operation we start does not fit. **This is the first thing to fix, not the names.**

> **The code behind stays shared — that is your own rule, already signed**
>
> Eleven new connectors do not mean eleven new pieces of code. A connector is a manifest — a data file — and the adapter behind it is shared by shape. PR-INT-037 is the rule, and it is

— a data file — and the adapter behind it is shared by shape. BR-INT-037 is the rule, and it is your own sentence: "every time such integration comes we dont want to change our integration framework."

So the eleven cost roughly two adapters, not eleven:

- **The five ERPs need none at all.**